TEACHME-LAKEHOUSE · TRAINING PROGRAMME

Module 2 — Applied to Tamimi

02

How Module 2's principles were actually implemented here — and what it cost to learn them

18 HOURS | 24 LESSONS | PREREQUISITE: MODULE 1

AUTHOR
Surender Sara

COMPANY
Northbay Solutions

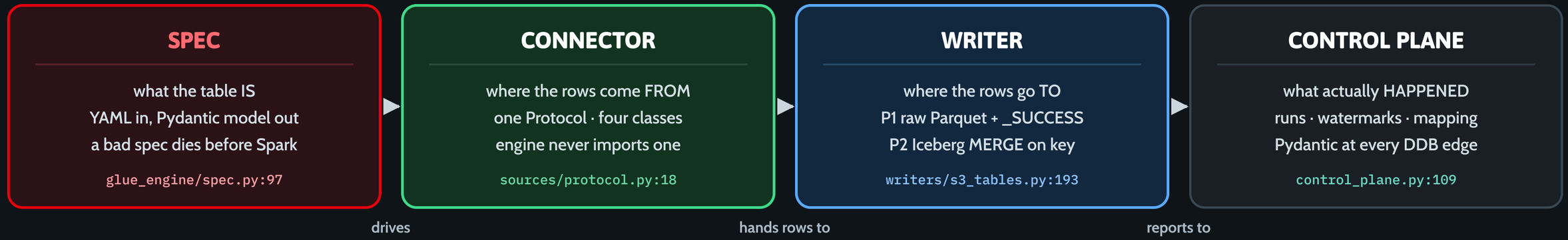
ISSUED
August 2026

MODULE 2 · LESSON 01

Anatomy of the Engine

One wheel, many YAML specs. Four seams — and a job whose only job is to wire them together.

1 · THE FOUR SEAMS



2 · THE JOB IS ONLY WIRING

ONE JOB, FIVE CALLS

`glue_engine/jobs/source_download.py`

`get_bronze_mapping()`	`:133`	which spec, which driver
`parse_bronze_spec()`	`:135`	the shape, validated or refused
`get_connector(source_type)()`	`:228`	the reader, chosen by name
`write_raw(df, ...)`	`:307`	parts first, _SUCCESS marker last
`put_watermark()` · `put_run()`	`:349`	how far we got, and whether it worked

3 · WHY THAT MATTERS

ADDING A TABLE

1 · a connector class — only if the source type is new

2 · a row in DDB bronze_mapping

3 · the YAML spec

No new Python. No new Glue job. `__init__.py:3-9`

IN PLAIN ENGLISH

Four sockets on one wrench. Swap the socket that fits your source — the handle, the ratchet and the logbook never change.

MODULE 2 · LESSON 01

L01 · Anatomy of the Engine

Slide: `_render/L01-engine-anatomy.html`

The point

`glue_engine/` is not a folder of scripts. It is four **seams** — spec, connector, writer, control plane — and a set of jobs whose only work is to wire them together in the right order. Once you can name the seam a failure came from, you know which file to open. Module 1 showed you the map; this is the machine room.

The claim to test all module long: **there is no per-table Python.** 58 P1 download specs and 67 Bronze specs, one wheel, zero per-table classes. Adding the next table is a YAML file and a DynamoDB row.

Key ideas

- **SPEC — what the table *is*.** The YAML in `src/glue/specs/` is parsed into a Pydantic model, and a bad spec is rejected *before* Spark starts: a `merge_key` naming a column that isn't in the schema, or a `pii_class: pii` column used as a merge key, raises at parse time rather than corrupting an upsert three hours later.
- **CONNECTOR — where the rows come *from*.** One `Protocol`, four implementations. The engine never imports a concrete connector; it asks a registry for one by name (L02).
- **WRITER — where the rows go *to*.** Two of them: P1 lands raw Parquet under `cycle=<cycle_id>/` and writes `_SUCCESS` **last**, so a partial cycle never looks finished; P2 MERGEs into an Iceberg table in S3 Tables.
- **CONTROL PLANE — what actually *happened*.** Every read and write of DynamoDB goes through a Pydantic model, in both directions. Reads validate too — that is what catches DDB drift instead of letting a renamed attribute silently read as `None`.
- **The job is the wiring, not the logic.** `source_download.py` resolves the mapping, parses the spec, asks for a connector, calls one of three read methods, writes, then records. Everything interesting is behind one of the four seams.
- **Seams are where you extend, and where you debug.** "Empty landing folder" is a writer/connector question. "Refusing to splice watermark" is a connector question. "Table not admitted to today's cycle" is a control-plane question.
- **Why pluggable at all:** the same wheel has to serve SAP HANA, an RDS SQL Server, a spreadsheet in S3, and — next — whatever Apparel Group runs. Anything table-specific that leaks into the engine is a bug you pay for once per client.

Words you'll hear

TERM	MEANS
Seam	A boundary the engine can swap an implementation across without changing callers
Spec	The per-table YAML: schema, source config, merge key, landing prefix
Connector	The class that knows how to read one *kind* of source
Writer	The component that persists a DataFrame (raw Parquet, or Iceberg MERGE)
Control plane	The DynamoDB tables that record routing, progress and outcome
P1 / P2	Download-to-raw (P1) and load-raw-to-Bronze (P2) — the two halves of ingestion
`_SUCCESS`	The marker written *last*, that makes a landed cycle readable by P2

L01 · Anatomy of the Engine

In this repo

- `src/glue/glue_engine/spec.py:97` — `BronzeSpec`. Read the three `@model_validator`s at `:189`, `:204`, `:226`: each one is a real class of bug that can no longer reach production.
- `src/glue/glue_engine/sources/protocol.py:18` — the whole connector contract, 100 lines including docstrings.
- `src/glue/glue_engine/writers/raw_landing.py:152` — `write_raw`: data files first, marker last, and it propagates *without* a marker on failure.
- `src/glue/glue_engine/writers/s3_tables.py:193` — `merge_into`, the Bronze upsert that makes replays idempotent.
- `src/glue/glue_engine/control_plane.py:109` — `put_run`, and note the `exclude_none=True` comment: a `None` on a GSI key attribute would make DynamoDB reject the write outright.
- `src/glue/glue_engine/jobs/source_download.py:133-366` — the five calls on the slide, in order.
- `src/glue/glue_engine/__init__.py:3-9` — the three-step "add a table" recipe, written by the people who built it.

Do this

Open `jobs/source_download.py` and, without reading the rest of the module, annotate every line between `try:` (`:131`) and the final `put_run` (`:366`) with which of the four seams it touches — spec, connector, writer, or control plane. You should be able to account for every line. Then find the one block that belongs to *none* of them (hint: it is around `:205`, and it is a policy decision) — that is L05.

You've got it when you can…

…be told "`sap.mbew` produced no `_SUCCESS` marker for cycle 2026-08-09" and say which seam you would open first, and why the *absence* of the marker is itself the diagnostic.

MODULE 2 · LESSON 02

The Engine Doesn't Know What SAP Is

It knows five method names. Everything SAP-shaped hides behind them — so the next source is a new class, not a new pipeline.

1 · THE CONTRACT — FIVE METHODS

```
@runtime_checkable
class SourceConnector(Protocol):
```

`configure(source_row, spec)`
creds, endpoint, spec validation — once per run

`read_full(spark, run_id)`
every row in scope. Returns a DataFrame.

`read_incremental(spark, wm, run_id)`
rows since wm. Returns (df, new_wm).

`read_range(spark, from, to, run_id)`
one bounded window. Can't? Raise NotImplementedError.

`emit_metrics()`
rows_read · retries · rows_with_errors → runs.metrics

The connector NEVER writes the watermark — it RETURNS it.

2 · THE REGISTRY

```
_REGISTRY: dict[str, type] = {}
@register("sap_hana")
class SapHanaConnector: ...
```

Import-time side effect. Four classes, four names, one dict:

sap_hana	sources/sap_hana.py:158
rds_jdbc	sources/rds_jdbc.py:78
excel_landing	excel_landing.py:42
s3_landing	landed_files.py:75

Register a name twice and it REFUSES. Ask for a name it doesn't have and the KeyError lists everything it does.

sources/__init__.py:21-51

3 · DRIVER BY MODE

WHICH CLASS, PER MODE

The table's mapping row can pick a different connector per read-mode:

```
driver_by_mode:
  full:        sap_hana
  range:       sap_hana
  incremental: sap_hana
```

No entry? Fall back to the spec's own source_type — that's every non-SAP table, unchanged.

All three read sap_hana since the ODP/CDC path was retired 2026-07-22.

driver_select.py:18-34

THE POINT A new source is a new CLASS, not a new PIPELINE. Oracle for Apparel Group = one file, one @register, one DDB row — zero engine edits.

IN PLAIN ENGLISH

It's a plug socket. The engine holds the socket and knows only its shape; SAP, NCR and a spreadsheet are just different plugs.

L02 · The Engine Doesn't Know What SAP Is ★

Slide: `_render/L02-source-protocol.html`

The point

The engine has no `if source == "sap"` anywhere. It knows five method names and a dictionary. Everything SAP-shaped — the HANA driver jar, the MANDT client filter, NVARCHAR dates, connection budgets — lives inside `SapHanaConnector` and is invisible from outside it.

That is not tidiness. It is the reason **the next client is a new class, not a new pipeline.** When Apparel Group arrives with Oracle, you write one file, decorate it with `@register("oracle_jdbc")`, add a `source_catalog` row, and the dispatcher, the barriers, the writers, the watermark logic and the run bookkeeping all work unchanged.

Key ideas

- **The contract is five methods.** `configure` (creds + validation, once per run), `read_full`, `read_incremental`, `read_range`, `emit_metrics`. That is the entire surface an engine-facing source has.
- **`read_incremental` returns the watermark; it never writes it.** The connector hands back `(df, new_watermark)` and the engine persists it *atomically with the run-success record*. A connector that advanced its own watermark could move the marker for a run that then failed to land.
- **A connector may decline a mode.** `read_range` on a full-snapshot source (`excel_landing`) raises `NotImplementedError` — the contract says so explicitly. Declining is part of the interface, not a bug.
- **The registry is a dict populated by import side effects.** `@register("sap_hana")` on the class puts it in `_REGISTRY`; the four side-effect imports at the bottom of `sources/__init__.py` are what make the registry non-empty. Delete that import block and nothing is registered.
- **Both failure modes of the registry are loud.** Registering a name twice raises immediately ("refusing to overwrite"); asking for an unregistered name raises a `KeyError` that *lists everything registered*, which is the fastest debugging aid in the file.
- **`driver_by_mode` chooses per read-mode, not per source.** A table's mapping row can name a different connector for `full`, `range` and `incremental`. Today every SAP mode resolves to `sap_hana` (the OData/ODP CDC path was retired 2026-07-22) — but the seam is still there, and it costs nothing.
- **No entry means fall back to `spec.source_type`.** That is why every non-SAP table needs no `driver_by_mode` at all.
- **`@runtime_checkable`** means `isinstance(conn, SourceConnector)` works — the tests use it to assert a new connector really satisfies the contract before it ever runs in Glue.

L02 · The Engine Doesn't Know What SAP Is ★

Words you'll hear

TERM	MEANS
`Protocol`	Python structural typing — satisfy the shape, no base class to inherit
Registry	`_REGISTRY: dict[source_type -> class]`, filled by decorators at import time
Side-effect import	Importing a module purely so its `@register` call runs
Read-mode	`full` / `range` / `incremental` — *which* of the connector's read methods runs
`driver_by_mode`	Per-table map from read-mode to `source_type`
`source_catalog`	The DDB table holding endpoint, `secrets_arn`, `network_config` per source
Watermark contract	The connector returns a new value; the engine persists it

In this repo

- `src/glue/glue_engine/sources/protocol.py:18-99` — the contract. Read `:59-61` ("do NOT advance the watermark internally") and `:96-98` (`rows_with_errors > 0` → the engine does not advance the watermark) out loud.
- `src/glue/glue_engine/sources/__init__.py:21-51` — `_REGISTRY`, `register`, `get_connector`, `registered_source_types`. The docstring at `:1-8` is the "how to add a source" recipe.
- `src/glue/glue_engine/sources/__init__.py:57-62` — the four side-effect imports that populate the registry.
- The four registered classes: `sap_hana.py:158`, `rds_jdbc.py:78`, `excel_landing.py:42`, `landed_files.py:75`.
- `src/glue/glue_engine/sources/excel_landing.py:173-180` — a connector legitimately refusing a mode.
- `src/glue/glue_engine/driver_select.py:18-34` — `resolve_driver_source_type`. A pure function with no boto3/pyspark import, so it is unit-testable anywhere.
- `src/glue/glue_engine/jobs/source_download.py:222-233` — read-mode → driver → `get_connector(...)()` → `configure(...)`. Four lines; that is the whole plug-in mechanism at runtime.

Do this

Sketch `OracleJdbcConnector` for an Apparel Group RMS table — just the class skeleton with the five method signatures and a `source_type` attribute — and list, for each method, the *only* thing that differs from `SapHanaConnector`. Then answer: which files elsewhere in `glue_engine/` would you have to edit to make it run? (Correct answer: one import line in `sources/__init__.py`. Nothing else.)

You've got it when you can…

…explain to a sceptical colleague why onboarding Oracle needs no change to `source_download.py`, and point at the exact line where the class is chosen by name — and then say what *would* force an engine change (a source that cannot express any of full / range / incremental).

MODULE 2 · LESSON 03

Partitioning the Giants

One connection cannot read 36 million rows. Sixteen could not connect at all. The lesson is the number in between.

1 · THREE KNOBS — ONLY ONE FIRES ON THE SAP PATH

jdbc_hash_partitions

LIVE

N contiguous date sub-ranges over the watermark column — one JDBC connection each, each a pushed-down BETWEEN.

```
spark.read.jdbc(predicates=[...])
```

sap_hana.py:718 _window_predicates · :1038

partition_column

Spark's own range split: lowerBound / upperBound / numPartitions. The code path exists — no SAP spec sets it.

SAP keys are NVARCHAR, not numeric ranges. See sap_konp.yaml:33-35.

jdbc_hash_field

Validated at :266, stored at :270 — then never read again by sap_hana. It is live on rds_jdbc.py:353-359.

On SAP it documents WHICH PK member you would hash. Know that before you tune.

2 · THE GUARD

REFUSE THE UNPARTITIONED GIANT

```
expected_row_count:       35,999,963
single_partition_max_rows: 40,000,000
above it + no partition_column -> ValueError
```

Default ceiling is 5,000,000. MBEW raises it to 40M, KONP to 400M — each an explicit, commented choice.

sap_hana.py:192, :389-411 · sap_mbew.yaml:51-52

3 · THE INCIDENT — MBEW

16 CONNECTIONS COULD NOT CONNECT

16 simultaneous connects saturated HANA's connect headroom.

```
Cannot connect to host (socket timeout)
```

FIX 1 — connect budget 15s → 60s — sap_hana.py:959-964

FIX 2 — hash_partitions 16 → 4 (≈ 9M each) — sap_mbew.yaml:33-36

LATER — VBRP went 4 → 8 (684M table) — sap_vbrp.yaml:22

THE ARITHMETIC

lanes × partitions

4 × 8 = 32

ceiling ≈ 160 connects

IN PLAIN ENGLISH

Checkout lanes. Too few and the queue never clears; too many and the doors jam. Tune the count against the doorway, not the queue.

L03 · Partitioning the Giants

Slide: `_render/L03-jdbc-at-scale.html`

The point

A single JDBC connection reading a 36-million-row table is one executor, one socket, one very long wait — and eventually a Glue timeout. The obvious fix is "more connections". The obvious fix has its own wall: **MBEW at 16 partitions could not connect at all.**

The number you want is bounded from below by row volume and from above by how many simultaneous connects the source will accept over the network path you have. This lesson is about finding it, and about the guard that stops you shipping an unpartitioned giant by accident.

Key ideas

- **On the SAP path, parallelism means date sub-ranges.** `jdbc_hash_partitions: "8"` splits the window into 8 contiguous `BETWEEN` predicates on the watermark column and passes them to `spark.read.jdbc(predicates=[...])` — one JDBC connection per predicate, each pushed down.
- **`jdbc_hash_field` is inert on `sap_hana`.** It is validated and stored, then never read again by that connector; it *is* live on `rds_jdbc`, which passes `hashfield` / `hashpartitions` to Glue's own options. On SAP it documents which PK member you *would* hash. Know this before you "tune" it.
- **`partition_column` + bounds is a real code path with no SAP callers.** No download spec sets it, and `sap_konp.yaml` records why: `KNUMH` is a non-numeric NVARCHAR key, not a valid Spark range-partition column.
- **The knob was inert once, too.** Until TML-70, `jdbc_hash_partitions` was validated and stored but never wired into the read — a full 5-year initial load ran at "Map × 1 connections total" (observed 2026-07-18). R45 later fixed the same gap on the *delta* path.
- **Unpartitioned giants time out, measurably.** `sap.zhocidc` 's single-connection delta blew through the 120-minute Glue job timeout on **four consecutive attempts**; the daily cycle never reached its `download_barrier` and the retry burned ~10 h of a lane per cycle.
- **The guard: declare your size.** If `expected_row_count > single_partition_max_rows` (default **5,000,000**) and no `partition_column` is configured, `configure()` raises before a single row is read. Declare nothing and you get a warning instead — and that warning text is exactly what the R45 job log said.
- **Connections are a shared budget.** `lanes × jdbc_hash_partitions` is the real number of simultaneous connects. VBRP's spec does the arithmetic in a comment: at Map `MaxConcurrency=4`, 8 partitions = 32 connects, safely under the ~160 ceiling.
- **Timeouts are part of the same fix.** The JDBC URL carries `connectTimeout=60000&communicationTimeout=120000` precisely because a 15-second connect budget over the TGW hop saturates when many partitions open at once.

L03 · Partitioning the Giants

Words you'll hear

TERM	MEANS
Predicate-partitioned read	`spark.read.jdbc(predicates=[...])` — one connection per WHERE clause
`jdbc_hash_partitions`	The count of parallel sub-ranges (a string, per Glue's option contract)
`jdbc_hash_field`	The PK member you'd hash on — live for `rds_jdbc`, documentation for `sap_hana`
`single_partition_max_rows`	The size above which an unpartitioned read is refused
`expected_row_count`	The live-verified row count that lets the guard do its job
Connection saturation	Too many simultaneous connects → "Cannot connect to host (socket timeout)"
TGW hop	Transit Gateway path from the VPC to on-prem SAP — where the latency lives
Lane	One concurrent Step Functions Map branch running a download job

In this repo

- `src/glue/glue_engine/sources/sap_hana.py:718-772` — `_window_predicates`. The docstring records the "Map × 1 connections" observation that motivated it.
- `src/glue/glue_engine/sources/sap_hana.py:1029-1048` — the predicates branch of `_read`; `:1059-1065` is the `partitionColumn` branch nothing currently uses.
- `src/glue/glue_engine/sources/sap_hana.py:952-965` — the JDBC URL, and the comment naming **MBEW at `jdbc_hash_partitions=16`** as the read that produced *"Cannot connect to host (socket timeout)"*.
- `src/glue/glue_engine/sources/sap_hana.py:363-411` — the HIGH-14 guard: the `ValueError` and the fall-back warning.
- `src/glue/specs/download/sap_mbew.yaml:25-52` — **read every comment in this block.** Why `MATNR` is the hash field, `jdbc_hash_partitions: "4"` *(was 16)*, ≈ 9M rows per partition, and why `expected_row_count: 35,999,963` is the client-100 count and not the 37.2M banner figure.
- `src/glue/specs/download/sap_vbrp.yaml:22` — the other direction: 8 partitions, with the `4 × 8 = 32` connect arithmetic spelled out.
- `src/glue/specs/download/sap_konp.yaml:33-47` — a deliberate single-partition read of a 379M-row table, and the raised ceiling that makes it explicit rather than accidental.
- `src/glue/glue_engine/sources/sap_hana.py:800-824` — R45: the four consecutive 120-minute timeouts on `sap.zhocidc`.

Do this

Take `sap_mbew.yaml` and compute, for `jdbc_hash_partitions` of 1, 4, 8 and 16: rows per partition, and simultaneous HANA connects at Map `MaxConcurrency=4`. Then say which of those four numbers fails, and *how* each one fails — they do not fail the same way. Check your answer against the comments in `sap_hana.py:959-964` and `sap_vbrp.yaml:22`.

You've got it when you can…

…read *"Cannot connect to host (socket timeout)"* in a Glue log and immediately ask the right next question — "how many partitions × how many lanes?" — instead of the wrong one, "is the network down?"

MODULE 2 · LESSON 04

The Delta Window

How the WHERE clause is built, why SAP dates are text — and why we would rather re-read a day than store a poisoned watermark.

1 · HOW THE PREDICATE IS BUILT

ONE SELF-CONTAINED SELECT

sap_hana.py:544 _build_query

```
SELECT "MANDT", "WERKS", "BUDAT", "SEQNO", ...
FROM   SAPHANADB.ZHOCIDC
WHERE  MANDT = '100'
  AND  BUDAT > '20260731'
```

1 · projection — the spec's own schema columns, quoted. Never SELECT *.

2 · client filter — without MANDT the SAP test client leaks into facts.

3 · the delta — spliced ONLY if it matches ISO-8601 or all-digits.

2 · SAP DATES ARE TEXT

NVARCHAR(8), NOT DATE

sap_hana.py:644 _fmt_date

FKDAT · BUDAT · ERDAT · ZDATE · SPTAG are all text columns.
The job arg arrives ISO; the column has no dashes at all.

```
watermark_date_format: "YYYYMMDD"   2026-07-01 -> '20260701'
```

Forget it and the BETWEEN matches nothing — a windowed load landed 0 rows, live 2026-07-17.

GUARD — configure() REFUSES a spec whose column list omits the watermark column. F.max() would raise late.

3 · WHY WE REFUSE TO PERSIST A DIRTY WATERMARK

WHAT MAX() SEES

'502812'	not a date
'22080401'	year 2208
'730121V1'	column shift
'S'	sorts last

IF WE STORED IT

Next cycle builds its predicate and dies at the splice guard:

```
refusing to splice watermark 'S'
```

Every future run. Wedged.

WHAT WE DO

MAX over PLAUSIBLE rows only:
8 digits · ≥ 19000101 · ≤ today
Still unspliceable? Keep the old value. Forward-only.

sap_hana.py:1167-1223

NO DATE? FULL REFRESH

27 SAP tables watermark on a constant — 25× MANDT, plus ZINPUT and DOMNAME. They get driver_by_mode {full} instead.

commit cbfa266 · 24/32 tables failed

IN PLAIN ENGLISH

The watermark is your bookmark. Better to re-read yesterday's page than to write the bookmark on a page that doesn't exist.

L04 · The Delta Window

Slide: `_render/L04-watermarks-cdc.html`

The point

A daily delta is one string interpolated into one `WHERE` clause. Everything that can go wrong with incremental ingestion goes wrong *in that string* — because SAP's date columns are not dates, they are `NVARCHAR(8)` text, and text columns hold whatever someone typed.

So the interesting decision in this file is not how we advance the watermark. It is **when we refuse to.** A `MAX()` that returns a status character `'S'` would be stored, then spliced next cycle, then rejected by our own guard — wedging that table forever. We would rather re-read yesterday.

Key ideas

- **`_build_query` composes, it does not template.** Projection (the spec's own columns, quoted — never `SELECT *`), then a predicate list: MANDT client filter, watermark comparison, optional static filter. Missing pieces just don't appear.
- **The MANDT filter is not optional.** SAP application tables are client-partitioned; without `WHERE MANDT = '100'` the test client's rows leak into your facts. `configure()` demands either a digit `client` or an explicit `client_independent: true`.
- **The watermark is validated before it is spliced.** It must match ISO-8601-with-time or an all-digit id. Anything else raises `"refusing to splice watermark ... into SQL"` — an injection defence that doubles as a corruption detector.
- **Empty string is "no watermark yet", not "malformed".** `''` is the control plane's own sentinel; treating it as a value once sent a first-ever run straight into the splice guard.
- **SAP dates are text, so windows need reformatting.** `--date_from 2026-07-01` arrives ISO; `BUDAT` holds `'20260701'`. `watermark_date_format: "YYYYMMDD"` makes `_fmt_date` strip the dashes. Forget it and the `BETWEEN` matches nothing — a windowed load landed **0 rows** on 2026-07-17.
- **Projection guard.** `configure()` refuses a spec whose column list omits the watermark column, because `df.agg(F.max(watermark_column))` would otherwise raise an `AnalysisException` *after* the entire JDBC pull has been paid for.
- **What `MAX()` actually returns on these columns:** `'502812'` (not a date), `'22080401'` (a typo year), `'730121V1'` (a source-side column-shift defect in ZECOM's `ZDATE`), and in the worst case a letter, which sorts *after* every digit. All live-observed.
- **So the max is taken over plausible rows only** — 8 digits, ≥ `19000101`, ≤ today — and if the candidate still would not splice, the **prior watermark is kept**. Garbage rows still land in Bronze; they just cannot become the bookmark.
- **Forward-only.** With `safety_buffer_days` the read window starts *before* the stored watermark, so a hard delete upstream can make the window's `MAX()` come back lower. Persisting that would rewind progress every cycle, so a lower value is refused too.
- **Re-reading is free, wedging is not.** The Bronze MERGE on `merge_key` is idempotent, so keeping the old watermark costs one repeated delta. Storing a bad one costs every future cycle until a human notices.
- **No usable date column → don't run a delta at all.** 27 SAP tables watermark on a constant (25 × `MANDT`, plus `ZINPUT` and `DOMNAME`). They are set to `driver_by_mode: {full: sap_hana}` so the delta path never runs. Before that fix, a live daily fire failed **24 of 32** tables with `refusing to splice watermark ''`, and the all-or-nothing barrier held Bronze for the whole cycle.

L04 · The Delta Window

Words you'll hear

TERM	MEANS
Watermark	The stored high-water value of the watermark column — a bookmark, not a timestamp of the run
Splice	Interpolating a value into SQL text (hence the pattern check before doing it)
`watermark_date_format`	`YYYYMMDD` for SAP's dashless text dates; ISO otherwise
`safety_buffer_days`	Widen the delta's lower bound by N days to catch late-posted rows
Dirty watermark	A `MAX()` that is not a plausible, spliceable date
Wedged	A table whose stored watermark makes every future run fail at the same guard
`_watermark_valid`	The per-row Bronze audit column labelling row-level watermark quality
Forward-only	A new watermark below the stored one is discarded

In this repo

- `src/glue/glue_engine/sources/sap_hana.py:544-592` — `_build_query`. Note the comment at `:570-575` on why *falsy* means absent and only a non-empty unparseable value is corruption.
- `src/glue/glue_engine/sources/sap_hana.py:1117-1231` — `read_incremental` end to end: `.cache()` before the multi-action sequence (`:1160`), the plausible-only aggregate (`:1167-1182`), the refusal to persist (`:1188-1206`), and the forward-only check (`:1207-1223`).
- `src/glue/glue_engine/sources/sap_hana.py:774-798` — `_watermark_to_iso` , and the two garbage values observed live on 2026-07-17.
- `src/glue/glue_engine/sources/sap_hana.py:465-474` — the watermark-column-in-projection guard.
- `src/glue/glue_engine/sources/sap_hana.py:644-664` — `_fmt_date` and `format_watermark_bound` .
- `src/glue/glue_engine/sources/sap_hana.py:594-642` — `_buffered_watermark` : widen the read, never move the stored value.
- `src/glue/glue_engine/sources/sap_hana.py:120-155` — `_watermark_valid_for_row` , the pure-Python reference spec for the native Spark expression at `:1097-1112` , plus the Java-vs-Python `$` difference that made the explicit length check load-bearing.
- **Commit** `cbfa266` — *"fix(ingestion): full-refresh the 27 SAP tables with non-date watermarks"*. The message is the incident report; read it before the diff.
- `CLAUDE.md:101-112` — the JDBC-only policy, and the standing caveat that a creation-date watermark cannot see hard deletes or in-place edits.

Do this

Take a table whose stored watermark is `'S'` . Write down, in order, exactly what happens on the next scheduled run — which function raises, what the `runs` row looks like, and whether the barrier lets the cycle proceed. Then find the two independent places in `sap_hana.py` that would have stopped `'S'` from being stored in the first place, and say which one fires first.

You've got it when you can…

…answer "why didn't the watermark move even though the job succeeded?" with a specific reason — empty delta, unspliceable `MAX()` , or a value behind the stored one — and name the log line each case emits.

MODULE 2 · LESSON 05

Full, Windowed, Backfill

Three read modes chosen by one line — plus the policies that stop a half-loaded table from calling itself done.

1 · MODE SELECTION — THE WHOLE RULE IS ONE LINE

PRECEDENCE

glue_engine/jobs/source_download.py:222

```
read_mode = "range" if date_from else ("full" if full_snapshot else "incremental")
```

RANGE · a bounded window

--date_from beats --full_snapshot.
read_range(); the watermark never advances.

FULL · the whole table

--full_snapshot, or DERIVED when the mapping declares only 'full' (:205-213).

INCREMENTAL · the default

wm_col > stored watermark; returns (df, new_wm). Empty keeps the old value.

2 · THE TWO POLICIES

AN INITIAL LOAD IS FULL

A full-only mapping never reaches the delta path: full_snapshot=True, date_from cleared.

It protects the Bronze parity invariant — Bronze's count MUST equal SAP's, in horizon.

2026-07-16: tvfkt · twewt · twtyt · zzz_mc2t landed EMPTY from a meaningless window.

source_download.py:205-213 · CLAUDE.md:114

AN EMPTY PULL

incremental / backfill → SUCCEED

a quiet window is normal; failing would break every daily cycle.

full snapshot → FAIL

an empty full extract is almost always broken. Refuse to land it.

bronze_pull.py:393 · spec on_empty wins

3 · init_state LIFECYCLE

GATE 0 REMEMBERS

:340-345

(none)	nothing finished yet
initial_loaded	un-windowed initial
cdc	first plain daily delta

A windowed chunk NEVER flips it — its siblings are still landing. The barrier does.

IN PLAIN ENGLISH

Move in before you tidy up. Load the whole house first, then take deliveries daily — and never say "moved in" while the van is still outside.

L05 · Full, Windowed, Backfill

Slide: `_render/L05-three-read-modes.html`

The point

Three read modes, chosen by one line of Python. The line itself is trivial; what surrounds it is not. Two policies and one state machine exist because each of them was learned from a live failure: a table that loaded a meaningless window and landed empty, a full extract that would have overwritten good data with nothing, and a giant that declared itself loaded while most of its history was still landing.

Key ideas

- **Precedence is literal:** `read_mode = "range" if date_from else ("full" if full_snapshot else "incremental")`. A date window beats a full-snapshot flag; the daily delta is what's left.
- **`full` can be *derived*, not just passed.** If the table's mapping declares only `full` (no `incremental`), the job forces `full_snapshot = True` **and clears** `date_from`. A full-refresh table has no real date column, so a uniform initial-load window would filter on a placeholder and match nothing.
- **That derivation has exactly one exception:** `chunk_full_reload`. `sap.vbrp` is *both* full-only *and* 126 windows' worth of rows; without the exception the branch silently un-windowed every chunk and turned each into a complete 684,592,846-row read.
- **An initial load MUST be a full load, never windowed.** The invariant it protects is Bronze parity: a Bronze base table's row count must equal SAP's *within the initial-load horizon* (default last 5 years). Sub-windowing below the horizon is a bug — on 2026-07-15 `zdsales` / `zncr01` / `s611` loaded to a tiny window and Bronze came out far below SAP.
- **Master and creation-date tables load truly full — no horizon at all.** `KNA1` keyed on `ERDAT` keeps only ~33% of customers at a 5-year window, dropping active master data.
- **Empty results mean opposite things in the two directions.** An empty *delta* is a quiet window: succeed. An empty *full snapshot* is almost always a broken extract: fail — and fail **before** landing, so no `_SUCCESS` marker is written and the watermark is untouched. A spec's `on_empty` overrides both.
- **`init_state` is Gate 0's memory:** `(none)` → `initial_loaded` (an un-windowed `run_kind=initial` succeeded) → `cdc` (the first plain daily delta after that).
- **A windowed chunk never flips it.** Chunks run concurrently across lanes and finish out of order, so no single chunk can know the load is done. Flipping on the first one to finish admitted `sap.s603` into the very next daily cycle at watermark `20220131` — mid-reload, 649M rows (live 2026-08-09). The `download_barrier` owns the flip, once, when every chunk of every table is terminal.
- **A windowed *initial* load still seeds its watermark**, forward-only, from the window's end — otherwise a giant that never gets a full pull goes `initial_loaded` with watermark `''` and every later delta dies at the splice guard.
- **A backfill is a side trip, not progress.** `read_range` returns a DataFrame only; the watermark does not advance, and each chunk writes under its own `chunk=<from>_<to>` sub-prefix so concurrent chunks cannot clobber each other.

L05 · Full, Windowed, Backfill

Words you'll hear

TERM	MEANS
Read-mode	`full` / `range` / `incremental` — which connector method runs
`full_snapshot`	Job arg (or derived flag) meaning "read everything in scope"
Backfill	A bounded `[date_from, date_to]` re-pull that does not advance the watermark
Initial-load horizon	`INITIAL_LOAD_LOOKBACK_YEARS`, default 5 — the defined scope of "full"
Bronze parity	Bronze's row count equals SAP's, measured with the same horizon filter
`on_empty`	Per-spec override of the zero-row policy: `succeed` / `warn` / `fail`
`init_state`	`(none)` → `initial_loaded` → `cdc`, plus `needs_reinit`
Gate 0	The admission check that keeps a table out of CDC until its initial load is done
`chunk_full_reload`	Opt-in flag: full-refresh, but delivered as many date windows

In this repo

- `src/glue/glue_engine/jobs/source_download.py:222` — the precedence line; `:235-267` are the three branches it selects.
- `src/glue/glue_engine/jobs/source_download.py:172-213` — the full-only derivation, the 2026-07-16 empty-landing incident (`tvfkt` / `twewt` / `twtyt` / `zzz_mc2t`), and the `chunk_full_reload` exception found live on 2026-08-09.
- `src/glue/glue_engine/jobs/source_download.py:272-296` — the empty-result policy, resolved by `bronze_pull.py:393` (`_resolve_empty_policy`).
- `src/glue/glue_engine/jobs/source_download.py:318-363` — the `init_state` lifecycle and why a windowed chunk must not flip it.
- `src/glue/glue_engine/jobs/source_download.py:246-258` — TML-64, the forward-only watermark seed for a windowed initial load.
- `src/lambdas/download_barrier/handler.py:166` — `_completed_initial_loads`, the phase owner that actually declares an initial load complete.
- `CLAUDE.md:91-99` — "Initial load = full load = the last N years", the master/creation-date exception, and the 2026-07-17 data-horizon evidence.
- `CLAUDE.md:114-125` — the Bronze parity invariant, and the instruction to verify it on **both** sides rather than assert it from the code.

Do this

Pick one table from `config/ingestion_tables.yaml` and answer four questions from config alone: (1) which read-mode does its next scheduled run take? (2) if it returns zero rows, does the run succeed or fail? (3) what is its `init_state` right now, and what would flip it? (4) if you had to re-load one month of it, which mode would you use and would the watermark move? Then verify (1) and (3) against its `watermarks` row in DynamoDB.

You've got it when you can…

…say why an empty pull is a *success* for one table and a *failure* for another on the same morning — and why declaring a windowed chunk `initial_loaded` is the more dangerous of the two mistakes.

MODULE 2 · LESSON 06

What's Actually on Disk

Open a Silver table and you find four levels of pointers sitting on Parquet that is never edited.

1 · THE CHAIN — WHAT AN ENGINE READS, IN ORDER

metadata.json
the table, as of now
schema · partition spec
all snapshots + current id
`metadata/00007-….metadata.json`

→

manifest list
exactly one per snapshot
names the manifests and
their partition ranges
`metadata/snap-8134…-1-….avro`

→

manifests
the file inventory
row counts, null counts,
min/max per column
`metadata/….-m0.avro`

→

data files
the rows themselves
Parquet, immutable —
never edited in place
`data/….parquet`

Every level is just a file under the table's own prefix. The writer proves it — it splits the pointer on "/metadata/" to find the table root.

2 · SNAPSHOTS — AN APPEND-ONLY LOG

Every commit appends one snapshot

`snapshot 1` first append ▶ `snapshot 2` MERGE ▶ `snapshot 3` MERGE ▶ `snapshot 4` CURRENT

Nothing is rewritten: snapshot 3 still names exactly the files it named.
That is time travel — and it is why metadata grows until something expires it (Lesson 08).

3 · WHAT ATOMIC MEANS

One pointer, swapped

A commit writes new files first, then
swaps ONE pointer: which metadata.json
is the current one. That is the atom.
A reader on the old pointer sees the
old table, whole — never a half one.
`writers/s3_tables.py:487-489 · :515-531`

IN PLAIN ENGLISH

Nothing in an Iceberg table is edited. New files are written, then one pointer moves — and that swap is the whole commit.

L06 · What's Actually on Disk

Slide: `_render/L06-opening-an-iceberg-table.html`

The point

Module 1 told you "Silver is Iceberg on S3 Tables". This lesson opens the box. An Iceberg table is **not** a database and it is **not** a folder of Parquet — it is a short chain of pointer files, and a commit is one swap at the top of that chain.

Read it top-down, the way an engine does:

`metadata.json` → **manifest list** (one per snapshot) → **manifests** (the file inventory) → **data files** (Parquet).

Everything you will debug in the next three lessons — cardinality errors, small files, schema drift — is a consequence of that shape.

Key ideas

- `metadata.json` **is the table.** It holds the current schema, the partition spec, the full list of snapshots, and which snapshot id is *current*. Point a reader at a different `metadata.json` and it sees a different table.
- **One snapshot = one manifest list.** The manifest list names the manifests that make up that snapshot, plus their partition ranges, so a planner can skip whole manifests before it reads them.
- **Manifests are the inventory, and they carry statistics** — row counts, null counts, min/max per column. Most "why is this query fast" answers live here: the engine prunes files on stats without opening them.
- **Data files are immutable.** Nothing is edited in place. An update writes new Parquet and a new snapshot that stops referring to the old file. That is why "MERGE" and "overwrite" both cost you new files.
- **Snapshots are an append-only log.** Snapshot 3 still names exactly the files it named after snapshot 4 lands. That is what makes time travel and rollback possible — and it is exactly why metadata grows forever without expiry (Lesson 08).
- **"Atomic" means one pointer swap.** A commit writes all the new files first, then swaps which metadata file is current, in one step. A reader that already resolved the old pointer keeps reading a complete older table. There is no window in which the table is half-written.
- **You can see the pointer from Spark.** The repo does exactly that after every write: it reads the newest `metadata_location` from the table's own `metadata_log_entries` metadata table and re-registers it in the Glue mirror catalog so Redshift Spectrum sees the newest snapshot.

Words you'll hear

WORD	WHAT IT MEANS HERE
`metadata.json`	The table's root pointer file: schema, partition spec, snapshot list, current snapshot id
Manifest list	One Avro file per snapshot; names the manifests belonging to that snapshot
Manifest	An Avro file listing data files plus per-column stats used for pruning
Snapshot	One immutable version of the table, produced by one commit
`metadata_location`	The S3 URI of the current `metadata.json` — what the Glue catalog stores
Commit	Write new files, then swap the current-metadata pointer. One step
Time travel	Reading an older snapshot, which is possible only because nothing is deleted

L06 · What's Actually on Disk

In this repo

- `src/glue/glue_engine/writers/s3_tables.py:487-489` — the writer reads the live pointer straight off the table: `SELECT file FROM <fqn>.metadata_log_entries ORDER BY timestamp DESC LIMIT 1`.
- `src/glue/glue_engine/writers/s3_tables.py:515-531` — `glue.create_table` with `Parameters.metadata_location=<latest>`. Line 528 splits that URI on `"/metadata/"` to recover the table root: proof that the metadata files live under `<table-root>/metadata/`.
- `src/glue/glue_engine/writers/s3_tables.py:505-513` — the mirror table is deleted and recreated on every write **because the `metadata_location` changes on every write**. If you only remember one line from this lesson, remember why that delete is there.
- `src/glue/glue_engine/writers/s3_tables.py:143-173` — `_first_write_create`: the CREATE TABLE that produces the very first `metadata.json`, including `PARTITIONED BY` (see Lesson 09).
- `docs/design-reference/decisions/0024-s3-tables-managed-iceberg.md` — why Bronze and Silver are S3 Tables at all, and the hard rule that follows from this file layout: **no S3 SDK access to table-bucket objects, only Iceberg APIs.**

Do this

1. In a Glue/Spark session against Dev, run `SELECT * FROM <catalog>.<ns>.<table>.metadata_log_entries ORDER BY timestamp DESC` on a Silver table. Count the rows. That is your commit history.
2. Then `SELECT * FROM <catalog>.<ns>.<table>.snapshots` and `.files`. Match one snapshot to its manifest list, and one manifest to a Parquet file. Say out loud which level each row came from.
3. Trigger one more Bronze load and re-run the first query. Exactly one new row should appear, and the `metadata_location` in the Glue mirror table should now point at it.
4. Explain, without looking, why `_register_in_mirror_catalog` has to delete before it creates.

You've got it when you can…

…draw `metadata.json → manifest list → manifests → data files` from memory, say which of those four an engine reads first and which one it usually doesn't need to read at all, and answer "what makes an Iceberg commit atomic?" with **"it swaps one pointer, after all the new files are already written"** — then point at the line in `s3_tables.py` that reads that pointer.

MODULE 2 · LESSON 07

Why This MERGE Failed

A target row may be claimed by at most ONE source row. Every cardinality error is a duplicate upstream.

1 · THE ARMS — WHAT THE WRITER ACTUALLY BUILDS

Three arms, assembled as a string

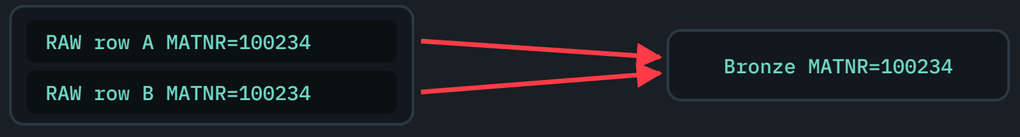
```
MERGE INTO <target> t USING <source> s  ON t.`k` = s.`k`
WHEN MATCHED AND s.`_change_op` = 'D'   THEN DELETE
WHEN MATCHED                            THEN UPDATE SET *
WHEN NOT MATCHED AND s.`_change_op` <> 'D'
                                        THEN INSERT *
```

The delete arm exists only when the frame carries _change_op (SAP ODP deltas).

writers/s3_tables.py:251-274 · jobs/bronze_to_silver.py:228-233

2 · THE ONE-MATCH CONSTRAINT

Two source rows, one target row




```
MERGE_CARDINALITY_VIOLATION — the whole job dies
```

Live on QA 2026-07-23: a retried source_download re-wrote the same cycle prefix, so the snapshot landed twice — sap.mbew, sap.mbewh.

3 · THE FIX — DEDUPE THE SOURCE, IN ALL THREE PLACES

IN THE JOB

A retried download can double-write the cycle prefix. Collapse the frame to one row per merge_key first.

```
df.dropDuplicates(spec.merge_key)
```

jobs/bronze_pull.py:313 · commit 359b3b2

IN THE WRITER

merge_into re-dedupes the classic upsert source itself: ROW_NUMBER over the key, latest ingest wins.

```
.where(col("_rn") == 1)
```

writers/s3_tables.py:241 · :539-561

IN dbt

Redshift refuses QUALIFY over a Spectrum external table, so staging ranks in a subquery: WHERE _rn = 1.

```
ROW_NUMBER() OVER (PARTITION BY werks, datum)
```

models/staging/stg_sap_zscc.sql:24-63

IN PLAIN ENGLISH

MERGE promises that no two source rows want the same target row. Break that promise and it stops — the duplicate is upstream.

L07 · Why This MERGE Failed

Slide: `_render/L07-merge-mechanics.html`

The point

`MERGE INTO` is how Bronze and Silver stay idempotent: replays, overlapping windows and Glue auto-retries land each row **once** instead of duplicating it. It works because of one rule, and it fails for one reason.

The rule: a target row may be claimed by **at most one** source row. Two source rows matching the same target row is `MERGE_CARDINALITY_VIOLATION`, and the job dies.

Therefore: a cardinality error is never a bug in the target. It is a duplicate that arrived with the source. You fix it upstream, before the MERGE, and this repo now does that in three separate places.

Key ideas

- **The arms.** `WHEN MATCHED THEN UPDATE SET *` handles the update; `WHEN NOT MATCHED THEN INSERT *` handles the insert. That pair is the classic upsert and is what most tables get.
- **The delete-aware third arm.** When the frame carries `_change_op` (SAP ODP deltas), the writer builds a different statement: `WHEN MATCHED AND s._change_op = 'D' THEN DELETE`, plus `WHEN NOT MATCHED AND s._change_op <> 'D' THEN INSERT`. A `'D'` for a row that is already absent is deliberately **not** inserted, and `_change_op` itself is excluded from the written column list — it is a Bronze audit column, not a Silver attribute.
- **The one-match constraint is a planner rule, not a data rule.** Iceberg refuses because the result would be non-deterministic: which of the two source rows should win? It will not guess.
- **The real incident (2026-07-23, QA).** A retried `source_download` re-wrote the same `cycle=<id>/` RAW prefix. Two overlapping `.mode("overwrite")` writes do **not** clean each other on S3, so the full snapshot landed twice → duplicate PKs → `MERGE_CARDINALITY_VIOLATION` in `bronze_pull` on `sap.mbew` / `sap.mbewh`, blocking the SAP→Gold E2E at the RAW→BRONZE hop. Fixed in commit **`359b3b2`**.
- **Dedup is safe here for a specific reason, not in general.** A JDBC watermark-range delta is a *current-state snapshot* — one row per PK. So collapsing to one row per `merge_key` only ever drops byte-identical copies, never a distinct update. Say that reason out loud before you add a `dropDuplicates` anywhere else.
- **When you have audit columns, pick a winner deterministically.** `_dedupe_latest` ranks with `ROW_NUMBER() OVER (PARTITION BY key ORDER BY _ingested_at DESC, _run_id DESC)` and keeps rank 1 — the same ordering as the offline parity cleanup, so an in-flight dedup and a retrospective one agree. Without those columns it falls back to `dropDuplicates(key)` (arbitrary winner).
- **First write is the sneaky one.** `MERGE INTO` on a non-existent table raises table-not-found and falls back to `CREATE TABLE + append`, which appends the frame **verbatim**. If the source was not deduped, duplicate keys become permanent Bronze PK duplicates from day one — that is the `zncr01` defect (446,611 rows landed against a 438,645-row source).
- **`QUALIFY` is the canonical idiom, but not on Spectrum.** Redshift raises `syntax error at or near ROW_NUMBER` for `QUALIFY` when the scan targets a Spectrum external table, so the staging models use an equivalent ranked subquery with `WHERE _rn = 1`.

L07 · Why This MERGE Failed

Words you'll hear

WORD	WHAT IT MEANS HERE
Upsert	Update if the key exists, insert if it doesn't — one statement
`merge_key`	The natural/primary key a Bronze spec declares for the MERGE
`MERGE_CARDINALITY_VIOLATION`	Two source rows matched one target row; Iceberg refuses to guess
Delete-aware MERGE	The extra arm that turns `_change_op='D'` into a real DELETE (gap G09)
Idempotent	Re-running the same input leaves the same row count — the whole point of MERGE
`QUALIFY`	Filter on a window function without a subquery. Unsupported over Spectrum externals
Source-side dedup	Collapsing the input to one row per key *before* the MERGE sees it

In this repo

- `src/glue/glue_engine/writers/s3_tables.py:193-281` — `merge_into`. Read the docstring first (`:196-228`), then the two SQL branches: delete-aware at `:251-266`, classic upsert at `:267-274`.
- `src/glue/glue_engine/writers/s3_tables.py:241` — `source = df if delete_aware else _dedupe_latest(df, key)`, and the comment above it explaining the `zncr01` defect and the first-write trap. `_dedupe_latest` itself is at `:539-561`.
- `src/glue/glue_engine/jobs/bronze_pull.py:302-314` — the `359b3b2` fix: `df = df.dropDuplicates(list(spec.merge_key))` immediately before `writer.merge_into(...)`, with the incident written into the comment.
- `src/glue/glue_engine/jobs/bronze_to_silver.py:213-233` — where the key comes from on the Silver side: the spec's `deduplicate` op. No `deduplicate` op → the job refuses to run rather than MERGE on a guessed key.
- `src/glue/glue_engine/spec.py:132-144` — the `merge_key` field, with the TML-68 rationale and the 446,611-vs-438,645 numbers. Example in use: `src/glue/specs/bronze/sap_mbew.yaml:14` — `merge_key: [MANDT, MATNR, BWKEY, BWTAR]`.
- `src/dbt/models/staging/stg_sap_zscc.sql:19-63` — the same problem one layer up: a duplicate `(site, date)` would fan out `unified_sales`' All-Dept joins and break the **Redshift** MERGE with "multiple matches". Note the comment recording why `QUALIFY` could not be used.

Do this

1. Read `merge_into` end to end and write down, from the code alone, the exact SQL it emits for (a) a frame with `_change_op` and (b) one without.
2. `git show 359b3b2`. Read the commit message before the diff — it is a model incident write-up: symptom, mechanism, blast radius, fix, why the fix is safe.
3. **Break it:** build a two-row DataFrame with the same `merge_key`, MERGE it into a Dev table, and read the error text. Then add `dropDuplicates` and watch it pass.
4. Answer this: why is the delete-aware source deliberately *not* passed through `_dedupe_latest`? (Hint: what would happen to a `'D'` marker?)
5. Take one Silver spec and find its `deduplicate` key. Convince yourself it is genuinely unique in Bronze — that key is the whole contract.

L07 · Why This MERGE Failed

You've got it when you can...

...read `MERGE_CARDINALITY_VIOLATION` in a Glue log and, without opening the target table, say **"two source rows share a merge key — something upstream landed twice"**; name the three places this repo dedupes (job, writer, dbt staging); and explain why deduping the source is safe for a watermark-range delta but would be dangerous for a true CDC stream carrying `'D'` markers.

MODULE 2 · LESSON 08

The Maintenance Problem

Every write leaves files and a snapshot behind. AWS sweeps some of it — one job we wrote is not scheduled.

1 · WHY SMALL FILES HAPPEN

One run = a few files + one snapshot

Bronze appends a delta every cycle. Silver MERGEs. Both commit a new snapshot each time, and each snapshot names a new manifest list. Across 48 tables on a daily cadence that is thousands of small Parquet files and a metadata chain that only gets longer.

Planning a query means reading that chain first — so this shows up as scans getting slower, never as an error.

2 · WHAT S3 TABLES DOES FOR YOU

Managed compaction — and only that

ADR-0024 chose S3 Tables so AWS compacts data files for us. We deliberately do NOT schedule rewrite_data_files — it would fight the managed maintenance. Terraform enables file removal per bucket:

```
iceberg_unreferenced_file_removal   unreferenced_days = 7   # bronze
                                    unreferenced_days = 30  # silver
```

Per-table maintenance is never declared: Glue creates tables, not Terraform.

infra/modules/s3-data-lake/main.tf:8, 11-47

3 · THE HONEST GAP — expire_snapshots HAS NO CALLER

The method exists

S3 Tables compacts data files. It does not shorten OUR history: every append / merge / overwrite adds a snapshot, and the manifest list plus metadata JSON grow with it.

```
CALL <cat>.system.expire_snapshots(table => …)
```

writers/s3_tables.py:407-449 (older_than_days=7, retain_last=100)

Nothing calls it

The docstring says so, in as many words: "there is no automatic caller yet". Until a weekly cron job instantiates a writer per table and calls this, snapshots are never expired — and query planning gets slower the longer a table lives.

writers/s3_tables.py:431-434 · audit M-23, still open

IN PLAIN ENGLISH

AWS sweeps up the small files for us. It does not shorten the history — we wrote that job and never plugged it in.

L08 · The Maintenance Problem

Slide: `_render/L08-small-files-compaction.html`

The point

Every write to an Iceberg table leaves two kinds of litter behind: **small data files**, and **a longer history**. Somebody has to sweep up, or reads get slower and slower for no visible reason.

On this platform the split is:

- **Data files → AWS.** S3 Tables runs managed compaction, and ADR-0024 chose it precisely so we would not have to author ~6 maintenance Glue jobs. We deliberately do **not** schedule `rewrite_data_files` — it would fight the managed maintenance.
- **History → nobody.** `expire_snapshots()` exists in the writer. **It has no scheduled caller.** This is a real, open gap in the platform, and it is taught here as one.

This lesson is deliberately honest about that. You will be asked about it in week 1.

Key ideas

- **Small files are a consequence of cadence, not of carelessness.** Bronze appends a delta every cycle; Silver MERGEs. Each write commits a new snapshot with a new manifest list. 48 tables × daily × months = thousands of small Parquet files and a metadata chain that only ever grows.
- **The cost shows up as slowness, never as an error.** Planning a query means walking the metadata chain first. Nothing fails; scans just get worse. That makes it the kind of problem that survives for a year unnoticed.
- **Compaction and snapshot expiry are different jobs.** Compaction rewrites many small data files into fewer big ones. Expiry drops old *snapshots* so the manifest list and metadata JSON stop growing. Managed compaction does not do the second one for us at the app level.
- **What Terraform actually configures.** Only bucket-level `iceberg_unreferenced_file_removal` : Bronze at 7 days, Silver at 30. That reclaims files nothing references any more — it is not compaction, and it is not snapshot expiry.
- **Per-table maintenance is never declared, and that is structural.** The table bucket and namespaces are Terraform's; **the tables are not** — they register themselves on first write from the Glue job. So no Terraform resource exists on which to set per-table maintenance, and those settings stay at the service default.
- **`expire_snapshots()` is real code.** It CALLs Iceberg's `system.expire_snapshots` stored procedure on the writer's own table, with `older_than_days=7` and `retain_last=100` (the retain floor is the time-travel/rollback safety net).
- **And nothing calls it.** The docstring says so in as many words: *"there is no automatic caller yet. This is intended to be invoked by a scheduled maintenance job (e.g. a weekly EventBridge-cron Glue job that instantiates a writer per table and calls this)."* The audit tracks it as **M-23, PARTIAL** — coded but unwired.
- **A related trap worth carrying with you:** because managed compaction rewrites files, `modifiedAt` on an S3 table is **not** a build timestamp. It has read hours newer than the last real write during a live incident. Use Glue `get-job-runs` to triage staleness, never the object timestamp.

L08 · The Maintenance Problem

Words you'll hear

WORD	WHAT IT MEANS HERE
Small-file problem	Many tiny Parquet files; per-file overhead dominates the scan
Compaction (`rewrite_data_files`)	Merge small data files into fewer large ones. Managed by AWS here
Snapshot expiry	Drop old snapshots so metadata stops growing. **Not scheduled here**
Unreferenced-file removal	Delete objects no snapshot references. Configured in Terraform, per bucket
`retain_last`	Minimum number of recent snapshots to keep regardless of age
Managed maintenance	The S3 Tables service doing the sweeping instead of a Glue job you wrote
M-23	The audit finding for this gap — still open

In this repo

- `src/glue/glue_engine/writers/s3_tables.py:407-449` — `expire_snapshots` . The docstring (`:414-434`) explains why compaction is delegated and expiry is not; the **NOTE at `:431-434`** is the "no automatic caller" admission. The actual `CALL <catalog>.system.expire_snapshots(...)` is at `:444-449` .
- `infra/modules/s3-data-lake/main.tf:11-47` — the two table buckets and the only maintenance we declare: `iceberg_unreferenced_file_removal` , `unreferenced_days = 7` (Bronze, `:14-22`) and `= 30` (Silver, `:33-41`). Line **8** is the load-bearing comment: *"individual tables are owned by the Glue jobs and not declared here."*
- `docs/design-reference/decisions/0024-s3-tables-managed-iceberg.md` — the decision that removed ~6 maintenance Glue jobs from the backlog, and the honest "Cons" section.
- `docs/handoff/audit-remediation-verification.md:79` — M-23 marked **PARTIAL**: *' `expire_snapshots()` added … but no scheduled caller"*. Note the line reference there (`s3_tables.py:345-387`) has drifted — the method is now at `:407-449` . Verify line numbers before you quote them.
- `docs/handoff/remediation-proposals-nogo.md:249` — the proposed fix: wire a scheduled maintenance Glue job or Step Functions cron; file compaction stays delegated.
- `docs/risks.md` — R53, for the `modifiedAt` -is-not-a-build-time lesson, discovered during a real staleness triage.

Do this

1. Open `expire_snapshots` and then `grep -rn "expire_snapshots" src/ infra/` . Confirm for yourself that the only hits are the definition and its own docstring. This is what "coded but unwired" looks like in a real codebase.
2. Count the snapshots on the busiest Dev Silver table (`SELECT count(*) FROM <fqn>.snapshots`). Multiply by your cadence and project 12 months out.
3. Sketch the missing job on paper: EventBridge weekly cron → Glue job → for each spec, build an `S3TablesWriter` and call `expire_snapshots` . What is the failure mode if it runs *while* a long Redshift Spectrum query is reading an old snapshot? (That is what `retain_last=100` is defending.)
4. Read ADR-0024's "Cons (honest)" section and be able to state one thing we gave up by choosing managed Iceberg.

L08 · The Maintenance Problem

You've got it when you can…

…split "maintenance" into **compaction / unreferenced-file removal / snapshot expiry**, say who owns each one on this platform (AWS / Terraform-configured AWS / **nobody**), point at `s3_tables.py:407-449` plus the NOTE at `:431-434` as evidence, and describe the symptom of the gap as *"query planning gets slower the longer a table lives"* rather than as an error anybody would see in a log.

MODULE 2 · LESSON 09

Changing a Table Without Breaking It

Adding a column is a metadata edit. Dropping, renaming or re-typing one stops a build — on purpose.

1 · WHAT'S SAFE, WHAT BREAKS

ADDITIVE — SAFE

· Add a column at the END
· Iceberg assigns it a new field id; old files stay valid
· Old rows read back NULL, so backfill before anyone reads it

`metadata-only — no data rewritten`

NON-ADDITIVE — BREAKS

· Drop or rename a column
· Narrow or re-type one
· Reorder the written frame
· Edit a partition spec and expect old data to move

`every one of these has bitten us`

2 · THE THREE GUARDS

dbt · on_schema_change: fail
A new or missing column stops the Gold build instead of being absorbed.
`src/dbt/dbt_project.yml:91 · unified_customer_count.sql:31`

engine · merge-schema, full rewrites only
Only full_refresh and create_or_replace evolve; partial writes never do.
`writers/s3_tables.py:337-345, 353-356 · risks.md R54`

engine · _guard_non_additive_drift
A rebuild that would DROP a live column is refused by name, not by arity.
`writers/s3_tables.py:297-324`

3 · TWO TRAPS THAT ONLY SHOW UP ON THE SECOND RUN

cc was BIGINT on Monday, NUMERIC on Tuesday

unified_customer_count unions three legs. The actuals legs' raw cc is BIGINT; the budget leg is SUM(cc_budget), which is fractional. A first build with only actuals materialises BIGINT — the next run widens it, on_schema_change='fail' trips, Gold needs --full-refresh.

```
CAST(cc AS NUMERIC(38,4)) -- in EVERY leg, so the type can never drift
```

`models/marts/gold/unified_customer_count.sql:36-42, 47, 58, 79 · audit M-28`

Partitioning is create-only

partition_by is consulted ONLY on the first write, when the CREATE TABLE is issued. Later writes inherit the layout — editing the YAML changes nothing.

`writers/s3_tables.py:116-117, 165-172`

IN PLAIN ENGLISH

Adding a field to a payload breaks nobody. Renaming one breaks every caller — so we stop the build rather than let it guess.

L09 · Changing a Table Without Breaking It

Slide: `_render/L09-schema-evolution-partitioning.html`

The point

Iceberg makes **additive** change cheap: adding a column is a metadata edit, old data files stay valid, nothing is rewritten. Everything else — dropping, renaming, re-typing, reordering, re-partitioning — is a migration, and this platform is deliberately built to **stop** rather than to absorb it.

The reason is a failure mode you cannot see: when a write fails, the old table stays readable, so downstream keeps serving pre-change data and nothing surfaces the problem. Loud failure is the cheaper outcome.

Key ideas

- **Safe: add a column at the end.** Iceberg assigns it a new field id; existing files simply don't have it and read back NULL. Backfill before anyone depends on it — a NULL that means "not populated yet" gets read as a business value.
- **Breaking: drop, rename, narrow, re-type, reorder.** Each changes what existing rows *mean*, and each has bitten this repo.
- `on_schema_change: fail` **is a choice, not a default.** Every Gold model inherits it from `dbt_project.yml`, with the comment "NEVER silently absorb schema drift". The alternatives (`ignore`, `append_new_columns`, `sync_all_columns`) all let a mismatch pass quietly into a table Power BI is reading. We would rather the build go red.
- **Schema evolution is allowed only on writes that rewrite every row.** `full_refresh` and `create_or_replace` pass Iceberg's `merge-schema` option, so an added column lands instead of failing. `append`, `overwrite_partitions` and `merge_into` deliberately do **not** get it — they touch a subset of rows and would leave the rest NULL in the new column. A unit test pins that.
- **Dropping is refused by name.** `merge-schema` is a union, so a column the frame stopped emitting would survive on the target and get NULLs written over real values. `_guard_non_additive_drift` raises first, naming the table and the columns — replacing a 40-column Spark arity error that never said what to do about it.
- **The real incident (R54).** `ops.py` gained `is_billing_plan`, so `abap_transform` began emitting **42** columns into **41**-column tables and died with `INSERT_COLUMN_ARITY_MISMATCH.TOO_MANY_DATA_COLUMNS` — on QA, and latent on Dev. Meanwhile the stale table stayed readable and downstream kept serving pre-change numbers. R56 was the same thing again on `sap.distress_603`.
- **Partitioning is a create-time decision.** `partition_by` is consulted **only** on first write, when the SQL-DDL `CREATE TABLE … PARTITIONED BY (…)` runs. Later writes inherit the existing layout. Editing the YAML on a live table changes nothing — and no existing data ever moves.
- **The type-widening trap (M-28).** `unified_customer_count` unions three legs. The actuals legs' raw `cc` is BIGINT; the budget leg is `SUM(cc_budget)`, which is fractional. A first build with only the actuals legs materialises `cc` as BIGINT — then the next run, once the budget leg carries rows, widens it to NUMERIC, `on_schema_change='fail'` trips, and Gold needs a manual `--full-refresh`. **The fix is to pre-CAST every leg to the same `NUMERIC(38,4)`, so the resolved column type can never drift regardless of which legs have rows.**
- **Same trap, one layer down.** `stg_sap_zscc` casts `zcustomer` / `zitem` to `NUMERIC(38,4)` for the same reason: leaving them VARCHAR raised *"UNION types numeric and character varying cannot be matched"*. Pin types at the edge of the model, not in the middle.

L09 · Changing a Table Without Breaking It

Words you'll hear

WORD	WHAT IT MEANS HERE
Additive change	Add a column at the end. Metadata-only; no data rewritten
Schema drift	The frame's columns no longer match the target's
`on_schema_change`	dbt's policy knob. We use `fail` on every Gold model
`merge-schema`	Iceberg write option that lets an added column land. Full-rewrite paths only
`INSERT_COLUMN_ARITY_MISMATCH`	Spark's "you gave me 42 columns for a 41-column table"
Type widening	BIGINT → NUMERIC, resolved at build time from whichever legs carried rows
Partition spec	The `PARTITIONED BY` layout, fixed when the table is created
Partition evolution	Changing that spec for *future* writes. Old data never moves

In this repo

- `src/dbt/dbt_project.yml:88-93` — the Gold defaults: `+incremental_strategy: merge`, and line **91** `+on_schema_change: fail # NEVER silently absorb schema drift`.
- `src/dbt/models/marts/gold/unified_customer_count.sql:27-34` — the model's own config, `on_schema_change='fail'` at `:31`. The **M-28 comment at `:36-42`** is the clearest explanation of the widening trap in the codebase; the pre-casts are at `:47`, `:58` and `:79`.
- `src/glue/glue_engine/writers/s3_tables.py:326-361` — `full_refresh`. Docstring `:337-345` records R54; the `merge-schema` option is at `:354`. Same pattern in `create_or_replace` at `:363-405`.
- `src/glue/glue_engine/writers/s3_tables.py:297-324` — `_guard_non_additive_drift`: read the raised message, it is a model of a good error.
- `src/glue/glue_engine/writers/s3_tables.py:116-117` — *'`partition_by` is consulted ONLY on first-write (when the table is created); subsequent appends inherit the existing Iceberg layout."* The DDL that consumes it is at `:165-172`.
- `src/glue/specs/bronze/sap_zsdcc.yaml:56-57` — a real partition spec, `months(date)`. Iceberg transforms like this are why the writer uses SQL DDL rather than Spark V2's `.partitionedBy()`.
- `docs/risks.md` — **R54** (the 41-vs-42 column drift, the engine fix, and the deliberate decision to exclude the partial-write paths) and **R56** (the same class recurring on `distress_603`). Read R54's mitigation column: it is the best short account of this whole lesson.
- `src/dbt/models/staging/stg_sap_zscc.sql:32-41` — the staging-level casts and the UNION type error they prevent.

L09 · Changing a Table Without Breaking It

Do this

1. Add a column to a Silver derived table's DataFrame in Dev and run the transform. Watch `full_refresh` absorb it. Now do the same on a path that uses `append` and predict the error before you read it.
2. Delete a column from that frame instead, and read the message `_guard_non_additive_drift` raises. Compare it to the raw Spark arity error it replaced.
3. Change `partition_by` in a spec for a table that already exists. Re-run. Confirm nothing changed, then explain why from `s3_tables.py:116-117`.
4. In `unified_customer_count`, mentally delete the three `CAST(... AS NUMERIC(38,4))`. Describe the exact sequence of two runs that breaks the model — first run fine, second run red.
5. Write the checklist you would follow to add a column to a Gold model end to end: Silver frame → Silver table → staging model → Gold model → `--full-refresh` or not.

You've got it when you can…

…classify any proposed schema change as **additive (safe) / non-additive (a migration)** in one sentence; explain why we chose `on_schema_change: fail` over silent absorption using the R54 failure mode (*"the stale table stays readable and keeps serving pre-change numbers"*); state that partitioning is fixed at create and editing the YAML later does nothing; and describe the M-28 fix as **"pin every UNION leg to the same type so the resolved column type can't drift between runs."**

MODULE 2 · LESSON 10

When a Join Won't Do

ZHOCIDC is a positional state machine, not a table. One receipt is an ordered H→S→T→F block, and every field changes meaning per row.

1 · ONE RECEIPT · FOUR RECORD TYPES · ONE FIELD

H · HEADER

Opens a basket, resets state
ZVARID1 → cashier no
ZCODE1='3' → skip the receipt

`abap/baskets.py:96-115`

S · SALE LINE

ZVARID4[9:18] → amount × 100
ZVARID4[0:8] → qty, [4]=':' flag
ZVARID3='-' → flips the sign

`abap/baskets.py:117-162`

T · TENDER

ZVARID4[0:2] → tender code
01 Cash · 02 Gift · 10-19 Card
a 2nd T → 'Multiple Tender'

`abap/baskets.py:164-172`

F · FOOTER = COMMIT

Drop zero-price items, emit header + items. No F row → the receipt never happened.

`abap/baskets.py:174-183`

SEQNO order — each row reads the state the last one left

2 · WHY A SET-BASED JOIN CANNOT SAY THIS

A JOIN SEES EACH ROW ALONE

Here row N's meaning depends on 1..N-1:
· basket_id comes from the last H seen
· ZVARID4 is re-read per ZTRNTYPE
· one barcode accumulates over lines
· nothing exists until F commits
No join key expresses "the row before".

3 · THE SPARK SHAPE — GROUP, FOLD, SPLIT

GROUP → FOLD → SPLIT

```
_RECEIPT_KEYS = ["werks","zdate","zptno","zreceipt"]
def _fold(pdf):          # plain pandas, no Spark
    rows = pdf.sort_values("seqno").to_dict("records")
    return build_baskets(rows, bukrs=…, vat_rate=…)
zhocidc.groupBy(*_RECEIPT_KEYS).applyInPandas(_fold,…)
```

Group on zdate, not budat — else one receipt splits in two.
`src/glue/glue_engine/abap/ops.py:41, 79-91`

THE CORE IS PURE

build_baskets takes a list of dicts, returns two lists. No Spark, no AWS, no S3 — it runs in pytest in milliseconds, so every edge case has a test.

`tests/unit/test_phase3_abap_transform.py:108`

IN PLAIN ENGLISH

A receipt is a till roll, not a spreadsheet row. You have to read it top to bottom — the total at the bottom only means anything because of the lines above it.

L10 · When a Join Won't Do ★

Slide: `_render/L10-sequential-to-spark.html`

The point

`ZHOCIDC` is the NCR-POS end-of-day log. It looks like a table and it is not one.

- Each row carries a **record type** in `ZTRNTYPE` — `H` header, `S` sale line, `T` tender, `F` footer/commit — and the *same* generic columns (`ZVARID1..4` , `ZCODE1..3` , `TAKLV`) mean **different things per type**. `ZVARID4` is one 19-character string: on an `S` row `[9:18]` is the amount × 100 and `[0:8]` is the quantity; on a `T` row `[0:2]` is the tender code.
- A receipt is an **ordered block** of those rows, `H … S … T … F` , read in `SEQNO` order. Row *N* only means something given rows *1..N−1*: the open `basket_id` came from the last `H` , an item's quantity accumulates over repeated barcodes, and **nothing is emitted until `F` commits**.
- A set-based join cannot express "the row before". So the port keeps the ABAP's shape: **group by receipt, then fold the group in order**. In Spark that is `groupBy(...).applyInPandas(...)` ; the fold itself is plain Python.
- The correctness-critical part — `build_baskets` — is **pure**: a list of dicts in, two lists out. No Spark, no AWS. That is what makes it unit-testable, and it is why the edge cases (training transactions, uncommitted receipts, sign flips) have real tests instead of hope.

Key ideas

- **Read the ABAP's control flow, not its output.** `process_sold_articles` (LZID8F03) is a linear loop with mutable work areas (`w_ilsxd` , `w_total` , `w_linno` , `w_tender_count`). The port keeps those as local variables inside one function; it does not try to reconstruct them with window functions.
- **The four ported branches.** `H` opens a basket and resets state (`ZCODE1='3'` = a training transaction → skip the whole receipt). `S` parses amount/qty/sign and accumulates into an item keyed by **barcode** (matching the ABAP's read-by-EAN, so two scans of the same product merge into one line). `T` decodes the tender code and flips to `'Multiple Tender'` on the second one. `F` drops zero-price items, rolls up `BASKETAMOUNT*` , and appends.
- **All-or-nothing per receipt.** No `F` → no header, no items. A header with sales but no commit produces *nothing* — proven by `test_build_baskets_uncommitted_receipt_not_emitted` .
- **The group key must equal the emitted identity.** `basket_id` is `'20' + zdate + zptno + zreceipt` (truncated to 16). The fold therefore groups on `(werks, zdate, zptno, zreceipt)` — **not** `budat` . That was Item 10: grouping on `budat` split a receipt whose POS date and posting date disagreed across two groups, and each group committed its own header under the same `(store_id, basket_id)` . Measured on dev Bronze 2026-08-08 over 43,500,445 header rows: 22,150 identities spanned more than one `budat` against 88 where SAP genuinely emits two headers — the grouping key manufactured ~99.6 % of the duplicates.
- **`applyInPandas` is a contract with two sides.** Spark shuffles every row to its group's partition; Python then materialises the whole group as a pandas DataFrame. Cheap here because a receipt is tens of rows — it would be ruinous on a key with a million-row group. Choose the *smallest* key that still contains all the state.
- **The JSON tag trick.** One pass emits both outputs: each folded row is `{_kind: 'H'|'I', _row: <json>}` , and the union is filtered twice into `sap.basket` and `sap.basket_item` . That is also why `folded.cache()` is there — see L12.
- **What is deliberately not ported yet** is written down in the module docstring: markdown/discount (`C`), weight items, external-vendor EAN remap, points (`G`), promo code (`K`), B2B (`y`), and the ecom/bulk paths. A deferral you can name is a plan; a deferral you can't is a bug.

L10 · When a Join Won't Do ★

Words you'll hear

WORD	WHAT IT MEANS HERE
Positional / state machine	The row's meaning depends on its position and on the state left by earlier rows
Fold (reduce)	Walk an ordered sequence carrying an accumulator — the shape a cursor loop has in a functional language
`applyInPandas`	Spark: shuffle rows into groups, hand each group to Python as a pandas DataFrame
Receipt identity	`(store_id, basket_id)` — `basket_id` alone does **not** encode the store
Pure core	A function with no I/O and no framework, so it can be tested in milliseconds
Commit record	The `F` row — until it arrives the receipt does not exist downstream

In this repo

- `src/glue/glue_engine/abap/baskets.py:55-184` — `build_baskets`, the whole state machine: `H` at `:96`, `S` at `:117`, `T` at `:164`, `F` at `:174`, and `_commit` at `:76-91`.
- `src/glue/glue_engine/abap/ops.py:30-91` — `_RECEIPT_KEYS` with the Item 10 comment and the measured numbers, `basket_op`, the `_fold` closure, `applyInPandas` at `:86`, `folded.cache()` at `:87`; `_explode_json` at `:94-101`.
- `src/glue/specs/transform/basket.yaml` — one spec, two outputs (`sap.basket`, `sap.basket_item`), `vat_rate: 0.15`, and the list of inputs deferred until their op logic lands.
- `docs/ABAP/ID8-EXTRACTOR-MAPPING.md:60-119` — the raw field layout table and the per-record-type rules; `:252-255` states the constraint outright: *"ZHOCIDC is a positional state machine… not a set-based join. This is the single hardest part to port."*
- `tests/unit/test_phase3_abap_transform.py:69-166` — the group-key test, the golden one-item receipt (`basket_id`, `posting_date = budat − 1`, tender `Cash`, excl = 15.00/1.15), the training-transaction skip, and the uncommitted receipt.
- `src/dbt/tests/assert_basket_receipt_identity_is_unique.sql` — the downstream guard, and a model of how to write one: it does **not** assert uniqueness (SAP itself retransmits blocks); it asserts the transform never *splits* one physical receipt. 656 duplicate identities out of 41,038,625 QA headers, every one explained.

Do this

1. Open `baskets.py` and hand-execute the four rows in `test_build_baskets_single_receipt_header_item_tender_footer`. Write down the value of `hdr`, `cur_items`, `w_total` and `w_tender_count` after each row.
2. Delete the `F` row from that test in your head. Predict the output; then run the test that already proves it.
3. Try to write the same logic as SQL. Get as far as you can with `LAG` / `LAST_VALUE` over `PARTITION BY receipt ORDER BY seqno`, then say precisely where it breaks (hint: the accumulate-by-barcode step, and the conditional commit).
4. Change `_RECEIPT_KEYS` to use `budat` instead of `zdate` and run `test_basket_group_key_matches_the_emitted_receipt_identity`. That red test is Item 10.

L10 · When a Join Won't Do ★

You've got it when you can…

…explain to a SQL-first colleague **why this one table gets a Python fold while everything else in the pipeline is a join** — and then show them the two lines that make it safe: the group key that equals the emitted identity, and the pure `build_baskets` core with its unit tests.

MODULE 2 · LESSON 11

Porting ABAP Faithfully

Three ported functions, three kinds of fidelity — a data-driven rate, a derived rule, and an off-by-one we kept on purpose.

1 · A RATE THAT IS DATA

vat_split()

ZMM_GET_RETAIL_WITH_TAX, operation 'M'

```
# TAKLV = 0  ->  exempt item
if exempt or tax_percent == 0:
    return (incl, 0.0)
excl = incl / (1.0 + tax_percent)
```

TAKLV = 0 means EXEMPT. A flat /1.15 invents tax SAP never charged. The rate itself is data — A078/KONP MWST, keyed by country and date.

src/glue/glue_engine/abap/helpers.py:144-154

2 · A RULE, NOT A LOOKUP

resolve_store_type()

LZID8F02 get_sites — derived from WERKS

```
E*             ->  EXPRESS
S/N/M 301-303  ->  BULK SALES
S305 / S306    ->  QCOMMERCE
R399           ->  BAKERY PROD.
```

Derived from the plant code, not a lookup — so no site can go missing because a text row was absent.
D* → WAREHOUSE was added 2026-07-30.

src/glue/glue_engine/abap/helpers.py:157-221

3 · A BUG, KEPT ON PURPOSE

parse_pack_size()

byte-for-byte port of ABAP strip_maktg

```
buf40 = raw[:40].rjust(40)
cnt = 40
while cnt > 1:   # offset 0 unread
    cnt -= 1
```

Both passes scan a right-justified buffer from 39 down to 1. Offset 0 is never inspected — an off-by-one we kept, so output matches SAP.

src/glue/glue_engine/abap/helpers.py:256-329

THE RULE — WHEN TO REPRODUCE A SOURCE BUG ON PURPOSE

KEEP it when the wrong output is already the answer the business reconciles against — these pack sizes are printed on shelf labels.

DON'T keep it when the bug DROPS data: zsdcc's inner join lost 273,867 ZDSALES rows / SAR 2.29 bn, so Silver uses LEFT and records why.

Either way — write the divergence into the spec banner and pin it with a unit test, so nobody helpfully "fixes" it back.

IN PLAIN ENGLISH

A port is not a rewrite. Sometimes "correct" means "identical to the thing everyone already trusts" — quirks included. Just write down which you chose.

L11 · Porting ABAP Faithfully

Slide: `_render/L11-reproducing-source-logic.html`

The point

Silver's job is **fidelity**: a derived Silver table must be identical to the SAP view it re-implements. That makes "port the ABAP" a design decision on every function, not a mechanical translation. Three real examples, three different answers.

- `vat_split` — **the rate is data.** The ABAP FM `ZMM_GET_RETAIL_WITH_TAX` looks up the MWST condition (`A078` → `KONP.KBETR/1000`) by country *and date*, and it treats `MARA.TAKLV = 0` as **exempt — no split at all**. A flat `incl / 1.15` invents tax SAP never charged on exempt items, and it silently misprices every historical row from before the 15 % rate.
- `resolve_store_type` — **a rule, not a lookup.** Every site's type is *derived* from the WERKS plant code (`E*` → EXPRESS, `S/N/M` + 301-303 → BULK SALES, `S899` → DSD CONSOLIDATION, `R399` → BAKERY PRODUCTION…). There is no text table to join, so there is no row to be missing and no language to fan out. Porting it as a lookup would have invented a dependency SAP does not have.
- `parse_pack_size` — **a bug kept on purpose.** It is a byte-for-byte port of ABAP `strip_maktg` : both passes scan a right-justified fixed buffer from offset 39 (or 19) **down to 1**, so offset 0 is never inspected. That off-by-one is reproduced deliberately, because the goal is parity with what SAP actually prints — pack sizes that are already on shelf labels and in the customer's reports.

The transferable skill is knowing **which of the three you are looking at** before you write the function.

L11 · Porting ABAP Faithfully

Key ideas

- **Reproduce a source bug when its output is already the accepted answer.** Descriptions and pack sizes flow to the business as-is; a "fixed" parser would make every product description differ from SAP's and every reconciliation fail. Reproduce it, and say so in the docstring — `parse_pack_size` spells out the off-by-one, the discarded `6X` multiplier, and the empty-description single-word case as *reproduced*, not accidental.
- **Do not reproduce a bug that drops data.** `zsdcc` 's join is the counter-example. SAP's own QuickView uses an INNER join to `TVKMT` , and reproducing it discarded 273,867 ZDSALES rows (SAR 2,285,602,930.33 — 12.43 % of `ZAMT_SOLD`) for six dept codes with no English text row. Gold never reads Bronze again, so that loss is unrecoverable. The operator overrode ABAP parity: `join_type: left` , orphan depts arrive with `vtext` NULL, and the *scoping* decision moves to Gold (`dim_dept.include`).
- **Do not reproduce a bug that is a sequential artifact.** `id8_product` 's `components_per_unit` is a plain LEFT JOIN to MARM. The ABAP's mutable `w_umrez` can carry a **stale value from the previous material** when its own lookup misses — a work-area artifact, not a rule. Replicating it would attach a random unrelated material's ratio. The port returns NULL and documents the divergence.
- **Do not mis-encode a source *idiom* as a bug.** ABAP `SELECT-OPTIONS` means "no restriction" when empty; a pre-filled screen value is only a default. Encoding `scan_611` 's default as `fkart == 'FP'` dropped every non-POS billing document — 281,994.50 of sale on a single day, in Silver, unrecoverably (R40). `select_options()` now normalises the range and returns `[]` for "no filter".
- **Make the ported unit fail loudly, not quietly.** `resolve_join_type` raises on an unknown value instead of defaulting to `inner` , precisely so a typo cannot silently restore the 273,867-row drop. `resolve_extwg` returns the **raw** unknown code rather than NULL, so the dbt `accepted_values` test can see it (a NULL never fails a `NOT IN` predicate).
- **Every divergence lives in two places: the docstring and a test.** That is what stops the next person "fixing" it back.

Words you'll hear

WORD	WHAT IT MEANS HERE
Fidelity / parity	Silver reproduces the SAP view exactly, including its quirks
Decompiled ABAP	The real source under `docs/ABAP/src/` — `LZID8F01..F05` , the FMs
`TAKLV`	SAP tax classification on the material; `0` = exempt, no VAT split
`WERKS`	Plant / site code — 1 letter + 3 digits, e.g. `S106`
`MAKTG`	Material short text; the pack-size token is parsed off its tail
Work area	An ABAP mutable local that persists across loop iterations — the source of sequential artifacts
`SELECT-OPTIONS`	An ABAP selection range; **empty means unrestricted**, not "no rows"

L11 · Porting ABAP Faithfully

In this repo

- `src/glue/glue_engine/abap/helpers.py:144-154` — `vat_split` : exempt or zero rate passes through untouched; otherwise `excl = incl / (1 + rate)` . Called per item at `baskets.py:159-161` with `exempt=(taklv == "0")` .
- `helpers.py:157-221` — `resolve_store_type` , the whole prefix rule plus the honest notes: `D*` → WAREHOUSE **added** 2026-07-30 after a live cross-tab (the ABAP doc was silent on `D`), and the `301-303` / `197-199` ranges are "the plain reading, not a re-verified SAP fact".
- `helpers.py:256-329` — `parse_pack_size` , traced offset-by-offset against `LZID8F01.abap:199-275` . Read the docstring before the code.
- `helpers.py:17-39` (`resolve_join_type` , raises) and `:332-353` (`select_options` , the R40 story).
- `docs/ABAP/ID8-EXTRACTOR-MAPPING.md:239-249` — the decompiled VAT FM: `tax_code = 0 ⇒ exempt, no split` , rate from `KONP.KBETR/1000` . `:183` is the store-type prefix table; `:167-170` warns that `strip_maktg` is "fiddly string logic, not a column copy".
- `src/glue/glue_engine/abap/ops.py:648-655` — the disclosed `components_per_unit` simplification, in `id8_product_op` 's docstring.
- `tests/unit/test_phase3_abap_transform.py:48-65` (VAT incl. the exempt case), `:169-201` (store type, parametrized), `:439-497` (pack size: weight suffix, discarded `6X` multiplier, unmatched unit, `**PROMO**` stripping, single-word → empty description).

Do this

1. Run `parse_pack_size("SPRITE CAN 6X330ML", uom)` on paper. Where does the `6X` go, and which loop iteration drops it?
2. Change `vat_split` to always divide by 1.15 and run the unit tests. Then work out, in SAR, what that does to a day of exempt-item sales.
3. Open `zsdcc.yaml` 's banner and `helpers.py::resolve_join_type` together. Explain why a typo like `join_type: lft` **raises** instead of falling back to `inner` .
4. Pick any ported function and classify it: reproduce the bug, refuse the bug, or the source has no bug and you mis-read an idiom.

You've got it when you can…

…hold both sentences at once — **"we copied SAP's bug on purpose"** and **"we refused to copy SAP's bug"** — and say which rule decides between them: *does the divergence change a number people already reconcile against, or does it lose data we can never get back?*

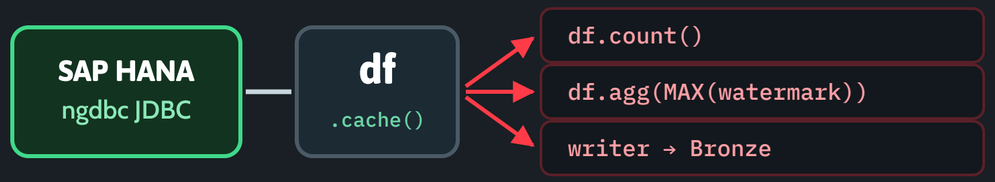
MODULE 2 · LESSON 12

Partitions, Shuffles, Skew, Cache

Four levers that decide whether a Glue job finishes in twenty minutes or dies at the 120-minute timeout.

1 · ONE PULL, THREE ACTIONS

EVERY ACTION RE-EXECUTES THE READ



Without .cache() each one re-issues the SELECT — 3× the SAP load.

`src/glue/glue_engine/sources/sap_hana.py:1151-1160`

2 · SPLIT THE READ, OR IT TIMES OUT

ONE CONNECTION IS NOT A PLAN

```
jdbc_hash_partitions: "8"      specs/download/sap_zhocidc.yaml
preds = ['"BUDAT" BETWEEN …', …]   # one JDBC connection each
self._read(spark, query, run_id, tag, predicates=preds)
```

R45 · the daily zhocidc delta ran on ONE connection: four consecutive Glue runs blew the 120-minute timeout and the cycle never reached its barrier.
More is not better — MBEW at 16 saturated HANA; tuned to 4 (≈9M rows each).

`sap_hana.py:800-833` · `specs/download/sap_mbew.yaml:33-36`

3 · WHAT applyInPandas ACTUALLY COSTS

THE SHUFFLE YOU ASKED FOR

groupBy(werks, zdate, zptno, zreceipt) moves EVERY ZHOCIDC row across the network, so one receipt's rows land together on one partition.
applyInPandas then materialises each group as a pandas DataFrame in a Python worker — the whole group in memory, serialised both ways.
That is the price of ordered logic. Pay it once, on the smallest key.

`src/glue/glue_engine/abap/ops.py:86`

SKEW, AND THE SECOND CACHE

A stage ends when its SLOWEST partition ends. One store-day with 10× the receipts holds the whole stage — that is skew, and no config setting fixes it.
folded.cache() exists because folded is consumed TWICE: once filtered to _kind='H', once to _kind='I'. Without it the entire fold runs twice.

Rule of thumb: persist any DataFrame you read more than once.

`src/glue/glue_engine/abap/ops.py:87-91`

IN PLAIN ENGLISH

A Spark job costs whatever its slowest slice costs. Read once, cut the work into even pieces, and never make the database do the same job three times.

L12 · Partitions, Shuffles, Skew, Cache

Slide: `_render/L12-spark-performance.html`

The point

Spark is lazy. A DataFrame is a *plan*, not data — and **every action re-runs the plan from the top** unless you tell it not to. Four consequences, all of them visible in this repo as real incidents.

- **`.cache()` before a multi-action sequence.** `read_incremental` does three things with one frame: `df.count()`, `df.agg(MAX(watermark))`, and the caller's write to Bronze. Without `df.cache()` that is **three full JDBC pulls** from SAP — three times the load, three times the cost, and non-determinism if the source moves between passes.
- **Partition the read, or one connection carries the whole table.** A JDBC read without `predicates` (or `partitionColumn`) is a **single connection**, regardless of what the spec says about parallelism.
- **A shuffle is the price of grouped or ordered logic.** `groupBy(...).applyInPandas(...)` moves every row across the network and materialises each group in a Python worker. Worth it for a receipt; ruinous for a key with a huge group.
- **Skew beats configuration.** A stage finishes when its *slowest* partition finishes. One store-day with 10× the receipts holds the whole stage, and no `numPartitions` setting fixes that.

L12 · Partitions, Shuffles, Skew, Cache

Key ideas

- **Actions, not transformations, cost money.** `filter` / `select` / `join` build the plan; `count`, `collect`, `first`, `write` execute it. Count the actions in a function and you have counted the source reads.
- **`.cache()` returns the same DataFrame** (it mutates the storage level), which is why `df.cache()` on its own line works. `MEMORY_AND_DISK` spills, so it is safe on the giants. In `read_incremental` the frame stays cached through the caller's write — `bronze_pull` owns the write and cannot unpersist from inside the connector, so Spark evicts at job end.
- **Two independent caches, same reason.** `sap_hana.py` caches the JDBC frame before count/agg/write; `ops.py` caches `folded` because it is consumed **twice** — once filtered to `_kind='H'`, once to `_kind='I'`. Without it the entire receipt fold executes twice.
- **How the delta gets partitioned.** `_incremental_predicates` turns `[watermark, today]` into N contiguous date sub-ranges (`_window_predicates`), one Spark JDBC `predicate` = one connection. The base query still carries `wm > watermark`, so the predicates only *split* rows it already selects — nothing missed, nothing double-counted. The **last sub-range stays open-ended** (R49) so future-dated billing-plan instalments are not truncated.
- **The real incident (R45).** `read_incremental` used to pass no predicates, so every daily delta ran single-connection and `jdbc_hash_partitions` was inert on that path. `sap.zhocidc` blew the 120-minute Glue timeout on four consecutive attempts, the cycle never reached its `download_barrier`, and the ASL retry burned ~10 h of a lane per cycle. The log named it: *"single-partition read with no expected_row_count declared."*
- **More partitions is not better.** MBEW at 16 saturated HANA's connect headroom → *"Cannot connect (socket timeout)"*; tuned to **4** (≈9 M rows per partition). VBRP runs 8, and the spec does the arithmetic: at Map `MaxConcurrency=4` that is 4 × 8 = 32 HANA connections, well under the ~160 ceiling. **Partitions × concurrent tables is the number that matters.**
- **The guard that catches the silent case.** `single_partition_max_rows` (default 5,000,000) fails configuration when a table declares an `expected_row_count` above the threshold and has *no* partitioned read — turning a job that would merely time out into one that refuses to start.
- **`applyInPandas` trade-offs.** Per group: a shuffle, an Arrow serialisation out, a Python process, and an Arrow serialisation back. It buys you arbitrary ordered logic in plain Python. Pay it on the smallest key that still contains all the state, and never on a key whose largest group does not fit in a worker.
- **Reading the symptom.** Job time roughly = *n* × single-read time → a missing cache. One executor busy, the rest idle → skew or a single connection. Stage retries with OOM inside a Python worker → an `applyInPandas` group that is too big.

L12 · Partitions, Shuffles, Skew, Cache

Words you'll hear

WORD	WHAT IT MEANS HERE
Action	An operation that actually executes the plan (`count` , `write` , `collect` , `first`)
Predicate pushdown	Spark hands the `WHERE` to the database; each JDBC `predicate` = one connection
Shuffle	Rows moved across the network so related rows share a partition
Skew	One partition far bigger than the rest; the stage waits for it
Persist / cache	Keep a computed frame so later actions don't recompute it (`MEMORY_AND_DISK`)
`jdbc_hash_partitions`	The spec's parallelism degree for a source read
Lane	One concurrent slot of the Step Functions Map that runs table downloads

In this repo

- `src/glue/glue_engine/sources/sap_hana.py:1151-1160` — the HIGH-14 comment and `df.cache()` : *"Without this the JDBC pull re-executes once per action — `df.count()` , the `df.agg(F.max())` watermark pass, AND the caller's downstream write."* Same fix at `:1266-1268` (`read_full`) and `:1314-1316` (`read_range`).
- `sap_hana.py:800-833` — `_incremental_predicates` and the full R45 write-up; `:718-772` — `_window_predicates` , the sub-range builder and the open-upper-bound rule.
- `sap_hana.py:367-410` — the `single_partition_max_rows` guard; the default lives at `:192` .
- `src/glue/specs/download/sap_zhocidc.yaml` — `jdbc_hash_field: SEQNO` , `jdbc_hash_partitions: "8"` , and the verified PK that justifies both.
- `src/glue/specs/download/sap_mbew.yaml:33-36` — 16 → 4 with the reason in the comment; `sap_vbrp.yaml:22-24` — 8 readers and the 4 × 8 = 32-connection arithmetic.
- `src/glue/glue_engine/abap/ops.py:86-91` — the `groupBy(...).applyInPandas(...)` line, `folded.cache()` , and the two filters that consume it twice.

Do this

1. In `read_incremental` , count the actions between the read and the return. Delete `df.cache()` in your head and state exactly how many times SAP is queried.
2. Take `sap.zhocidc` with `jdbc_hash_partitions: 8` and a 30-day delta. Write out the eight predicates `_window_predicates` produces. Which one is open-ended, and why must it be?
3. Work out the concurrent-connection budget for a cycle: partitions per table × Map `MaxConcurrency` . Which spec would you change first to stay under it?
4. In `basket_op` , remove `folded.cache()` on paper and trace what `_explode_json` triggers for each of the two outputs.

You've got it when you can…

…look at an unfamiliar Spark function and answer three questions without running it: **how many times does this execute the source read, where is the shuffle, and what is the biggest single group** — then point to the `.cache()` , the predicates and the group key that decide each one.

MODULE 2 · LESSON 13

Incremental on Redshift, Properly

merge + unique_key, a 45-day window that silently drops older restatements, and the dist/sort you will get wrong first.

1 · MERGE ON THE GRAIN

unique_key IS THE GRAIN

```
materialized='incremental',
incremental_strategy='merge',
unique_key=['site','date','aagm','scenario'],
on_schema_change='fail', dist='date', sort=['date','site']
```

unique_key MUST equal the model's real grain. Too wide and MERGE inserts duplicates; too narrow and Redshift raises a cardinality error.

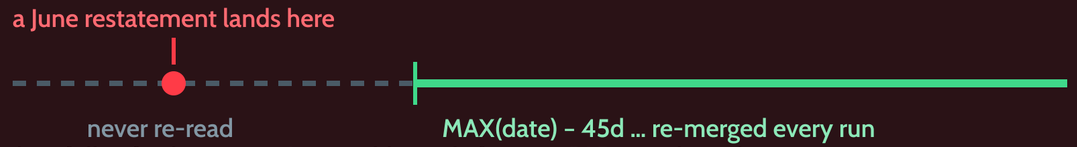
All-Dept rows carry aagm='__ALL__' — a NULL key never matches itself.

`src/dbt/models/marts/gold/unified_sales.sql:76-83`

2 · THE WINDOW — AND WHAT IT DROPS

THE 45-DAY REPROCESS WINDOW

```
WHERE date >= COALESCE(
  (SELECT MAX(date) - INTERVAL '45 days' FROM {{ this }}), '1900-01-01')
```



Only --full-refresh recovers it: monthly, and after any SAP month-end re-open.

`src/dbt/macros/scenario_helpers.sql:61-68 · var gold_reprocess_days`

3 · THREE SETTINGS THAT BITE LATER

on_schema_change='fail'

A new Silver column does not silently widen Gold — the build FAILS and a human decides.
M-28: every UNION leg pre-casts cc to NUMERIC(38,4) so it cannot drift.

`gold/unified_customer_count.sql:36-42`

LATE-BINDING VIEWS

Staging selects from an EXTERNAL schema, silver_external. A normal Redshift view refuses: "External tables are not supported in views".
+bind: false → NO SCHEMA BINDING.

`src/dbt/dbt_project.yml:63-72`

DIST / SORT

dist='date' co-locates a day; sort=['date','site'] makes range scans cheap — right for BI by date.
Wrong the moment you join on site: unified_sales_by_am reshuffles it.

`gold/unified_sales_by_am.sql:47-49`

IN PLAIN ENGLISH

"Incremental" means "only redo the recent part". That is a promise about time — so anything the source corrects outside the window stays wrong until you rebuild.

L13 · Incremental on Redshift, Properly

Slide: `_render/L13-dbt-incremental-depth.html`

The point

Every Gold fact in this project is `materialized='incremental'`, `incremental_strategy='merge'`. That is four separate promises, and each one has a failure mode.

- `unique_key` **is the grain.** dbt turns it into the MERGE's `ON` predicate. Too wide → the match never fires and rows re-insert; too narrow → several source rows match one target row and Redshift raises a cardinality error.
- **The incremental predicate is a *time window*, and it drops what falls outside.** `gold_incremental_predicate` re-processes the trailing `gold_reprocess_days` (45) only. A SAP posting restated 60 days back is never re-read, so Gold keeps the stale value — **no error, no log line**.
- `on_schema_change='fail'` means Gold never silently absorbs a Silver schema change. Good — provided the model's own column *types* are pinned, or the build fails on its own union.
- **Staging must be late-binding** because it reads an external schema, and `dist` / `sort` are right for the query you designed for and wrong for the next one.

L13 · Incremental on Redshift, Properly

Key ideas

- **Write the grain in the header, then make `unique_key` equal it.** `unified_sales` is Site × Date × Dept × Scenario → `unique_key=['site','date','aagm','scenario']`. `unified_distress` carries a seven-column key for exactly this reason, and its header says so: *"The `unique_key` MUST equal this grain or the incremental MERGE hits Redshift's…"* cardinality rule.
- **NULL is not a key.** `NULL = NULL` is false in a MERGE predicate, so the synthetic All-Dept rows use the sentinel `aagm = '__ALL__'`. With a NULL they matched nothing and re-inserted on **every** incremental run, inflating the table forever (W4.c).
- **The window is a backstop, not a substitute for a rebuild.** M-26 widened it from a hardcoded 14 days to a configurable 45 (`var('gold_reprocess_days')`) because SAP corrections routinely land past 14 days. Anything older is only recovered by `dbt run --select <model> --full-refresh`, on the documented monthly cadence and always after a SAP month-end re-open. Ops can widen it per run with `--vars`; they cannot widen it retroactively.
- **The predicate constrains the *shape* of your final SELECT.** The macro appends `WHERE date >= …` to the end of the model, and `GROUP BY` immediately followed by `WHERE` is a syntax error. So aggregation lives in a CTE and the model ends with a bare `SELECT * FROM …` — see the comment in `unified_sales_by_am`.
- **`on_schema_change='fail'` + UNION ALL = pin your types.** M-28: `cc` was BIGINT on the actuals legs and NUMERIC on the budget leg. A first build with only actuals materialised BIGINT, and the next run failed the moment the budget leg widened it. Fix: pre-cast **every** leg to `NUMERIC(38,4)` so the resolved type cannot drift between runs.
- **Late-binding views are not a style choice.** A plain Redshift view cannot select from an external schema — *"External tables are not supported in views"*, observed 2026-06-05. The whole staging layer is therefore `+materialized: view` + `+bind: false` (`WITH NO SCHEMA BINDING`). The trade-off: nothing validates the source exists until query time.
- **`dist` / `sort` encode an assumption about the query.** `dist='date'` + `sort=['date','site']` is right for BI filtered by date. It is wrong the moment you join on something else: `unified_sales_by_am` joins `unified_sales` to the AM map **on** `site`, so Redshift redistributes the table for that model. Low-cardinality or heavily-skewed dist keys are worse still — they concentrate rows on a few slices.
- **A silent drop deserves a loud test.** The budget CTE filters on `d.include` — the *right* side of a LEFT JOIN — so an unmatched dept is dropped with no error. It drops 0 rows today; `assert_budget_upload_depts_all_resolve.sql` is what keeps that true. The comment explicitly says *don't* "fix" it into an INNER JOIN: same semantics, and the test is the guard.

L13 · Incremental on Redshift, Properly

Words you'll hear

WORD	WHAT IT MEANS HERE
Incremental model	Build once, then only merge new/changed rows on later runs
`unique_key`	The column list dbt matches on in the MERGE — i.e. the grain
Cardinality error	One target row matched by several source rows in a MERGE
Incremental predicate	The `WHERE` dbt appends to bound how much history a run re-reads
Full refresh	`--full-refresh` — drop and rebuild, the only way to fix pre-window history
Late-binding view	`WITH NO SCHEMA BINDING`; resolves its source at query time, not create time
`dist` / `sort`	Redshift: how rows spread across slices, and their on-disk order

In this repo

- `src/dbt/dbt_project.yml:88-93` — the Gold defaults: `incremental` + `merge` + `on_schema_change: fail`, applied to every fact. `:63-72` — the staging block with `+bind: false` and the 2026-06-05 failure recorded above it. `:50-54` — `gold_reprocess_days: 45` with the M-26 rationale.
- `src/dbt/macros/scenario_helpers.sql:44-68` — `gold_incremental_predicate`: `MAX(date) - INTERVAL 'N days'`, `COALESCE` to `1900-01-01` on the first run, and the paragraph stating plainly that corrections older than the window need a full refresh.
- `src/dbt/models/marts/gold/unified_sales.sql:76-83` — the config block; `:63-70` — why the `__ALL__` sentinel exists; `:282` — the predicate as the model's last line.
- `src/dbt/models/marts/gold/unified_sales_by_am.sql:70-73` — "aggregation lives in the CTE so the final SELECT has NO trailing GROUP BY"; `:47-49` — the join on `site` against a `dist='date'` table.
- `src/dbt/models/marts/gold/unified_customer_count.sql:36-42` — M-28, the `NUMERIC(38,4)` pre-cast on every leg.
- `src/dbt/models/sources.yml:19-22` — the `silver_external` external schema every staging view reads.
- `src/dbt/models/marts/gold/unified_distress.sql:12,62-65` — a seven-column `unique_key` and the note that it must equal the grain.

Do this

1. Compile `unified_sales` (`dbt compile --select unified_sales`) with and without `is_incremental()`. Read the generated MERGE and find your `unique_key` in it.
2. Pick a date 60 days back and ask: if SAP restated that day tonight, which run fixes Gold? Write the exact command.
3. Drop `'scenario'` from `unified_sales`' `unique_key` on paper. Which two rows now collide, and is the result a duplicate or a cardinality error?
4. Add a column to a Silver table and predict the failure under `on_schema_change='fail'`. Then decide what the right response is — is it ever "change it to `append_new_columns`"?

You've got it when you can…

…answer *"Gold shows the wrong number for a day in June"* with the right first question — **is that day inside the 45-day window?** — and then name the three settings that could each have caused it: the `unique_key`, the incremental predicate, and the last time anyone ran `--full-refresh`.

MODULE 2 · LESSON 14

Two Bugs That Doubled the Numbers

Both were silent. Both produced a plausible number that was exactly twice the truth. Both are now pinned by a test.

1 · THE LANGUAGE FAN-OUT

JOIN A TEXT TABLE ON ITS KEY — GET ITS LANGUAGES

ZDSALES
KTGRM = '12'
sale = 1,000

SPRAS='E' GROCERY
SPRAS='A' (Arabic)
SPRAS='D' (German)

2 ROWS OUT
sale = 2,000
every measure ×2

```
right_where: SPRAS = 'E'    # REQUIRED — restores 1:1
```

TVKMT is language-keyed (MANDT, SPRAS, KTGRM): 43 EN · 43 AR · 3 DE rows.

`src/glue/specs/transform/zsdcc.yaml:22-30, 59`

2 · THE SYNTHETIC 'ALL DEPT' ROW

ONE TABLE, TWO GRAINS, ON PURPOSE

GROCERY	400
FRESH	350
BAKERY	250
All Dept (aagm='__ALL__')	1,000

SUM(sale)
across ALL rows
= 2,000
every KPI ×2

```
WHERE dept != 'All Dept'    -- CRIT-04, in every roll-up
```

Both grains are wanted: Power BI filters [Dept] = "All Dept" directly.

`gold/unified_sales.sql:159-188 · unified_sales_by_am.sql:62-66`

3 · WHAT STOPS THEM COMING BACK

GRAIN DISCIPLINE

Write the grain in the header, then make unique_key equal it. A table with two grains must say so, loudly.

`gold/unified_sales.sql:6-10`

THE TEST FOR BUG 2

The All-Dept row must equal SUM of the per-dept rows for that site+date. Tolerance 1.0. Zero rows = pass.

`tests/assert_all_dept_reconciles_to_per_dept.sql`

THE TEST FOR BUG 1

A unit test pins the SPEC, not the data: the diagnostic op must use the SAME SPRAS as zsdcc's right_where.

`tests/unit/test_phase3_abap_transform.py:797`

IN PLAIN ENGLISH

Two ways to count the same money twice: matching a row to a translation of itself, and adding a subtotal back into its own total.

L14 · Two Bugs That Doubled the Numbers ★

Slide: `_render/L14-correctness-traps.html`

The point

Both bugs shipped a number that looked completely normal — no error, no NULLs, no failed job — and was **exactly twice** the truth. They come from the two ways a grain can go wrong.

- **Bug 1 — the language fan-out (a join that multiplies rows).** `TVKMT` is a *language-keyed* text table: its primary key is `(MANDT, SPRAS, KTGRM)`. `KTGRM` (the dept code) is unique only **within** a language slice. Joining `ZDSALES ⋈ TVKMT` on `KTGRM` alone matches the English row **and** the Arabic row for the same dept, so every sales row comes out twice — doubling `LC` and `CC` silently. In Bronze client 100: 43 EN · 43 AR · 3 DE.
- **Bug 2 — the synthetic `'All Dept'` row (a table with two grains).** `gold.unified_sales` deliberately contains **both** per-dept rows *and* an All-Dept rollup that is their sum, so Power BI's `[Dept] = "All Dept"` measures resolve directly. Any `SUM(sale)` across the dept axis that does not exclude it therefore counts every riyal twice.

Bug 1 is fixed by **restricting the join** (`right_where: SPRAS = 'E'`). Bug 2 cannot be "fixed" — the extra grain is wanted — so it is handled by **discipline plus a test** (`WHERE dept != 'All Dept'` in every roll-up).

L14 · Two Bugs That Doubled the Numbers ★

Key ideas

- **Any join to a text/description table is a fan-out risk.** Ask one question every time: *what is this table's real primary key, and is my join key the whole of it?* If not, restrict the other key columns before joining (`right_where`), and project away the duplicates (`right_select` drops `MANDT`). SAP's own QuickView does the same thing on its selection screen (`TVKMT~SPRAS in <sel>`).
- **The same bug bit a second table, and was caught by the same question.** `DD07T` is keyed `(DOMNAME, DDLANGUAGE, AS4LOCAL, VALPOS, AS4VERS)` ; `domvalue_l` is not in that key. Joining on the value alone doubled customers: measured live on QA 2026-07-24, DD07T held 14 rows for 7 `ZSEGMENT` values, so 6,165,640 of 6,898,089 active customers were emitted twice and Silver carried 13,063,729 rows. The fix is the same shape — filter to one language *and* the active DDIC version, then keep one row per value deterministically.
- **A restriction is not always safe by itself.** Filtering `TVKMT` to `SPRAS='E'` and keeping the INNER join dropped 273,867 ZDSALES rows (SAR 2,285,602,930.33) for six dept codes with no English text row — unrecoverable, because Gold never re-reads Bronze. So `zsdcc` uses `join_type: left` : fidelity of *rows* wins, and the six orphan depts arrive with `vtext` NULL. Reporting scope stays in Gold (`dim_dept.include`).
- **Two grains in one table is a legitimate design — if it is written down.** `unified_sales` declares its grain and its six UNION-ALL legs in the file header. The All-Dept legs use `SUM(ZSDCC.sale)` for sale but `ZSCC.cc` for the customer count, because summing per-dept `cc` would double-count a basket that touched three departments. That is a *third* grain trap, handled in the same place.
- **CRIT-04 is the recurring instance.** Every roll-up of `unified_sales` must filter `dept != 'All Dept'` — `unified_sales_by_am` does it in the CTE, with the reason in a comment. Forget it once and every AM/store KPI (`sale_total` , `cc_total`) roughly doubles.
- **Test the invariant, not the symptom.** `assert_all_dept_reconciles_to_per_dept` asserts the All-Dept row equals `SUM(per-dept)` for the same `(site, date, scenario)` , tolerance 1.0, actuals only (budget All-Dept rows are preserved verbatim from Excel, not synthesized). `recon_unified_sales_has_synthetic_all_dept` asserts the opposite direction — that the rows still *exist*, so DAX doesn't go blank.
- **Test the config, not just the data.** The guard against Bug 1 coming back is a **unit test on the spec**: the diagnostic op must use the same `SPRAS` that `zsdcc.yaml` 's `right_where` applies, and `join_type` must be declared `left` explicitly rather than inherited from a default.
- **Know which invariants you *cannot* assert.** `SUM(per-dept cc) >= All-Dept cc` reads as though it must hold — and SAP does not honour it (A048 on 2026-02-06: per-dept 746 vs store 3,935). Asserting it would fail forever on correct data, and a permanently red test is no signal.

L14 · Two Bugs That Doubled the Numbers ★

Words you'll hear

WORD	WHAT IT MEANS HERE
Grain	What one row means — the level a fact table is stored at
Fan-out	A join that returns more rows than the driving table has
Language-keyed table	An SAP text table with `SPRAS` in its primary key (`TVKMT` , `MAKT` , `T023T` , `DD07T` …)
`SPRAS`	SAP language key — `E` English, `A` Arabic, `D` German
`KTGRM` / `AAGM`	The department code; `norm_aagm` strips SAP's zero-padding
Sentinel	`'All Dept'` / `'__ALL__'` — a deliberate non-NULL marker value
CRIT-04	This project's name for the All-Dept double-count class of bug
Conformed dimension	One `dim_site` / `dim_dept` shared by every fact, so totals can be compared

In this repo

- `src/glue/specs/transform/zsdcc.yaml:22-30` — the 🔴 FAN-OUT banner with the live counts and the sentence *"joining on KTGRM ALONE… MULTIPLIES every ZDSALES row ~2x — silently DOUBLING LC and CC"*; `:32-48` — the Item 8 LEFT-join decision; `:56-62` — `right_where` / `right_select` / `join_type` as actual spec parameters.
- `src/glue/glue_engine/abap/ops.py:121-127` — `flat_join_op` 's docstring, the general rule ("pre-filter the right side to the grain the join key implies"); `:347-355` — the DD07T fan-out with the measured customer numbers.
- `src/dbt/models/marts/gold/unified_sales.sql:6-10` — the grain and the six legs, declared in the header; `:119-157` — the All-Dept rollup CTEs and why `cc` comes from ZSCC, not from summing ZSDCC; `:159-188` — `all_dept_cy` with the `__ALL__` sentinel.
- `src/dbt/models/marts/gold/unified_sales_by_am.sql:62-66` — the CRIT-04 exclusion, with the reason written next to it.
- `src/dbt/tests/assert_all_dept_reconciles_to_per_dept.sql` — the reconciliation test, including *why* budget rows and CC-less store-days are out of scope.
- `src/dbt/tests/assert_all_dept_cc_equals_customer_count.sql` — the `cc` half, and the honest section on the inequality that is *not* asserted.
- `src/dbt/tests/recon_unified_sales_has_synthetic_all_dept.sql` — the guard in the other direction.
- `tests/unit/test_phase3_abap_transform.py:756-800` — the two spec-level unit tests: `join_type` must be explicitly `left` , and the diagnostic's `spras` must equal the language in `right_where` .

Do this

1. For every join in `_material_attrs` (`ops.py`), name the right-hand table's primary key and say why the join cannot fan out. Two of them are only safe because of a `SPRAS = 'E'` filter — find them.
2. Run `SELECT SUM(sale) FROM gold.unified_sales WHERE scenario = '2026 A'` and then the same with `AND dept != 'All Dept'` . Explain the ratio before you look.
3. Delete `WHERE dept != 'All Dept'` from `unified_sales_by_am` and predict which dbt test fails and which one doesn't. (Only one of them looks at AM grain.)
4. Write the fan-out guard you would add for a *new* text-table join — the SQL that returns rows only when the right side is not 1:1 on the join key.

L14 · Two Bugs That Doubled the Numbers ★

You've got it when you can...

...do two things on sight: point at any join and state the right-hand table's primary key from memory, and point at any `SUM()` over a fact table and say **which grain it is summing** — including whether that table contains its own subtotals.

MODULE 2 · LESSON 15

What Decides Today's Work

One cron fires. One Lambda decides. Everything after that is just execution — and one hard 256 KB ceiling.

1 · FROM CRON TO GLUE — WHO DOES WHAT

EVENTBRIDGE
one rule · one cycle
00:00 KSA · no DST
`cron(0 21 * * ? *)`

→

DISPATCHER LAMBDA
READS bronze-mapping + catalog
groups enabled rows by source_id
`lambdas/dispatcher/handler.py`

→

CYCLE MANIFEST
WRITE, before anything
expected[] = must finish
`cycle#2026-08-10`

→

STEP FUNCTIONS
one per source · Map over
$.tables · barrier as the tail
`per_source_ingestion.tf:55`

→

GLUE JOBS
source_download (P1)
bronze_pull (P2)
`glue_sources.tf`

2 · ONE LAMBDA, FOUR KINDS OF RUN — THE PAYLOAD DECIDES

SCHEDULED — the daily cycle

`payload {} — no keys at all`

cycle_id is today's UTC date. Every enabled bronze-mapping row, exactly one item each.

`handler.py:613 · _dispatch`

INITIAL — one-time bulk load

`{"initial_load":true,"source":"sap"}`

5-year horizon split into month chunks. Its OWN init-... cycle, full_refresh=True.

`handler.py:1417 · _chunk_windows:1306`

RERUN — same-day recovery

`{"rerun":true,"cycle_id":"2026-08-10"}`

Reopens the cycle, releases the gold and conform locks, unique name -rHHMMSS.

`handler.py:1027 · rerun_download = P1 only`

BACKFILL — a date window

`{"rerun":true,"source":"sap","date":…}`

Its own bf-... cycle. Watermark untouched. Gold rebuilds with --full-refresh.

`handler.py:1701 · gold_barrier.py:246`

3 · THE LIMIT THAT BITES

256 KB · THE HARD EDGE

StartExecution rejects an input over 262,144 bytes.

```
55 tables x 11 windows  =  605 items
605 items x ~441 bytes  =  266,688 B
ValidationException     →  started: []
```

FIX 1 · slim the item — a P1 item carries no Silver keys: ~165 bytes saved on every one.

FIX 2 · widen the chunk: chunk_months 12 → 330 items.

Same 256 KB caps a Map result → ResultPath: null.

`handler.py:426-453 · TICKET-initial-load-sfn-input-size.md`

IN PLAIN ENGLISH

The cron is a doorbell — it carries no instructions. The dispatcher reads the config, writes down today's to-do list, and only then starts anyone working.

L15 · What Decides Today's Work

Slide: `_render/L15-dispatcher-step-functions.html`

The point

The cron carries no instructions. It is a doorbell.

Everything about *what runs tonight* is decided in one Python function — `_dispatch` in the dispatcher Lambda — and written down in DynamoDB **before a single Glue job starts**. Step Functions and Glue are pure execution; they never choose anything. If you want to know why a table ran (or didn't), you read the dispatcher and the cycle manifest, not the state machine.

Three things to take away:

1. **One EventBridge rule → one dispatcher → one cycle manifest → one SFN per source.** The cadence bands (hourly/daily/weekly) were collapsed into a single daily fire.
2. **The payload chooses the mode.** Same Lambda, five entry points: scheduled, `initial_load`, `rerun`, `rerun_download`, and `rerun` +dates (backfill).
3. **`StartExecution` will not accept an input over 262,144 bytes**, and at real scale the default initial load walks straight into that wall. That is not trivia — it is the reason `chunk_months` exists as an event override.

L15 · What Decides Today's Work

Key ideas

- **The manifest is written first, deliberately.** `put_cycle` runs *before* any `start_execution`, "so a fast source SFN can't reach its barrier before the manifest exists". It is a **create-only** write (`ConditionExpression="attribute_not_exists(pk)"`), so a double-fire of the cron cannot resurrect a concluded cycle or reset `started_at` (which would restart the sweeper's deadline clock).
- **`expected[]` is a promise, not a wish.** A source that is enabled in `bronze_mapping` but has no configured SFN ARN is *excluded* from `expected[]` and reported as a failure — because a table in `expected[]` that can never run makes the barrier wait forever and silently downgrades every cycle to `gold_skipped`.
- **Execution names are deterministic on purpose:** `<source>-<cycle_id>`. A second cron fire the same day collides with the existing name and no-ops. But Step Functions **reserves execution names for 90 days regardless of state**, so `ExecutionAlreadyExists` does *not* mean "still running". Treating it as such made every re-dispatch of an ABORTED/FAILED cycle a silent no-op for 90 days (observed on QA 2026-07-31). The fix: read the prior execution's status, and if it is terminally dead, re-dispatch under `<name>-rHHMMSS`.
- **Month-chunking exists because Glue jobs have a wall clock.** A 5-year initial load is split into calendar-aligned windows (`_month_windows` → `_chunk_windows`), one `tables[]` item per (table × window), each its own resumable `source_download` run. Widen the chunk and you get fewer Glue startups (~75 s each) but bigger, riskier pulls; narrow it and you get resumability but startup dominates wall-clock.
- **Not every table may be windowed.** A table whose `driver_by_mode` has `full` but no `incremental` has no usable date watermark — a date predicate matches *everything*, so each "window" re-pulls the whole table. Proven from live run rows: `sap.konp` returned 371,946,132 rows for each of 3 windows. Those tables get one un-windowed chunk instead. The exception is a table that explicitly opts in via `chunk_full_reload` (e.g. `sap.vbrp`, 684 M rows), whose watermark *is* a real date.
- **The 256 KB ceiling, concretely.** 55 SAP tables × 11 six-month windows = **605 items ≈ 266,688 bytes** → `ValidationException`, nothing started, and the response still said `"status": "initial_load_started"` with `started: []`. Two mitigations shipped: the P1 item was slimmed (it no longer carries `spec_path` / `enable_silver` / `silver_spec_path` / `silver_table` — those belong to P2 and are rebuilt by the download barrier), saving ~165 bytes per item; and `chunk_months` is settable per event.
- **There is a second, different 256 KB limit.** A `Map` state's *aggregated result* also has to fit in the 256 KB state payload. Keeping the Glue `startJobRun.sync` response made a 300-item Map blow `States.DataLimitExceeded` after every chunk had succeeded. Hence `ResultPath: null` on every Task in every state machine — the barrier reads DynamoDB, never the Step Functions payload.
- **Lane interleaving.** Lane assignment is by *table name*, and the Map's in-flight set is a sliding window of consecutive items — so a table-major build put the whole in-flight window on one Glue lane. Round-robining the items across lanes turned a serial 605-chunk load back into a parallel one.

L15 · What Decides Today's Work

Words you'll hear

WORD	WHAT IT MEANS HERE
Cycle	One logical run of the whole pipeline. `cycle_id` is the UTC date for a scheduled run, or `init-…` / `bf-…` / `rebaseline-…` for a scoped one
Cycle manifest	The `cycle#<id>` row in the coordination table: what was launched, and what state the cycle is in
`expected[]`	The tables the barriers wait on. Only tables that were actually launched go in it
P1 / P2	P1 = `source_download` (remote → raw S3). P2 = `bronze_pull` → `bronze_to_silver` (raw → Iceberg)
Dispatch / fan-out	Starting one Step Functions execution per source with that source's table list as input
Chunk / window	One `(table, date-range)` item. A 5-year load is many chunks per table
Gate 0	The rule that a table may not join the daily CDC cycle until its one-time initial load has completed
`run_kind`	Provenance stamped on every run row: `scheduled` / `initial` / `cdc` / `rerun` / `backfill`

In this repo

- `src/lambdas/dispatcher/handler.py:578-951` — `_dispatch`: scan enabled `bronze_mapping`, group by `source_id`, build the launch plan, `put_cycle`, fan out. Read the mode routing at `:591-608` first — it is five `if`s and it is the whole API.
- `:426-477` — `_make_table_item`, whose docstring carries the 256 KB arithmetic and explains exactly which keys a P1 item is *not* allowed to carry.
- `:1285-1322` — `_month_windows` / `_chunk_windows`. Twenty lines; read them and you know exactly what a chunk is.
- `:161-207` — `_is_full_only`, and the measured evidence for why a full-only table must not be windowed.
- `:972-1024` — `_start_source_sfn` and the 90-day execution-name trap.
- `:77-116` — the three tuning knobs: `INITIAL_LOAD_LOOKBACK_YEARS`, `INITIAL_LOAD_CHUNK_MONTHS`, `FULL_RELOAD_CHUNK_MONTHS`. The comment on the last one is a small masterclass in why you measure a distribution instead of assuming one.
- `infra/env/dev/per_source_ingestion.tf:43-227` — the P2 state machine: `Map` over `$.tables`, `MaxConcurrency 10`, per-item `Catch → ItemFailed → Succeed` (failure isolation), `ResultPath: null` everywhere, then the barrier tail at `:201-222`.
- `infra/env/dev/source_download.tf:19-135` — the P1 state machine: 24 h envelope, `MaxConcurrency 6`, and the lane-spread `JobName.$` trick.
- `infra/modules/dispatcher_lambda/eventbridge.tf:11-47` + `variables.tf:134-138` — one rule, `cron(0 21 * * ? *)` = 00:00 KSA (KSA has no DST, so the offset is fixed). Note `schedule_enabled` — the rules exist in Dev but are DISABLED, so Dev is manual-only.
- `docs/TICKET-initial-load-sfn-input-size.md` — the 256 KB incident written up properly, including why batching by `tables` is *not* a safe workaround (it fragments one logical load into several cycles).

L15 · What Decides Today's Work

Do this

1. Invoke the Dev dispatcher with `{}` and read the response body. Name every field: `expected_count`, `started`, `failures`, `gate0_blocked`. Then find the `cycle#<today>` row in `tamimi-lakehouse-coordination-dev` and check `expected[]` matches.
2. Compute the input size yourself. Take today's `tables[]` list from the response, `json.dumps` it, and print `len(...)`. Now multiply by the number of chunks a 5-year load would create. Decide what `chunk_months` you would pass.
3. Re-invoke the dispatcher twice within a minute. Prove to yourself that nothing ran twice, and say **which** of the two mechanisms stopped it (the deterministic execution name, or `put_cycle`'s create-only condition). Then answer: what would have happened if last night's execution had been *aborted*?
4. Open `per_source_ingestion.tf` and find every `ResultPath = null`. For each one, say what would have been in the payload if it had been kept.

You've got it when you can…

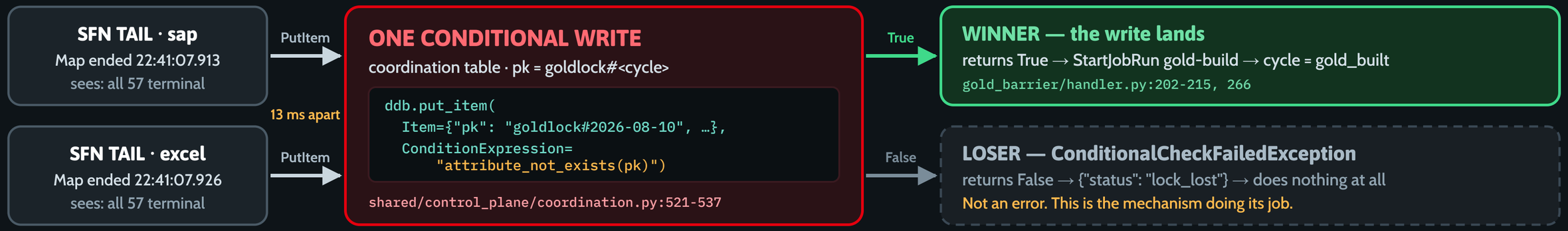
…take a payload you have never seen — say `{"initial_load": true, "source": "sap", "chunk_months": 3}` — and predict, before running it: which cycle_id it creates, roughly how many Glue runs it will produce, whether the SFN input will fit in 256 KB, and which barrier will decide when it's finished.

MODULE 2 · LESSON 16

How Two Jobs Don't Collide

Every source's Step Function ends by calling the same barrier. They can arrive together. Exactly one of them acts.

1 · THE RACE, DRAWN — TWO CALLERS, ONE WRITE



DynamoDB serialises writes to a single item — only one caller can ever find it absent. No queue, no leader election, no timeout to tune.

2 · WHAT A BARRIER DECIDES, IN THIS ORDER

CONCLUDE A PHASE = RESOLVE THE LATEST STATUS PER TABLE

1. Has the cycle already concluded? → only TERMINAL states no-op (R31).
2. Is every expected table terminal? → its LATEST run row, by started_at.
3. Did it truly reach Bronze? → a SUCCEEDED bronze_pull row; chunks ≥ planned.
4. Claim the lock, THEN judge → any FAILED table = gold_skipped + alert.

`coordination.py:598-628 latest_run_by_table · :643-655 tables_missing_bronze`

3 · THE R31 FIX

WHY "== running" WAS WRONG

```
- if cycle.state != "running":  # no-op
+ if cycle.state in CONCLUDED_CYCLE_STATES:
```

A cycle ADVANCES: running → download_built → transform_built → gold_built. Gating on "running" no-opped every barrier after the download gate.

`coordination.py:52-61 · gold_barrier/handler.py:142`

IN PLAIN ENGLISH

Two people reach for the same latch. It turns once. Whoever turns it does the work — the other walking away is the design working, not failing.

L16 · How Two Jobs Don't Collide ★

Slide: `_render/L16-barrier-pattern.html`

The point

Every per-source Step Function ends by calling the same barrier Lambda. On a busy night two of them finish milliseconds apart, both read the same manifest, both see the same all-green picture, and both decide "I should start Gold now."

Exactly one of them does.

Not because of a queue, not because of a leader election, not because of a lock service with a lease you have to tune — but because of **one conditional write to one DynamoDB item**:

```
client.put_item(
    TableName=coordination_table,
    Item={"pk": "goldlock#2026-08-10", ...},
    ConditionExpression="attribute_not_exists(pk)",
)
```

DynamoDB serialises writes to a single item. Whichever request the service orders first finds the item absent and succeeds; the second one gets `ConditionalCheckFailedException`, the helper returns `False`, and that invocation returns `{"status": "lock_lost"}` and does nothing at all. **That is not an error path. That is the mechanism working.** Half the barrier invocations in a healthy pipeline end in `lock_lost`.

L16 · How Two Jobs Don't Collide ★

Key ideas

- **Single-flight = "at most one caller acts", not "exactly one caller runs".** Every phase has its own lock item in the same coordination table, all with identical code: `downloadlock#` , `transformlock#` , `conformlock#` , `goldlock#` . Four locks, one pattern.
- **A barrier is edge-triggered and therefore fragile on its own.** It is invoked once, as an SFN tail Task. If it dies, nothing re-checks — which is exactly why the `cycle_sweeper` exists (L18). The lock is what makes re-checking *safe*.
- **Order matters: hold before you claim.** A barrier that is merely *waiting* must return **before** touching the lock. A claimed lock can never be re-claimed, so claiming one and then discovering you can't proceed wedges the cycle permanently. Both the R22 completeness hold and the R80(d) chunk hold sit deliberately above the `try_claim_*` call, mutating nothing.
- **Release is the escape hatch.** If the claim succeeds but the *start* then fails (Glue rejects `StartJobRun` , an SFN won't start), the barrier deletes the lock and re-raises, so the SFN Task retry or the sweeper can re-attempt. If the start *succeeded* and only the bookkeeping write failed, it must **not** release — that would risk a second build.
- **Concluding a phase = resolving latest-status-per-table.** One table has several run rows per cycle: one per stage (`source_download` , `bronze_pull` , `abap_transform` , `bronze_to_silver`), plus one per retry. "The table's status" is only well-defined as *the most recent attempt*, resolved by `started_at` (ISO-8601 sorts lexically, so a string compare is a chronological one). Never rely on DynamoDB query ordering.
- **Terminal ≠ done — the R22 lesson.** Terminality is satisfied by *any* stage's row. A table whose `bronze_pull` never wrote a row at all still resolves to its SUCCEEDED P1 `source_download` row and reads as finished. That is how QA cycle `sap-init-151805` concluded SUCCEEDED with **27 of 51 tables actually bronzed**. The gate now demands positive evidence: a SUCCEEDED `bronze_pull` row for every expected base table.
- **Latest ≠ complete — the R80(d) lesson.** A chunked table is *many* runs sharing one name. Collapsing it to its latest run passed `sap.vbrp` on QA cycle `init-2021-08-09_2026-08-09-212758` when five of its ten windows had been dropped. So the manifest carries `download_expected_chunks` — counted off the very items that were dispatched, never recomputed from the windowing rules — and the download barrier gates on `succeeded ≥ planned` .
- **The R31 fix — gate on TERMINAL states, not on `== "running"` .** The cycle *advances*: `running → download_built → transform_built → gold_built` . Every barrier used to no-op unless `state == "running"` , so the moment the download barrier set `download_built` , every downstream barrier declared the cycle "already concluded" and silently stopped. Silver and Gold never built. The fix is a set, not a string:

```
CONCLUDED_CYCLE_STATES = frozenset({
    "complete", "download_skipped", "transform_skipped",
    "gold_skipped", "gold_built",
})
```

Proceeding on an intermediate state is safe **precisely because** the single-flight lock makes it idempotent. The lock is what buys you the right to be relaxed about the state check. - **All-or-nothing is the verdict, not the gate.** Once a barrier holds the lock: any FAILED table → set the cycle `*_skipped` , publish an SNS alert, and do not start the next phase. A partial raw landing must never be loaded into Bronze; a partial Bronze must never build Gold.

L16 · How Two Jobs Don't Collide ★

Words you'll hear

WORD	WHAT IT MEANS HERE
Barrier	A Lambda invoked at the tail of a phase that decides whether the *whole* phase is finished
Single-flight	At most one caller performs the action, enforced by a conditional write
Conditional write	`PutItem` with `ConditionExpression="attribute_not_exists(pk)"` — succeeds only if the item isn't there
`ConditionalCheckFailedException`	The loser's answer. Caught, converted to `False`, and treated as a normal outcome
Edge-triggered / level-triggered	Fires once on an event / re-evaluates the world on a timer. Barriers are the first; the sweeper is the second
Terminal	`succeeded` | `failed` | `cancelled` — the run is over, whatever the outcome
Concluded	A cycle state from which no barrier may proceed (`gold_built`, `*_skipped`, `complete`)
All-or-nothing	Any failed table in the phase → the next phase does not start
`lock_lost`	The healthy no-op returned by every barrier invocation that did not win the claim

In this repo

- `src/shared/shared/control_plane/coordination.py:521-537` — `try_claim_gold_lock`. Sixteen lines, `try` / `except ConditionalCheckFailedException` / `return False`. Compare with `try_claim_download_lock` (`:454-470`), `try_claim_transform_lock` (`:488-504`) and `try_claim_conform_lock` (`:556-572`) — identical, on purpose.
- `:52-61` — `CONCLUDED_CYCLE_STATES` and the comment that records the R31 defect.
- `:598-628` — `latest_run_by_table`, the "one status per table" resolver, with the `stage=` filter the download and transform barriers use.
- `:631-655` — `base_bronze_tables` + `tables_missing_bronze`: the R22 completeness half of the gate.
- `src/lambdas/gold_barrier/handler.py:136-215` — the canonical barrier, in order: read manifest → concluded-state check (`:142`) → latest status per table (`:150-158`) → completeness hold (`:181-193`) → claim (`:202-215`).
- `src/lambdas/download_barrier/handler.py:242-332` — the same shape one phase earlier, plus the chunk-completeness hold at `:277-317`. Note the comment on `:314-317`: the hold is deliberately *before* the lock.
- `:166-194` — `_completed_initial_loads`. A lovely piece of reasoning: a chunked initial load finishes **out of order**, so no single chunk can conclude "the load is done" — only the phase can, and the barrier is the only single-flight actor that sees the whole phase.
- `src/lambdas/transform_barrier/handler.py:97` and `src/lambdas/silver_barrier/handler.py:329` — the same `CONCLUDED_CYCLE_STATES` guard, so you can see the fix landed everywhere.
- `infra/env/dev/per_source_ingestion.tf:201-222` — the barrier tail is `Retry` + `Catch → Succeed`. A barrier error must never fail a good ingestion.

L16 · How Two Jobs Don't Collide ★

Do this

1. Read `try_claim_gold_lock` and then write the race out on paper: two Lambdas, two `PutItem` s, one item. Say what each one returns and what each one does next. Now do it again where the *first* caller crashes immediately after winning — who cleans up, and how? (Answer is in L18.)
2. Find the three places a barrier can return `waiting` . For each, say why returning early is safe, and what would break if that check sat *after* the lock claim instead of before it.
3. In `tamimi-lakehouse-coordination-dev` , find a `goldlock#…` row. Read `claimed_by` — that is the `aws_request_id` of the invocation that won. Now find that request in the gold-barrier CloudWatch logs and read the sibling invocations that lost.
4. Break it in your head: change `CONCLUDED_CYCLE_STATES` back to `state != "running"` . Trace a cycle from `running` and name the first phase that stops. Then explain why the single-flight lock is what makes the correct version safe.

You've got it when you can…

…answer "two jobs raced the barrier — which one wins, and why is that safe?" with **"whichever one DynamoDB orders first on that single item; the other gets `ConditionalCheckFailedException` , returns `lock_lost` , and does nothing — which is the design working"** — and then name the two *other* checks (terminal-per-table and completeness) that have to pass before anyone is even allowed to reach for the lock.

MODULE 2 · LESSON 17

The System's Memory

Four DynamoDB tables. Learn them by the question each one answers — then you always know where to look.

1 · FOUR TABLES, FOUR QUESTIONS

runs

"What happened?"

`run_id (HASH) + stage (RANGE)`

One row PER STAGE, all sharing one run_id — never overwritten as the table advances. TTL 90 days.

`GSI by_table → one table's history`
`GSI by_cycle → one whole cycle`

WRITE every Glue job, per stage
READ every barrier + the sweeper

`control_plane/runs.py:54-104`

watermarks

"How far did we get?"

`source_id (HASH) + table_name`

last_watermark_value is the generic position; a SAP hybrid also keeps a JDBC date AND an ODP delta token.

`init_state pending → initial_loaded → cdc → needs_reinit`

WRITE the connector, on success
READ Gate 0, then the next CDC

`control_plane/watermarks.py:30-93`

pipeline-state

"Is it healthy right now?"

`pipeline_id = '<kind>:<id>'`

One table, four widgets. Pydantic picks the payload off state.kind, so each kind keeps its own schema.

`pipeline: · alarm: · freshness: reconciliation: drift vs threshold`

WRITE jobs + the recon harness
READ dashboard (stream NEW_IMAGE)

`control_plane/pipeline_state.py:83`

lineage_edges

"What came from what?"

`edge_id = up|down|edge_type`

The id is deterministic, so re-emitting an edge overwrites it — the DAG can never grow duplicates.

`source_to_bronze · bronze_to_silver silver_to_gold · gold_to_view`

WRITE each transform, at the end
READ impact analysis · lineage UI

`control_plane/lineage_edges.py:21-59`

2 · THIS IS WHAT THE OPS CONSOLE READS — AND WHAT YOU TYPE AT 3 AM

EVERY OPERATOR QUESTION IS A DYNAMODB READ

```
runs-dev         index by_cycle, cycle_id = 2026-08-10   → per-table status + error_message
coordination-dev cycle#2026-08-10                        → gold_built | gold_skipped | running
```

Never infer success from a Step Functions status — a P1 execution can read SUCCEEDED over failed chunks (R34).

THE OTHER THREE

coordination — the cycle manifest and the single-flight locks (that's L16).

bronze-mapping + source-catalog = config.

IN PLAIN ENGLISH

Nothing here remembers anything by itself. Four notebooks do it: what ran, how far we got, how it looks now, and where it came from.

L17 · The System's Memory

Slide: `_render/L17-control-plane-schema.html`

The point

Glue jobs are stateless. Step Functions forgets an execution's payload the moment it ends. Nothing in this pipeline remembers anything by itself.

Four DynamoDB tables do all the remembering, and each one answers exactly one question:

TABLE	THE QUESTION
`runs`	*What happened?*
`watermarks`	*How far did we get?*
`pipeline-state`	*Is it healthy right now?*
`lineage_edges`	*What came from what?*

Learn them that way and you never have to guess where to look. Every step of the operations runbook — and every widget the ops console shows — is a read against one of these.

(There are three more tables in the same module: `coordination` holds the cycle manifest and the single-flight locks, which is L16's subject, and `bronze_mapping` + `source_catalog` are the *config* the dispatcher reads, which is L15's.)

Key ideas

`runs` — "what happened?"

- **Composite key: `run_id` (HASH) + `stage` (RANGE).** One row *per stage*, all sharing the table's `run_id`. A table's `bronze_pull` and `bronze_to_silver` are separate rows — not one row overwritten as it advances. That is why a single logical table shows up twice (or more) in a cycle's rows, and why the barriers must resolve "the table's status" as *the latest stage by* `started_at`.
- **Two GSIs, both sparse (only rows that set the key appear):**
- `by_table = (table_name, started_at)` — one table's history. Operator monitoring, re-run lookups.
- `by_cycle = (cycle_id, started_at)` — one whole cycle's runs. **This is the index every barrier queries.**
- `silver_to_gold` rows are the exception: Gold is many-to-many (one dbt build, many models), so no single `run_id` can span it. Those rows get their own per-model `run_id` and correlate back through `cycle_id`. Lineage lives in `lineage_edges`, not here.
- Every row carries `status`, `rows_in` / `rows_out`, `error_message`, `run_kind` (`scheduled` / `initial` / `cdc` / `rerun` / `backfill`) and a 90-day `ttl`. **`error_message` is the exact blocker for that table** — it is the first thing to read on any failure.
- **`by_cycle` is eventually consistent.** A barrier invoked microseconds after the last write can legitimately fail to see it. The state machines account for this with an explicit `Wait` (15 s) before the final gold-barrier call — see `WaitForGsi` in `infra/env/dev/transform_ingestion.tf:161-169`. Single-flight makes the second call a harmless no-op if Gold already fired.

L17 · The System's Memory

`watermarks` — "how far did we get?"

- **Key: `source_id` (HASH) + `table_name` (RANGE).** Read at job start, written at job end. **A failed run does not advance it** — that invariant is the whole reason CDC is recoverable.
- Three positions, not one, because SAP tables are a hybrid: `last_watermark_value` (the generic position — an S3 key for Excel, a date for RDS), `jdbc_watermark` (high-water date from the last JDBC read), and `odp_delta_token` + `token_updated_at` (the CDC resume pointer and its freshness clock, which drives the token-age expiry alarm).
- **`init_state` is a little state machine of its own**, and it is what Gate 0 reads: `pending` → `initial_loaded` → `cdc`, with `needs_reinit` as the "the ODP delta token expired, force a JDBC re-baseline" branch. `None` means "not applicable" (Excel tables have no init/CDC lifecycle).
- The flip to `initial_loaded` is written by the **download barrier**, not by the Glue job — because a chunked initial load finishes out of order and only the barrier sees the whole phase.

`pipeline-state` — "is it healthy right now?"

- **One table, four widgets.** The key is polymorphic: `'<kind>:<id>'`, where kind is `pipeline` | `alarm` | `freshness` | `reconciliation`. Chosen over four tables to save cost and give the operator UI one API surface; the trade-off (weaker per-kind schema enforcement) is bought back with a **Pydantic discriminated union** on `state.kind`, so each kind still validates against its own model.
- Note the Python/DDB naming seam: the Pydantic field is `entity`, aliased to `pipeline_id` for serialisation, because that is what the deployed table's HASH key is called.
- Stream is enabled with `NEW_IMAGE` only — the dashboard just needs the latest state, not deltas.
- A worked example of schema care: `ReconciliationState.drift_pct` is `float | None`, because "no comparable number" and "zero drift" are different answers. The earlier non-optional `ge=0.0` field forced the harness to clamp its "cannot reconcile" sentinel to `0.0` — **rendering a broken check as a perfect one.**

`lineage_edges` — "what came from what?"

- **Key: `edge_id`, and it is deterministic** — `f"{upstream}|{downstream}|{edge_type}"`. Re-emitting the same edge overwrites the same item, so repeated runs never grow the DAG.
- `edge_type` ∈ `source_to_bronze` | `bronze_to_silver` | `silver_to_gold` | `gold_to_view`.
- `transform_id` is the dbt model name, Glue job ARN or Lambda name that drew the edge — the handle you use to navigate from a lineage edge *back to the code*.
- Written by each transform at completion. Read by impact analysis and the dashboard's `/ops/lineage` route.

The ops surface

All the control-plane tables live in one Terraform module — seven of them today, though the module header still says six, because `coordination` was added after it was written. All `PAY_PER_REQUEST`, all encrypted with the audit CMK (their semantic class is "operational / compliance data", the same as CloudTrail). Streams are enabled on exactly two: `runs` (`NEW_AND_OLD_IMAGES`) and `pipeline_state` (`NEW_IMAGE`, for the dashboard).

L17 · The System's Memory

The operational rule that follows from all of this: **never infer success from a Step Functions execution status.** A P1 download execution has been observed reporting `SUCCEEDED` over a fan-out in which every chunk failed. The barrier held correctly and no bad data landed — but the execution status lied. Query `runs` by cycle.

Words you'll hear

WORD	WHAT IT MEANS HERE
Control plane	The DynamoDB tables that hold state *about* the pipeline (as opposed to the data itself)
HASH / RANGE key	DynamoDB's partition key and sort key. Together they identify one item
GSI	Global Secondary Index — a second way to query the same table (here: by table, by cycle)
Sparse index	Only items that carry the index's key attribute appear in it
Polymorphic key	One key column holding `'<kind>:<id>'`, so several record types share a table
Discriminated union	Pydantic picks the right payload model from a `kind` field, so each kind keeps its own schema
Watermark	The high-water mark of what has already been read from a source
Delta token	ODP's CDC resume pointer. Expires if unused past the queue-retention window
TTL	DynamoDB's automatic expiry. Run rows self-delete at 90 days
Eventually consistent	A GSI read may not yet reflect a just-committed write

In this repo

- `src/shared/shared/control_plane/runs.py:1-104` — `RunItem`. The module docstring (`:1-24`) explains the composite key and both GSIs better than any diagram; `RunStage` at `:42-44` and `RunKind` at `:51` are the two vocabularies you will use every day.
- `src/shared/shared/control_plane/watermarks.py:20-93` — `InitState` (`:20-27`) and the dual-watermark fields (`:65-93`).
- `src/shared/shared/control_plane/pipeline_state.py:23-101` — the four state payloads, the `StateUnion` discriminator at `:75-78`, and the `ReconciliationState` docstring at `:54-65` (read that one; it is a good lesson about nullability).
- `src/shared/shared/control_plane/lineage_edges.py:21-59` — `_compute_edge_id` and the `model_validator` that fills it in.
- `infra/modules/control_plane/main.tf:117-184` — the `runs` table with both GSIs declared (`:155-169`). Then `watermarks` at `:84-114`, `lineage_edges` at `:214-238` ("Read by operator dashboard /ops/lineage route"), `pipeline_state` at `:240-268` ("Operator dashboard subscribes for real-time updates"), and `coordination` at `:186-212`.
- `docs/OPERATIONS-RUNBOOK.md` §0, §2, §3 — the operator's view of exactly these tables, with the CLI you will actually type. §2's callout on "two rows per table, one per stage" is the same fact as `runs.py`'s composite key, said in operator language.
- `src/lambdas/run_status/handler.py:1-22` — the crash-net that guarantees a dead Glue job still leaves a terminal `runs` row, so the table stays trustworthy.

L17 · The System's Memory

Do this

1. For today's cycle, run the runbook's `by_cycle` query. Count the rows, then count the tables. Explain the difference out loud (stages × retries).
2. Pick one table that ran. Find *all* its rows, sort by `started_at`, and write down the sequence of stages. Now do what a barrier does: name its single resolved status.
3. Read one `watermarks` row for a SAP table. Which of the three positions is populated? What does its `init_state` allow to happen tomorrow night?
4. Given "Power BI shows yesterday's number for `unified_sales`" — write down, in order, which of the four tables you query and what you expect each to tell you. That is the L23 debugging drill in miniature.
5. Find a `lineage_edges` row whose `downstream` is a `gold.*` table, then use its `transform_id` to open the code that produced it.

You've got it when you can…

…be handed a question — "did it run?", "how far did it get?", "is it alerting?", "where did this column come from?" — and name the table, the key, and the index you would query, without looking anything up.

MODULE 2 · LESSON 18

When It Breaks at 3 AM

Re-running is the normal move, not the desperate one. Here is exactly why that is safe — and what to re-run.

1 · WHY A RETRY CANNOT DOUBLE-BUILD THE DAY

FIVE THINGS THAT MAKE A SECOND ATTEMPT A NO-OP

EXEC NAME	Deterministic <source>-<cycle_id>; a dead prior run re-fires as -rHHMMSS.
THE MANIFEST	put_cycle is create-only — it can never reset a concluded cycle.
THE LOCKS	One conditional write per phase; the loser returns and does nothing.
THE WRITES	Bronze overwrites its snapshot, Silver MERGEs on key — never duplicates.
PARTIAL FAIL	One table's failure is isolated in the Map — then skips Gold for all.

`handler.py:972-1024 · coordination.py:371-392 · per_source_ingestion.tf:185-190`

WHY GOLD LOOKS BEFORE IT LEAPS

```
03:12:04  StartJobRun → Glue ACCEPTS r-8821
03:12:34  the response never comes back
03:12:35  SFN retries → a fresh lock claim
          → a SECOND Gold build, same day
```

StartJobRun has no idempotency token, so the barrier scans the last 50 runs for this --cycle_id in a live or done state — and skips. Any error → fails OPEN.

`gold_barrier/handler.py:50, 83-111, 253-263`

2 · THE SAFETY NET UNDER EVERY BARRIER

cycle_sweeper — LEVEL-TRIGGERED, EVERY 15 MINUTES

RESUME every pass, no deadline — re-invoke the barrier the state implies.

`running → download` `download_built → transform` `transform_built → gold`

CONCLUDE only when overdue AND stopped moving. 12 h opens the question, 3 h idle answers it, 96 h is the ceiling → force-fail, then gold_skipped.

It ignores rows it wrote itself — or a force-fail pass reads its own timestamps back as signs of life.

`cycle_sweeper/handler.py:77-81 resume map · :145-170 _should_conclude · :107-142 _last_activity`

3 · WHICH RECOVERY?

FOUR MODES, ONE DISPATCHER

`rerun` — today's table failed. Reopens cycle, frees locks.

`rerun_download` — the raw pull failed. P1 only; P2 re-gates.

`rerun + dates` — backfill: own bf- cycle, watermark untouched.

`initial_load` — re-baseline: own init- cycle, full_refresh.

`dispatcher/handler.py:1027 · :1144 · :1701 · :1417`

IN PLAIN ENGLISH

At 3 AM you don't debug — you read the runs table, pick a mode, and re-run. Every write here is safe to repeat, so a re-run is never the risk.

L18 · When It Breaks at 3 AM

Slide: `_render/L18-failure-and-recovery.html`

The point

The alert says a table failed and Gold was skipped. You have three decisions to make, in this order:

1. **What actually failed?** → read the `runs` row's `error_message` (L17).
2. **Is it safe to just re-run it?** → yes, and the slide lists the five independent reasons why.
3. **Which kind of re-run?** → `rerun`, `rerun_download`, backfill, or re-baseline. They are not interchangeable.

The design principle behind all of it: **re-running is the normal move, not the desperate one.** Every write in this pipeline is safe to repeat, and the system is built so that a second attempt is either identical work or a no-op. If re-running feels risky, you have misunderstood something — go and find which of the five guarantees you don't believe.

Key ideas

Idempotency, end to end

Five independent mechanisms, at five different layers:

1. **The execution name.** `<source>-<cycle_id>` is deterministic, so a duplicate dispatch collides and no-ops. The trap: Step Functions reserves names for **90 days regardless of state**, so `ExecutionAlreadyExists` is not proof of "still running". A terminally dead prior run is re-dispatched under `<name>-rHHMMSS`.
2. **The manifest.** `put_cycle` is create-only (`attribute_not_exists(pk)`) — a re-fire cannot overwrite a manifest that is already `gold_built`, nor reset `started_at` and restart the sweeper's clock.
3. **The locks.** One conditional write per phase; the loser does nothing (L16).
4. **The writes themselves.** Bronze overwrites its snapshot; Silver MERGEs on key or overwrites partitions. Re-running a table never duplicates rows.
5. **Resolution by latest run.** A barrier judges each table by its most recent row, so a fresh SUCCEEDED run supersedes the old FAILED one without anyone deleting anything.

Why the Gold barrier looks before it leaps

`StartJobRun` has **no client idempotency token**. So this sequence is possible:

```
03:12:04  StartJobRun → Glue ACCEPTS the run, r-8821 starts
03:12:34  the API response never comes back (socket timeout)
03:12:35  the SFN Task retries the barrier → a fresh lock claim succeeds
          → a SECOND gold build for the same day
```

Note that the lock does **not** save you here: the first attempt released it (or the retry landed in a re-opened cycle), and from the second invocation's point of view everything looks fine. Locks protect against *concurrent deciders*; they do not protect against *an ambiguous side effect you already caused*.

So before starting, the barrier calls `get_job_runs(MaxResults=50)` and looks for a run whose `--cycle_id` argument matches this cycle and whose state is `STARTING` / `RUNNING` / `WAITING` / `SUCCEEDED`. If it finds one, it treats the day as already built, reconciles the cycle state, and returns `gold_already_running`.

It **fails open**: any error in the pre-check (throttle, missing permission) returns `None` and the caller proceeds to start — the pre-existing behaviour, never worse than before the guard existed. That is the right default for a guard that is an optimisation over correctness rather than the correctness mechanism itself.

L18 · When It Breaks at 3 AM

The cycle sweeper — the level-triggered net

Barriers are edge-triggered: invoked once, at the end of a phase. If a barrier *dies*, nothing re-evaluates it and the cycle sits forever. The sweeper runs every 15 minutes and does two different jobs:

- **RESUME** (every pass, no deadline). For any non-concluded cycle, re-invoke the barrier its state implies: `running` → download barrier, `download_built` → transform barrier, `transform_built` → gold barrier. Barriers are self-gating (`no_cycle` / `already_concluded` / `waiting`) and single-flight, so a nudge is free unless the gate is genuinely satisfiable. This exists because of a real incident: the download barrier crashed on a manifest `ValidationError` for four consecutive invocations while P1 had SUCCEEDED and the gate had been satisfiable the whole time.
- **CONCLUDE** (only when overdue **and** no longer moving). Force-fail every expected table that is still non-terminal — writing a synthetic FAILED `bronze_pull` row for a table with no row at all — then re-invoke the gold barrier so it claims the lock and concludes `gold_skipped` + alert.

Age alone is not evidence of death, and treating it as such destroyed real work: QA force-failed three healthy initial reloads (67–70 tables each) whose only fault was taking longer than the deadline. So now the 12 h deadline only *opens the question*; 3 h of no new run rows *answers* it; 96 h is an absolute ceiling so "never hangs forever" still holds.

Two subtleties worth internalising:

- **The sweeper ignores rows it wrote itself.** A force-fail pass stamps `ended_at` on every non-terminal table; counting those would let it read its own writes back as proof the cycle is alive, and refuse to conclude for another 3 hours. Seen for real on 2026-08-09, 70 rows timestamped at the sweep instant.
- **The force-fail pass excludes P1 rows.** A SUCCEEDED `source_download` means the raw extract landed, not that the table bronzed. Counting it made a table with no `bronze_pull` row look terminal, so it was never force-failed and the gold barrier's completeness hold had nothing to break it — the cycle hung instead of concluding.

Partial failure

Two rules that look contradictory and aren't:

- **Per-item isolation.** In the `Map` , a failed table catches to `ItemFailed` which is a `Succeed` state — so one bad table never stops the other 54 from processing. The failed table's run row is already FAILED (the job self-reports before raising).
- **All-or-nothing at the gate.** Any FAILED table in `expected[]` → the next phase does not start. A partial raw landing must not load into Bronze; a partial Bronze must not build Gold.

Isolation maximises how much useful work one cycle gets done; all-or-nothing stops a half-complete picture reaching a report. The `runs` table is what reconciles them.

L18 · When It Breaks at 3 AM

Which recovery — and when

MODE	PAYLOAD	USE WHEN	WHAT IT DOES
rerun	`{"rerun":true,"cycle_id":"…"}`	a table failed in **today's** cycle	Reopens the cycle (`state → running`), releases the gold **and** conform locks, restarts the source SFN under a unique name. Omit `tables` = re-run everything that failed; name `tables` to force a green table to re-run
rerun_download	`{"rerun_download":true,"cycle_id":"…"}`	the **raw pull** (P1) failed	P1 only. Never starts P2 — the download barrier does that once the gate re-passes. Also re-points `download_expected_chunks` , because a re-pull dispatches **one** un-windowed item per table and the old 10-window plan would be permanently unsatisfiable
backfill	`{"rerun":true,"source":"sap","date_from":…,"date_to":…}`	**old** data was wrong	Its own scoped `bf-…` cycle. Bounded `BETWEEN` read; **the incremental watermark is untouched**, so tomorrow's normal run is unaffected. Sets `full_refresh=True` , so Gold builds with `--full-refresh` — otherwise a window older than Gold's incremental predicate would land in Silver and never surface
re-baseline	`{"initial_load":true,"tables":[…],"force":true,…}`	the ODP delta token expired (`init_state = needs_reinit`)	A JDBC range re- CDC gap, in its o `rebaseline-<d` On success `ini- initial_loadec` CDC run can res token. The dispa this **automatical** sees `needs_rei`

Two rules people get wrong: **derived / conform tables are never named in `tables`** (they have no source SFN — list their *inputs* and the conform phase rebuilds them), and **auto-detect never re-runs a green table** (an empty `tables` means "re-run the failures"; to redo something that succeeded you must name it).

L18 · When It Breaks at 3 AM

Words you'll hear

WORD	WHAT IT MEANS HERE
Idempotent	Doing it twice has the same effect as doing it once
Edge- / level-triggered	Fires once on an event / re-checks the world on a timer. Barriers vs the sweeper
Force-fail	The sweeper writing a synthetic FAILED row so a stuck cycle can reach a verdict
Stall / deadline / ceiling	3 h without a new run row / 12 h old / 96 h absolute — the three sweeper clocks
Fail open	On an internal error, proceed as if the guard weren't there
Re-baseline	Re-establish the CDC starting point with a JDBC pull after a token expiry
Backfill	A date-scoped re-pull in its own cycle that leaves the watermark alone
All-or-nothing	Any failure in the phase → the next phase does not start
Crash net	`run_status`, which writes a terminal row for a Glue job that died before it could

In this repo

- `src/lambdas/cycle_sweeper/handler.py:1-34` — the docstring is the design rationale for the whole lesson: why edge-triggered barriers need a level-triggered counterpart. Then `_RESUME_BARRIER` at `:77-81`, `_last_activity` at `:107-142` (the "don't read your own writes" fix), `_should_conclude` at `:145-170`, the force-fail pass at `:292-322`, and `_put_failed` at `:349-370`.
- `src/lambdas/gold_barrier/handler.py:50` — `_INFLIGHT_OR_DONE`; `:83-111` — `_existing_gold_run` and the fail-open contract; `:249-263` — the pre-check in use; `:278-295` — release-on-start-failure vs never-release-after-success, with the reasoning spelled out in comments.
- `src/lambdas/download_barrier/handler.py:418-460` — the same shape one phase earlier: any P2 start failure → alert, release the lock, do **not** mark `download_built`.
- `src/lambdas/dispatcher/handler.py:1027-1141` `_rerun` · `:1144-1282` `_rerun_download` (note the chunk-count re-point at `:1241-1253`) · `:1417-1673` `_initial_load` · `:1701+` `_backfill` · `:1369-1414` `_auto_rebaseline`.
- `:501-532` — `_invoke_gold_barrier`, the "nothing started at all" backstop, and `:875-911` for when it fires.
- `infra/env/dev/per_source_ingestion.tf:185-190` — `ItemFailed = { Type = "Succeed" }`: per-item failure isolation in three lines. And `:88-112` for the retry policy, whose budget must outlast the longest-running job (220 attempts ≈ 9.2 h > the 480-minute Glue timeout).
- `infra/env/dev/cycle_sweeper.tf:174` — `rate(15 minutes)`.
- `src/lambdas/run_status/handler.py` — the crash net, and also the per-stage failure notifier that sends the email you woke up to.
- `docs/OPERATIONS-RUNBOOK.md §4` — the operator-facing version of the recovery table, with copy-paste payloads.

L18 · When It Breaks at 3 AM

Do this

1. Fail a table on purpose in Dev (point a spec at a nonexistent column). Watch: the run row goes FAILED, the cycle concludes `gold_skipped`, the alert email arrives. Then fix it, `rerun`, and watch the cycle reach `gold_built`. Do this once and the rest of the lesson is obvious.
2. Read `_should_conclude` and answer: a 57-table initial load has been running for 20 hours and wrote a run row 40 minutes ago. What does the sweeper do — and what did the *old* age-only version do?
3. Delete a `goldlock#…` row by hand and re-invoke the gold barrier. Predict what happens first. Now do the same thing while a gold build is genuinely in flight, and say which guard stops the second build.
4. For each of these, pick a recovery mode and justify it in one sentence: (a) `sap.zsdcc` failed at `bronze_to_silver` an hour ago; (b) last March's figures were restated at source; (c) the ODP token expired while CDC was down for a week; (d) the raw landing for six tables is missing but Bronze looks fine.
5. Trace the "post-submit timeout" scenario through `gold_barrier/handler.py` line by line, and identify the exact line that saves you.

You've got it when you can…

…be handed a red cycle at 3 AM and, without guessing: name the failed stage and its `error_message` from `runs`, say whether the cycle will conclude by itself or needs the sweeper, choose the right recovery mode and explain in one sentence why the other three are wrong, and state which of the five idempotency guarantees means your re-run cannot make things worse.

MODULE 2 · LESSON 19

How the Infrastructure Is Built

28 reusable modules. Three environments that only differ by values. And the guard rails that stop a typo deleting Redshift.

1 · A MODULE — BUILT ONCE, ENV-BLIND

A MODULE IS A BUILDING BLOCK

`infra/modules/<name>/ — main.tf · variables.tf · outputs.tf · versions.tf`

It takes inputs. It knows no environment name,
no account id, no CIDR, no bucket name.
Hard-code an env inside it and it stops being a module.

```
vpc · vpc-peering · tgw-route · iam · kms · secrets
s3 · s3-data-lake · ssm · cloudtrail · lake-formation
glue-job · step-function · redshift-serverless · … 28
```

2 · AN ENVIRONMENT — COMPOSITION ONLY

AN ENV IS A LIST OF MODULE CALLS

Dev, QA and Prod call the SAME modules — 24 module blocks
in one file. Everything that differs lives in tfvars.

```
module "vpc" {
  source = "../../modules/vpc"
  vpc_cidr_range =
      var.network.vpc_cidr_range
}
```

SAME CALLS · 3 ROOTS

dev	21 .tf	10.248.4.0/24
qa	21 .tf	10.248.6.0/24
prod	20 .tf	10.248.8.0/24

Values differ. Code does not.

3 · THE THREE GUARD RAILS THAT KEEP IT HONEST

PIN THE PROVIDER

An unpinned provider means today's
plan is not tomorrow's plan.

```
aws = "~> 6.28"   required >= 1.9
28 of 28 modules carry versions.tf
```

`infra/env/dev/versions.tf:1-22`

MAKE STATE UNDELETABLE

Anything holding data or history
refuses to be destroyed. Literal, always on.

```
lifecycle { prevent_destroy = true }
KMS · S3 · CloudTrail · Redshift ns
```

`infra/modules/kms/main.tf:45-50 (HIGH-17)`

ISOLATE THE STATE

Each env owns its own backend.tf
and its own state key — read the value:

```
key = ".../dev/terraform.tfstate"
one bucket + lock for all 3 — CRIT-01
```

`infra/env/dev/backend.tf:18-26`

IN PLAIN ENGLISH

A module is a class; an env root is the file that instantiates it three times with different constructor arguments. prevent_destroy is `final`.

L19 · How the Infrastructure Is Built

Slide: `_render/L19-terraform-architecture.html`

The point

Every AWS resource in this platform — the VPC, the six KMS keys, the Redshift namespace, all of it — is declared in Terraform under `infra/`. There is no console-clicked infrastructure that matters, and the whole estate is built from exactly two kinds of file.

- **A module** (`infra/modules/<name>/`) is a reusable building block. It takes inputs. It knows no environment name, no account id, no CIDR. **28 of them today.**
- **An environment root** (`infra/env/{dev,qa,prod}/`) is a *composition* — a list of module calls plus a `terraform.tfvars` of values. `infra/env/dev/main.tf` alone contains **24 `module` blocks**; the three env roots hold 21 / 21 / 20 `.tf` files.
- Everything that differs between Dev, QA and Prod is a **value**, not code. Dev's VPC is `10.248.4.0/24`, QA's `10.248.6.0/24`, Prod's `10.248.8.0/24` — same `modules/vpc`.

Key ideas

- **The module/env boundary is the whole design.** If a file under `infra/modules/` contains the word `dev`, an account id, or a live subnet id, it has stopped being a module. That is not style — it is why one bug fix in `modules/kms` lands in three environments at once.
- **A module is four files:** `main.tf` (resources), `variables.tf` (the inputs it accepts), `outputs.tf` (what it hands back), `versions.tf` (what it needs to build). Every one of the 28 has all four.
- **Env roots wire modules to each other, not to AWS.** `module.iam` receives `module.kms.key_arns["bronze"]`; `module.redshift` receives `module.vpc.data_subnet_ids`. Terraform derives the apply order from those references — nobody writes a dependency graph by hand.
- **Guard rail 1 — pin the provider.** `aws = "~> 6.28"`, `required_version >= 1.9`, declared in the env root *and* in all 28 modules. Without it, an AWS provider release between your plan and your colleague's plan silently changes what "no change" means.
- **Guard rail 2 — `prevent_destroy` on anything stateful.** KMS keys, every S3 bucket, the CloudTrail bucket, and the Redshift Serverless namespace carry `lifecycle { prevent_destroy = true }`. Terraform requires that to be a **literal**, so it cannot be toggled by a variable — the code comments spell out that `var.deletion_protection` controls something else entirely (the KMS deletion *window*, the S3 `force_destroy` flag). Read the resource, not the variable name.
- **Guard rail 3 — state isolation, and what it actually is today.** Each env has its own `backend.tf` and its own state **key**. But all three still name the *same* bucket and the *same* DynamoDB lock table (`tamimi-lakehouse-tfstate-633740007496`); only `key` differs. The audit calls this **CRIT-01** — a Dev apply has `s3:DeleteObject` on Prod state. The `dev` / `qa` backend files carry a long comment describing the per-env split as if it were done. **It is not done in the values.** Read the value, not the comment.
- **Two smaller safeties worth copying:** `allowed_account_ids` on the provider (fail the plan if the assumed identity is in the wrong account), and `-lock-timeout=30m` on every CI plan/apply so a second push queues behind the first instead of dying on `ConditionalCheckFailedException`.

L19 · How the Infrastructure Is Built

Words you'll hear

WORD	WHAT IT MEANS HERE
Module	Reusable block under `infra/modules/<name>/` ; env-blind by rule
Composition / env root	`infra/env/<env>/` — the file that calls modules and passes values
`tfvars`	The per-environment value file; the only place an env is allowed to differ
State	Terraform's record of what it built; lives in S3, locked by DynamoDB
`prevent_destroy`	A literal lifecycle guard that makes `terraform destroy` fail on that resource
Provider pinning	`aws = "~> 6.28"` — the plan you produce is the plan a colleague reproduces
Blast radius	How much a single bad apply can damage. CRIT-01 is a blast-radius finding

In this repo

- `infra/modules/` — the shelf: `vpc` , `vpc-peering` , `tgw-route` , `iam` , `kms` , `secrets` , `s3` , `s3-data-lake` , `ssm` , `cloudtrail` , `lake-formation` , `glue-job` , `glue-connection` , `glue-catalog` , `lambda` , `step-function` , `eventbridge` , `redshift-serverless` , `mwaa` , `sns` , `sqs` , `budgets` , `control_plane` , `customer_dims` , `dispatcher_lambda` , `dbt_silver_to_gold` , `dim_upload` , `catalog_federation` — **28 directories, 28 `versions.tf`** .
- `infra/env/dev/main.tf:11-22` — `module "vpc"` , the shape every other block follows; `:27-40` the KMS block that lists the six layers; `:45-55` IAM receiving KMS outputs.
- `infra/env/dev/versions.tf:1-22` — `required_version >= 1.8.0` , `aws ~> 6.28.0` , plus `archive` / `null` / `random` . Compare `infra/modules/kms/versions.tf` — same pins, module-side.
- `infra/env/dev/backend.tf:18-26` — the S3 backend. Read lines 20 and 23 against the comment at lines 3-16.
- `infra/modules/kms/main.tf:45-50` — `prevent_destroy` with the HIGH-17 comment explaining why it cannot be variable-driven. Same pattern in `modules/s3/main.tf:71-83` , `modules/redshift-serverless/main.tf:72-76` , `modules/cloudtrail/main.tf:76-79` .
- `infra/env/dev/providers.tf:1-10` — `allowed_account_ids` (M-36) and `default_tags` .
- `docs/handoff/audit_deep_analysis.md:122-133` — CRIT-01 in full, with the remediation blueprint.

L19 · How the Infrastructure Is Built

Do this

1. Open `infra/modules/vpc/variables.tf` and `infra/env/dev/main.tf:11-22` side by side. Every variable the module declares is either passed here or defaulted. Now find where `10.248.4.0/24` is written — it is in `terraform.tfvars`, and nowhere else.
2. Grep the modules directory for an environment name: `grep -rn "\"dev\"\|633740007496" infra/modules/`. Anything you find is a bug in the making.
3. Compare `infra/env/dev/main.tf` and `infra/env/prod/main.tf`. Find the one `module` block whose *arguments* genuinely differ (hint: `sap_tgw_route` — Prod passes `data_route_table_ids`, Dev passes `private_route_table_ids`). Ask why. That is L22's story.
4. Add `prevent_destroy` in your head to the DynamoDB control-plane tables and predict what breaks. (Answer: any `for_each` key removal is blocked at plan time — the same trade-off the `s3` module documents at `main.tf:71-83`.)

You've got it when you can…

…draw the two-layer picture — **28 env-blind modules on the shelf, three env roots that call them with different values** — name the three guard rails (`versions.tf` pinning, `prevent_destroy`, per-env state), and then say honestly what the third one *actually* is today: one bucket, one lock table, three keys — which is CRIT-01, not isolation.

MODULE 2 · LESSON 20

How Code Reaches Production

There is no AWS access key anywhere in this pipeline. There is a signed badge, a one-hour pass, and a human on the Prod gate.

1 · IDENTITY — NO STATIC KEYS, EVER

BITBUCKET
the pipeline step

`oidc: true`
mints a fresh JWT
for THIS step only

→

TOKEN ON DISK
not a credential

`AWS_WEB_IDENTITY_TOKEN_FILE`
written, used, gone

→

AWS STS
swaps badge for pass

`AssumeRoleWithWebIdentity`
3600 s, then expired

```
Trust:  sub = "{repo-uuid}:<branch>:*"   ·   aud = workspace ARI
Roles:  AWS_ROLE_DEV_ARN · AWS_ROLE_QA_ARN · AWS_ROLE_PROD_ARN
```

Trap: locals.tf:24 pins dev to "main" — the pipeline deploys Dev from develop.

2 · THE BRANCH DECIDES THE ACCOUNT

ONE PIPELINE PER BRANCH

Bitbucket picks the pipeline from the branch name;
the pipeline picks which role ARN to assume.

```
pull-requests   →  lint · mypy · pytest · tf validate
develop         →  DEV    apply  (auto)
release/*       →  QA     apply  ← MANUAL
main            →  PROD   apply  ← MANUAL
prod-deploy     →  PROD   apply  ← MANUAL
```

3 · WHAT THE PIPELINE REFUSES TO DO

NO UNATTENDED PROD

QA and Prod applies sit and wait for a
person to click. Only Dev is automatic.

```
trigger: manual
deployment: production
```

bitbucket-pipelines.yml:424, 492-495, 692

APPLY THE PLAN YOU READ

A saved plan fails closed if state moved.
-auto-approve rubber-stamps a fresh one.

```
plan -out=/tmp/prod-full.tfplan
apply /tmp/prod-full.tfplan
```

bitbucket-pipelines.yml:569-571 (HIGH-11)

PIN THE SUPPLY CHAIN

Nothing unversioned lands on a runner
that is holding cloud credentials.

```
sha256 ngdbc-2.20.11.jar
uv==0.10.0 pydantic==2.13.4
```

bitbucket-pipelines.yml:201-203 (HIGH-18)

IN PLAIN ENGLISH

CI holds no password. It shows AWS a signed badge, gets a one-hour pass to one account — and Prod still waits for a human.

L20 · How Code Reaches Production

Slide: `_render/L20-cicd-deploy-gates.html`

The point

Search `bitbucket-pipelines.yml` for `AWS_ACCESS_KEY_ID`. It is not there, and it never was. The pipeline authenticates to AWS with a **short-lived OIDC web-identity token** that Bitbucket mints per step, and every deploy to QA or Prod stops and waits for a human.

Three questions answer the whole lesson:

1. **How does CI prove who it is?** OIDC — a signed JWT swapped at STS for a one-hour role session.
2. **Which account does it land in?** The branch decides, via one of three role ARNs.
3. **What stops a bad change?** A manual gate, a saved plan file, and version pins on everything the runner downloads.

L20 · How Code Reaches Production

Key ideas

- `oidc: true` **on a step** makes Bitbucket mint a JWT into `$BITBUCKET_STEP_OIDC_TOKEN`, scoped to that step. The pipeline writes it to a file, points `AWS_WEB_IDENTITY_TOKEN_FILE` at it, sets `AWS_ROLE_ARN`, and the AWS SDK does the rest — `sts:AssumeRoleWithWebIdentity`, no credential ever stored.
- **The trust policy is what actually decides.** `global/oidc/main.tf:41-58` allows the exchange only when the token's `aud` equals the workspace ARI *and* its `sub` matches `{<repo-uuid>}:<branch-pattern>:*`. The role's `max_session_duration` is **3600 s**. If the branch does not match, the exchange fails — the pipeline YAML has no say.
- **Read that pair together, because they can drift.** `global/oidc/locals.tf:23-27` maps `dev = "main"`, `qa = "qa-*"`, `prod = "prod-*"`; `bitbucket-pipelines.yml` deploys Dev from `develop`, QA from `release/*`, Prod from `main` and `prod-deploy`. Those two lists must agree or the assume-role fails with `InvalidIdentityToken` / `AccessDenied`. Always check the trust policy first when a deploy step cannot see AWS.
- **One role per environment, no sharing.** `AWS_ROLE_DEV_ARN` (Repository variable), `AWS_ROLE_QA_ARN` and `AWS_ROLE_PROD_ARN` (Deployment variables). Each role carries three managed policies, split only because a managed policy version caps at **6,144 bytes**: `-deploy-data-<env>`, `-deploy-compute-core-<env>`, `-deploy-compute-services-<env>`.
- **The deploy order is artifacts-first, and it is not optional.** `aws_lambda_function` fails at apply if its zip is not already in S3, so every pipeline runs: targeted `apply -target=module.kms -target=module.s3` → build and upload wheels/scripts/specs/layer/Lambda zips → full plan → full apply → seed the DynamoDB control plane. The seed step exists because `bronze_pull` aborts on "Spec hash drift" if `bronze_mapping.spec_hash` is stale against the specs just uploaded.
- **The gate.** `trigger: manual` blocks the step until a person clicks: QA's "Apply full QA composition", Prod's "Deploy to Prod (MANUAL APPROVAL REQUIRED)", and `prod-deploy`'s "Apply full Prod composition". Only Dev auto-applies (the import-window gate was retired once the planned imports finished).
- **A gate is theatre unless the approved plan is the applied plan.** The audit's HIGH-11 was exactly this: the human approved `plan A` in one container, and a later step ran `apply -auto-approve` — which computes and rubber-stamps a *fresh* `plan B`. The fix on `main` generates `plan -out=/tmp/prod-full.tfplan` **inside the gated step**, prints it, and applies that file. A saved plan fails closed if state moved since it was written.
- **Supply chain: pin or fail closed.** The runner holds cloud credentials, so nothing unversioned may land on it. `uv` installed via `pip install "uv==${UV_VERSION:-0.10.0}"` (no `curl | sh`); AWS CLI version + `sha256sum -c`; the SAP JDBC driver `ngdbc-2.20.11.jar` downloaded then compared byte-for-byte against `NGDBC_SHA256` before it is uploaded to the artifacts bucket Glue loads at runtime; Lambda-layer deps pinned exactly (`pydantic==2.13.4`, `python-ulid==2.7.0`, `aws-lambda-powertools==2.43.1`, `boto3==1.43.19`) and cross-targeted to `aarch64-manylinux_2_17` because Lambda runs Graviton.
- `-lock-timeout=30m` **on every backend-touching command.** Bitbucket runs successive pushes concurrently; the default `0s` makes the second run die instantly on the state lock. 30 minutes makes it queue. The trade-off — a genuinely stale lock now stalls 30 minutes before erroring — is documented with the `force-unlock` recovery in the file header.

L20 · How Code Reaches Production

Words you'll hear

WORD	WHAT IT MEANS HERE
OIDC	OpenID Connect — Bitbucket signs a token, AWS trusts the signature, no shared secret
Web identity	The token file the SDK trades at STS for temporary credentials
Subject claim (`sub`)	`{repo-uuid}:<branch>:<step-uuid>` — what the trust policy pattern-matches
Deploy role	`tamimi-lakehouse-bitbucket-deploy-<env>` , one per environment
Manual gate	`trigger: manual` — the step will not run until a person clicks it
Saved plan	`plan -out=file` then `apply file` ; fails closed if state changed
Artifacts-first	Upload the Lambda zips before the apply that references them
Pinning	An exact version plus, where possible, a checksum

In this repo

- `bitbucket-pipelines.yml:60-109` — the shared `&lint-and-test` step: ruff, mypy, `pytest -m "unit or invariant"` , `terraform fmt -check` , and `validate` for all three envs plus `global/oidc` . This is the only thing a pull request runs.
- `:136-139` — the four lines that *are* the OIDC exchange: `AWS_REGION` , `AWS_ROLE_ARN=$AWS_ROLE_DEV_ARN` , `AWS_WEB_IDENTITY_TOKEN_FILE` , `echo $BITBUCKET_STEP_OIDC_TOKEN > …` .
- `:152-155` — the Dev targeted bootstrap (`-target=module.kms -target=module.s3`) with the plan logged before the apply.
- `:201-203` — the `ngdbc.jar` SHA256 comparison, written as an explicit `test "$computed" = "$expected"` with a failure message, not `sha256sum -c` .
- `:424` — QA's `trigger: manual # HIGH-11` ; `:492-495` — Prod's `trigger: manual` + `deployment: production` ; `:692` — the same on `prod-deploy` .
- `:569-571` — `plan -out=/tmp/prod-full.tfplan` → `show` → `apply /tmp/prod-full.tfplan` , with the comment explaining why this is not `-auto-approve` .
- `:33-52` — the state-lock note: the real 2026-08-03 failure (pipeline #434), why 30m, and how to recover a stale lock.
- `global/oidc/main.tf:30-59` — the OIDC provider and the deploy role's `assume_role_policy` (`aud` StringEquals, `sub` StringLike).
- `global/oidc/locals.tf:12,23-41` — the Bitbucket thumbprint (with the `openssl` command to refresh it), the per-env branch patterns, and the subject-claim construction.
- `global/oidc/permissions.tf:27-64` — the ARN globs every statement is scoped to; `:212-221` , `:376-385` , `:504-513` — the three policies and their attachments.

Do this

1. Trace one deploy end to end in `bitbucket-pipelines.yml` : push to `develop` → lint step → targeted kms/s3 apply → artifact upload → full apply → seed. Name the AWS identity in use at each step, and where it came from.
2. Open `global/oidc/locals.tf:23-27` next to the `branches:` keys in `bitbucket-pipelines.yml` . Write down, for each environment, the branch the pipeline uses and the branch the trust policy allows. If they differ, that deploy cannot assume its role.
3. Delete `trigger: manual` from the Prod step in your head and describe the failure mode in one sentence. Then delete `-out=/tmp/prod-full.tfplan` and describe a different one.
4. Find every `${VAR:?…}` in the file. Each is a fail-closed guard — the step dies if the repo variable is unset rather than silently downloading something unverified.

L20 · How Code Reaches Production

You've got it when you can…

…explain that CI never holds an AWS key — it presents a per-step signed token, STS returns a one-hour session for exactly one environment's role — and then name the three things that stand between a merge and production: **a human on `trigger: manual`, a saved plan file that fails closed, and a SHA256 on every binary the runner downloads.**

MODULE 2 · LESSON 21

Least Privilege, For Real

Taught from our own audit. Two HIGH findings: the code as it was, and the code as it is now.

1 · THE ROLE THAT OWNED EVERYTHING

HIGH-06 · THE GLUE JOB ROLE

Every Glue job assumes this one role. The audit found it could read or delete any object in the account — tfstate included.

```
BEFORE   AmazonS3FullAccess · s3tables:* glue:* on "*"
         a bad spec could exfiltrate or wipe any bucket
```

```
AFTER    s3:Get/Put/DeleteObject on 3 NAMED buckets
         s3tables / glue actions on project ARNs only
```

infra/modules/iam/main.tf:58-116 · permissions_boundary on all 5 roles

2 · THE KEY THAT TRUSTED EVERYONE

HIGH-07 · SIX KMS CMKs PER ENV

bronze · silver · redshift · secrets · audit · lambda. Each one trusted 15 AWS services on Resource:"*" with no condition.

```
BEFORE   Principal: 15 services   Resource: "*"
         confused deputy — ANY account could ride a service in
```

```
AFTER    Condition = { StringEquals = {
           "aws:SourceAccount" = <this account> } }
```

infra/modules/kms/main.tf:74-82 · per-layer narrowing hook still unused

3 · THE THREE THINGS "SCOPED" ACTUALLY MEANS HERE

WHAT SCOPING CANNOT FIX

Some AWS actions accept no resource ARN. Those stay on "*" — by design, not neglect.

```
lakeformation:GetDataAccess → "*"
ec2:Describe* · glue:GetCatalog
```

infra/modules/iam/main.tf:101-105

LAKE FORMATION IS THE DOOR

Strict mode: the default permission is NONE. glue-role gets CREATE_TABLE; redshift SELECT.

```
LF tags: layer · sensitivity · domain
IAM on its own opens nothing here
```

catalog_federation/lf_grants.tf:28-91

SECRETS BY REFERENCE

Terraform owns the secret's name and its CMK. Never the value — so never the state.

```
SECRET_ID = <arn>  # Glue reads it
ignore_changes = [secret_string]
```

infra/modules/secrets/main.tf:20-31

IN PLAIN ENGLISH

Least privilege is not a setting, it is a diff. Two real ones: a key that stopped trusting everyone, a role that lost every bucket.

L21 · Least Privilege, For Real

Slide: `_render/L21-security-governance.html`

The point

This lesson is taught from our own security audit, because the honest version teaches better than the tidy one. Two HIGH findings, the code as it was, the code as it is now:

- **HIGH-06** — the Glue execution role carried the AWS-managed `AmazonS3FullAccess` plus `s3tables:*` and `glue:*` on `Resource: "*"`. Every Glue job could read or delete **any object in the account**, including the Terraform state bucket and the 7-year Object-Lock CloudTrail bucket.
- **HIGH-07** — all six KMS CMKs granted 13–15 AWS service principals `Encrypt/Decrypt/ReEncrypt*/GenerateDataKey*/DescribeKey` on `Resource: "*"` **with no** `Condition`. That is the textbook cross-service confused deputy: any resource in any account that can induce one of those services to call KMS could use our keys.

Both are fixed. Neither fix is "add a policy" — both are diffs you can read.

L21 · Least Privilege, For Real

- **What "scoped" means.** After HIGH-06 the Glue role's raw S3 access is three named buckets (`-landing-` , `-raw-` , `-artifacts-` , each `${resource_prefix}-…-${account_id}`); `s3tables:Get*/Put*/List*` is bounded to `arn:aws:s3tables:<region>:<account>:bucket/*` ; the Glue catalog actions are bounded to `database/${resource_prefix}_*` , `table/${resource_prefix}_*/*` and `job/${resource_prefix}-*` . Every one of the five workload roles also carries a `permissions_boundary` .
- **What "scoped" does *not* mean.** Some AWS actions accept no resource ARN and legitimately stay on `"*"` : `lakeformation:GetDataAccess` (with an in-code comment saying exactly that), `ec2:Describe*` , `glue:GetCatalog` -style catalog reads. Least privilege is not "zero stars" — it is "every star is deliberate and commented". When you review IAM here, your job is to check that each `"*"` has a reason next to it.
- **The confused-deputy fix is one `Condition` .** `aws:SourceAccount = <this account>` on the `AllowAwsServiceUse` statement. The CloudWatch Logs statement immediately below had always been correctly scoped by encryption context (`ArnLike` on `kms:EncryptionContext:aws:logs:arn`) — the audit's remediation was literally "copy the pattern that is already in this file".
- **Six CMKs, one per layer** — `bronze` , `silver` , `redshift` , `secrets` , `audit` , `lambda` — in each of Dev, QA and Prod. The point of six keys is that a compromise of one does not decrypt the others. **Open gap:** the module ships `var.service_principals_by_layer` to narrow each key's trusted-service list (redshift key → `redshift.amazonaws.com` only, audit key → CloudTrail/SNS/SQS), but **no environment passes it**, so all six still share the 15-principal default. The confused-deputy hole is closed; the six-key separation is not yet honoured.
- **Lake Formation is the door, not IAM.** LF runs in **strict mode**: `create_database_default_permissions = []` and `create_table_default_permissions = []` , i.e. the `IAMAllowedPrincipals` fallback is off and the default permission on anything new is NONE. IAM alone opens nothing. Access comes from explicit `aws_lakeformation_permissions` grants, codified in Terraform so all three envs stay reproducible.
- **The two principals that actually hold grants.** The Glue role gets `CREATE_TABLE / ALTER / DROP / DESCRIBE` on the `bronze_<env>` and `silver_<env>` mirror databases plus `ALL` on their tables (wildcard) — it deletes and recreates the mirror table on every write. The Redshift role gets `DESCRIBE` on the silver mirror DB and `SELECT / DESCRIBE` on its tables, which is what lets Spectrum read `silver_external.<table>` from dbt.
- **LF-TBAC** — the tag vocabulary exists as three LF tag keys with fixed value sets: `layer` (bronze/silver/gold), `sensitivity` (public/internal/confidential/pii), `domain` (sap/ncr/ecommerce/shared). Today the grants in `lf_grants.tf` are **resource-scoped, not tag-scoped**; the tags are provisioned and ready for tag-based grants as the table count grows. Say that plainly rather than claiming TBAC is in force.
- **Secrets by reference, always.** Terraform creates the secret's *name*, its CMK and its rotation config — never its value. A placeholder version is written once and then pinned with `lifecycle { ignore_changes = [secret_string] }` , so a real credential set out-of-band is never reverted and never enters state. Consumers pass an **ARN**: the SAP Glue connection sets `SECRET_ID = module.secrets.secret_arns["sap-db"]` and Glue resolves username/password at run time. Redshift goes further — `manage_admin_password = true` means AWS owns the admin secret and no password touches Terraform at all.
- **The audit's own headline is worth repeating to the team:** the things the coding team was allowed to touch are done to a high standard. The three that remain (CRIT-01 Prod state isolation, CRIT-02 the `tamimi-lakehouse` vs `tamimi-dlh` slug schism, CRIT-03 the deploy-role privilege-escalation path) all need a human decision, not a commit.

L21 · Least Privilege, For Real

- **What "scoped" means.** After HIGH-06 the Glue role's raw S3 access is three named buckets (`-landing-` , `-raw-` , `-artifacts-` , each `${resource_prefix}-…-${account_id}`); `s3tables:Get*/Put*/List*` is bounded to `arn:aws:s3tables:<region>:<account>:bucket/*` ; the Glue catalog actions are bounded to `database/${resource_prefix}_*` , `table/${resource_prefix}_*/*` and `job/${resource_prefix}-*` . Every one of the five workload roles also carries a `permissions_boundary` .
- **What "scoped" does *not* mean.** Some AWS actions accept no resource ARN and legitimately stay on `"*"` : `lakeformation:GetDataAccess` (with an in-code comment saying exactly that), `ec2:Describe*` , `glue:GetCatalog` -style catalog reads. Least privilege is not "zero stars" — it is "every star is deliberate and commented". When you review IAM here, your job is to check that each `"*"` has a reason next to it.
- **The confused-deputy fix is one `Condition` .** `aws:SourceAccount = <this account>` on the `AllowAwsServiceUse` statement. The CloudWatch Logs statement immediately below had always been correctly scoped by encryption context (`ArnLike` on `kms:EncryptionContext:aws:logs:arn`) — the audit's remediation was literally "copy the pattern that is already in this file".
- **Six CMKs, one per layer** — `bronze` , `silver` , `redshift` , `secrets` , `audit` , `lambda` — in each of Dev, QA and Prod. The point of six keys is that a compromise of one does not decrypt the others. **Open gap:** the module ships `var.service_principals_by_layer` to narrow each key's trusted-service list (redshift key → `redshift.amazonaws.com` only, audit key → CloudTrail/SNS/SQS), but **no environment passes it**, so all six still share the 15-principal default. The confused-deputy hole is closed; the six-key separation is not yet honoured.
- **Lake Formation is the door, not IAM.** LF runs in **strict mode**: `create_database_default_permissions = []` and `create_table_default_permissions = []` , i.e. the `IAMAllowedPrincipals` fallback is off and the default permission on anything new is NONE. IAM alone opens nothing. Access comes from explicit `aws_lakeformation_permissions` grants, codified in Terraform so all three envs stay reproducible.
- **The two principals that actually hold grants.** The Glue role gets `CREATE_TABLE / ALTER / DROP / DESCRIBE` on the `bronze_<env>` and `silver_<env>` mirror databases plus `ALL` on their tables (wildcard) — it deletes and recreates the mirror table on every write. The Redshift role gets `DESCRIBE` on the silver mirror DB and `SELECT / DESCRIBE` on its tables, which is what lets Spectrum read `silver_external.<table>` from dbt.
- **LF-TBAC** — the tag vocabulary exists as three LF tag keys with fixed value sets: `layer` (bronze/silver/gold), `sensitivity` (public/internal/confidential/pii), `domain` (sap/ncr/ecommerce/shared). Today the grants in `lf_grants.tf` are **resource-scoped, not tag-scoped**; the tags are provisioned and ready for tag-based grants as the table count grows. Say that plainly rather than claiming TBAC is in force.
- **Secrets by reference, always.** Terraform creates the secret's *name*, its CMK and its rotation config — never its value. A placeholder version is written once and then pinned with `lifecycle { ignore_changes = [secret_string] }` , so a real credential set out-of-band is never reverted and never enters state. Consumers pass an **ARN**: the SAP Glue connection sets `SECRET_ID = module.secrets.secret_arns["sap-db"]` and Glue resolves username/password at run time. Redshift goes further — `manage_admin_password = true` means AWS owns the admin secret and no password touches Terraform at all.
- **The audit's own headline is worth repeating to the team:** the things the coding team was allowed to touch are done to a high standard. The three that remain (CRIT-01 Prod state isolation, CRIT-02 the `tamimi-lakehouse` vs `tamimi-dlh` slug schism, CRIT-03 the deploy-role privilege-escalation path) all need a human decision, not a commit.

L21 · Least Privilege, For Real

Words you'll hear

WORD	WHAT IT MEANS HERE
Confused deputy	Service A is tricked into using your key on someone else's behalf
`aws:SourceAccount`	Condition key that pins a service-principal grant to one account
CMK	Customer-managed KMS key — one per layer, six per environment
Permissions boundary	A ceiling policy; the role can never exceed it, whatever is attached
LF strict mode	Default catalog permission is NONE; `IAMAllowedPrincipals` disabled
LF-TBAC	Tag-based access control — grant on `sensitivity=pii`, not on table names
Mirror database	`bronze_<env>` / `silver_<env>` in the default Glue catalog, for Spectrum
Secret by reference	Code holds the ARN; the value is fetched at run time and never stored

In this repo

- `infra/modules/iam/main.tf:58-62` — the HIGH-06 comment recording that `AmazonS3FullAccess` was removed and why the managed `AmazonS3TablesFullAccess` covers the Iceberg writer's data plane; `:64-79` the narrowed `s3tables` statement; `:80-100` the narrowed `glue` statement; `:101-105` the `lakeformation:GetDataAccess` `"*"` with its justification; `:106-116` the scoped bucket access; `:13-20` the bucket-ARN derivation; `:39` the `permissions_boundary`.
- `infra/modules/iam/main.tf:309-317` — the Redshift role comment: Spectrum uses `s3tables:GetTableData`, "so we do NOT need `s3:*` on this role". Two different doors into the same bytes (see M1 L13).
- `infra/modules/kms/main.tf:74-82` — the HIGH-07 `Condition` with the AWS least-privilege doc link; `:5-31` the shared 15-principal default list; `:33-35` the per-layer lookup; `:85-104` the CloudWatch Logs statement that was already correct.
- `infra/modules/kms/variables.tf:22-31` — `service_principals_by_layer`, with a worked example in the comment. Grep the env roots: nothing passes it.
- `infra/modules/lake-formation/main.tf:25-44` — strict mode, and the `aws_iam_session_context` trick that resolves the assumed-role session back to its role ARN so the Bitbucket OIDC identity counts as a data-lake admin; `:71-80` the LF tags; `variables.tf:22-30` the tag vocabulary.
- `infra/modules/catalog_federation/lf_grants.tf:28-91` — the six grants, with a header explaining that under strict mode the writer's `glue:CreateTable` and Spectrum's read both fail without them.
- `infra/modules/secrets/main.tf:7-31` — metadata-only ownership and `ignore_changes`; `infra/env/dev/main.tf:84-102` — the five secret names and the comment forbidding table/schema names in secrets.
- `infra/env/dev/sap_hana_source_download.tf:75-95` — `SECRET_ID` wired from the module output, "keeps creds out of Terraform state".
- `docs/handoff/audit_deep_analysis.md:183-195` — HIGH-06 and HIGH-07 in full; `docs/handoff/audit-remediation-verification.md:40-41` — the verified-fixed evidence.

L21 · Least Privilege, For Real

Do this

1. `git log -p -- infra/modules/kms/main.tf` and find the commit that adds the `Condition` block. That four-line diff is the whole HIGH-07 fix. Read it, then write down in one sentence what an attacker could have done before it.
2. In `infra/modules/iam/main.tf`, list every statement whose `resources` is `["*"]`. For each, decide from the comment whether it is deliberate. If a comment is missing, that is your review finding.
3. Set `service_principals_by_layer = { redshift = ["redshift.amazonaws.com"] }` on the Dev `module "kms"` block and run `terraform plan`. Read which key policies change and predict what would break if you narrowed the `audit` key the same way (hint: CloudWatch alarms publishing to a CMK-encrypted SNS topic).
4. Try to find a credential value in the repo or in state. You should fail — and be able to say exactly which two mechanisms guarantee that (`ignore_changes` on the placeholder, and `manage_admin_password` for Redshift).

You've got it when you can…

…take a reviewer through both diffs from memory — `AmazonS3FullAccess` → **three named buckets**, and **service-principal grant → the same grant plus** `aws:SourceAccount` — say which `"*"` s are legitimate and why, explain that Lake Formation strict mode means IAM alone opens nothing, and name the one governance gap still open (the six CMKs still share one 15-service principal list).

MODULE 2 · LESSON 22

Getting to SAP Safely

Three subnet tiers, one route across the Transit Gateway, TLS on the wire — and a hard ceiling made of IP addresses.

1 · THREE SUBNET TIERS, THREE WAYS OUT

PUBLIC · TWO /28s

The Internet Gateway and the NAT sit here.
No workload of ours runs in them.
Egress: straight to the internet.

```
10.248.4.192/28 · 10.248.4.208/28
dev 1 NAT · prod 1 NAT per AZ
```

PRIVATE · TWO /28s

The dbt Glue runner and the Lambdas.
Egress via NAT — plus 7 interface endpoints so AWS calls skip the NAT.

```
secretsmanager kms glue sts logs
sqs monitoring — interface, :443
```

DATA · THREE /26s

Redshift and every SAP Glue ENI.
No NAT. No interface endpoints.
Gateway endpoints and the TGW only.

```
10.248.4.0/26 · .64/26 · .128/26
59 usable IPs each
```

`VPC flow logs: traffic_type "ALL" → CMK-capable CloudWatch group, 90-day retention (HIGH-08)  ·  infra/modules/vpc/main.tf:236-300`

2 · THE ROUTE TO SAP — AND THE TLS ON IT

THE ROUTE TO ON-PREM SAP

One aws_route per route table, pointing at the shared TGW. Prod puts them on the DATA tables.

```
dev   172.30.1.0/24 + 172.30.0.0/16
qa    172.30.1.0/24
prod  172.30.5.45/32   ← one host only
```

`infra/env/prod/main.tf:642-666 · terraform.tfvars:46-55`

TLS ON THE JDBC PATH

SAP credentials and full table extracts cross it. Both must be on:

```
...?encrypt=true
   &validateCertificate=true
JDBC_ENFORCE_SSL = "true"
```

`dev/sap_hana_source_download.tf:100`

3 · THE CEILING

WHY P1 CONCURRENCY IS 6

Each Glue worker takes one IP in the subnet its connection pins to.

```
"Number of IP addresses on subnet is 0"
/26 = 59 usable · 3 lanes x 2 runs
x 12 workers = 24 ENIs at peak
```

`env/dev/source_download.tf:58-68`

IN PLAIN ENGLISH

The data subnets have no door to the internet. The one way out is the tunnel to SAP — and free IP addresses cap the worker count.

L22 · Getting to SAP Safely

Slide: `_render/L22-networking-sap-connectivity.html`

The point

SAP is not in AWS. It is on-premises, behind someone else's firewall, reachable only across a shared Transit Gateway. Four things have to be true before a byte moves, and the fourth is the one that surprises application developers:

1. The Glue worker must sit in a subnet whose **route table** points the SAP CIDR at the TGW.
2. The VPC must have an **available attachment** to that TGW (managed out-of-band, by another team).
3. The connection must be **encrypted** — TLS on the JDBC URL *and* on the Glue connection.
4. There must be a **free IP address** in that subnet. This is what caps download concurrency, not CPU and not SAP.

L22 · Getting to SAP Safely

- **Three subnet tiers, three different exits.** Public (two `/28` s) holds the Internet Gateway and the NAT — no workload of ours runs there. Private (two `/28` s) holds the dbt Glue runner and the Lambdas; it reaches the internet via NAT and reaches AWS APIs through **seven interface endpoints** (`secretsmanager` , `kms` , `glue` , `sts` , `logs` , `sqs` , `monitoring`) so those calls stay on the AWS backbone. Data (three `/26` s) holds Redshift Serverless and every SAP Glue ENI — **no NAT route and no interface endpoints**.
- **The data subnets are deliberately no-egress.** Their route tables carry `local` , the S3 and DynamoDB **gateway** endpoints, and (in Prod) the TGW route to SAP. Nothing else. That is why the Glue jobs bootstrap Python with `--no-index` from a vendored wheel closure in S3 instead of PyPI, and why the DynamoDB gateway endpoint is load-bearing: in QA on 2026-07-21 the `source_download` runs died on connect timeouts to `dynamodb.eu-west-1.amazonaws.com` the moment their ENIs moved into the `/26` s.
- **Interface vs gateway matters.** Gateway endpoints (S3, DynamoDB) are **route-table** entries — free, and they work in a subnet with no NAT. Interface endpoints are **ENIs with a security group** in specific subnets — here, the private ones only. If your workload is in a data subnet and calls Secrets Manager, it is not using an endpoint; check before you assume.
- **Redshift Serverless forces three data subnets in three AZs** ("at least 9 free IPs in 3 subnets, each in a different AZ"). That constraint is why the data tier is three `/26` s, which is also what gives the SAP lanes their address pool.
- **The TGW route is one `aws_route` per route table.** The module fans `route_table_ids × routes` into a keyed map, keyed on route-table **index** rather than id so `for_each` stays known at plan time. It also carries a `lifecycle { precondition }` that checks for an *available* VPC→TGW attachment, because `CreateRoute` toward an unattached TGW fails after ~5 minutes with the misleading `InvalidTransitGatewayID.NotFound` — which is exactly what happened on 2026-07-20 when the network-hub team deleted this account's attachment out of band.
- **Which route table you attach it to is a correctness question, and Prod is the one that got it right.** Dev and QA pass `module.vpc.private_route_table_ids` ; Prod passes `module.vpc.data_route_table_ids` . The SAP Glue connections pin their ENIs to the **data** subnets, so those ENIs read the data route tables. On 2026-08-04 every Prod SAP pull timed out at the socket while the private tables carried a SAP route nothing used. There is a second reason too: `modules/vpc` declares `aws_route_table.private` with a **dynamic inline `route` block** for NAT, and mixing inline routes with standalone `aws_route` on the same table is unsupported — each apply reconciles one and strips the other. `aws_route_table.data` has no inline routes, so standalone routes stick.
- **Blast radius shrinks per environment.** Dev routes `172.30.1.0/24` **and** `172.30.0.0/16` ; QA routes only the `/24` ; Prod routes a single host, `172.30.5.45/32` . The audit's HIGH-19 was that the original config routed `172.16.0.0/12` and `192.168.0.0/16` on-prem — nearly the whole private address space, in both directions. Those are gone; Dev's `/16` is the remaining "partial".
- **TLS on the JDBC path is two switches, not one.** The URL must carry `?encrypt=true&validateCertificate=true` *and* the Glue connection must set `JDBC_ENFORCE_SSL = "true"` . Before HIGH-09, the extractor defaulted to `encrypt=False, sslValidateCertificate=False` and the connection shipped with `JDBC_ENFORCE_SSL="false"` — SAP ERP credentials and full sales/customer extracts crossing the TGW in cleartext. The prerequisite nobody skips: the HANA server certificate has to be in the Glue connection's truststore or validation fails.
- **VPC flow logs exist now (HIGH-08).** VPC-scope, `traffic_type = "ALL"` , to a CloudWatch log group with 90-day retention and its own delivery role. `flow_log_kms_key_arn` defaults to `""` — no env passes a CMK today, so the group uses CloudWatch's default encryption. Worth knowing when someone asks "are the flow logs CMK-encrypted?": the wiring is there, the value is not.
- **The ENI budget is the real concurrency ceiling.** A `/26` gives 59 usable addresses. Each Glue worker consumes one, and Glue's ENI teardown lags a finished run by 1–2 minutes. The P1 download therefore runs **3 lanes** — three Glue connections pinned to `data-0/1/2` — with `max_concurrent_runs = 2` per lane and `G.2X × 12` workers. `3 × 2 × 12 ≈ 24 ENIs` at peak, ~35 free. The Step Functions `Map` sets `MaxConcurrency = 6` , which **must** equal the sum of the lanes' `max_concurrent_runs` , or a retry stacks an extra run onto a subnet instead of queueing on the pool. Overshoot and chunks die with `Number of IP addresses on subnet is 0` .
- **`max_retries = 0` on the download jobs is part of the same budget.** A Glue-*internal* retry starts `<run>_attempt_1` without passing through `StartJobRun` , so it ignores the Map's lane budget and stacks another 12 ENIs. Retry lives only in the Map's `Retry` block, which re-issues `StartJobRun` and therefore honours the pool.
- **There is a second, softer ceiling:** `MaxConcurrency × max(jdbc_hash_partitions)` is the concurrent HANA connection count. Specs cap `jdbc_hash_partitions ≤ 8` , so `6 × 8 = 48` of a ~64 connection

L22 · Getting to SAP Safely

- **Three subnet tiers, three different exits.** Public (two `/28`s) holds the Internet Gateway and the NAT — no workload of ours runs there. Private (two `/28`s) holds the dbt Glue runner and the Lambdas; it reaches the internet via NAT and reaches AWS APIs through **seven interface endpoints** (`secretsmanager`, `kms`, `glue`, `sts`, `logs`, `sqs`, `monitoring`) so those calls stay on the AWS backbone. Data (three `/26`s) holds Redshift Serverless and every SAP Glue ENI — **no NAT route and no interface endpoints**.
- **The data subnets are deliberately no-egress.** Their route tables carry `local`, the S3 and DynamoDB **gateway** endpoints, and (in Prod) the TGW route to SAP. Nothing else. That is why the Glue jobs bootstrap Python with `--no-index` from a vendored wheel closure in S3 instead of PyPI, and why the DynamoDB gateway endpoint is load-bearing: in QA on 2026-07-21 the `source_download` runs died on connect timeouts to `dynamodb.eu-west-1.amazonaws.com` the moment their ENIs moved into the `/26`s.
- **Interface vs gateway matters.** Gateway endpoints (S3, DynamoDB) are **route-table** entries — free, and they work in a subnet with no NAT. Interface endpoints are **ENIs with a security group** in specific subnets — here, the private ones only. If your workload is in a data subnet and calls Secrets Manager, it is not using an endpoint; check before you assume.
- **Redshift Serverless forces three data subnets in three AZs** ("at least 9 free IPs in 3 subnets, each in a different AZ"). That constraint is why the data tier is three `/26`s, which is also what gives the SAP lanes their address pool.
- **The TGW route is one `aws_route` per route table.** The module fans `route_table_ids × routes` into a keyed map, keyed on route-table **index** rather than id so `for_each` stays known at plan time. It also carries a `lifecycle { precondition }` that checks for an *available* VPC→TGW attachment, because `CreateRoute` toward an unattached TGW fails after ~5 minutes with the misleading `InvalidTransitGatewayID.NotFound` — which is exactly what happened on 2026-07-20 when the network-hub team deleted this account's attachment out of band.
- **Which route table you attach it to is a correctness question, and Prod is the one that got it right.** Dev and QA pass `module.vpc.private_route_table_ids`; Prod passes `module.vpc.data_route_table_ids`. The SAP Glue connections pin their ENIs to the **data** subnets, so those ENIs read the data route tables. On 2026-08-04 every Prod SAP pull timed out at the socket while the private tables carried a SAP route nothing used. There is a second reason too: `modules/vpc` declares `aws_route_table.private` with a **dynamic inline `route` block** for NAT, and mixing inline routes with standalone `aws_route` on the same table is unsupported — each apply reconciles one and strips the other. `aws_route_table.data` has no inline routes, so standalone routes stick.
- **Blast radius shrinks per environment.** Dev routes `172.30.1.0/24` **and** `172.30.0.0/16`; QA routes only the `/24`; Prod routes a single host, `172.30.5.45/32`. The audit's HIGH-19 was that the original config routed `172.16.0.0/12` and `192.168.0.0/16` on-prem — nearly the whole private address space, in both directions. Those are gone; Dev's `/16` is the remaining "partial".
- **TLS on the JDBC path is two switches, not one.** The URL must carry `?encrypt=true&validateCertificate=true` *and* the Glue connection must set `JDBC_ENFORCE_SSL = "true"`. Before HIGH-09, the extractor defaulted to `encrypt=False, sslValidateCertificate=False` and the connection shipped with `JDBC_ENFORCE_SSL="false"` — SAP ERP credentials and full sales/customer extracts crossing the TGW in cleartext. The prerequisite nobody skips: the HANA server certificate has to be in the Glue connection's truststore or validation fails.
- **VPC flow logs exist now (HIGH-08).** VPC-scope, `traffic_type = "ALL"`, to a CloudWatch log group with 90-day retention and its own delivery role. `flow_log_kms_key_arn` defaults to `""` — no env passes a CMK today, so the group uses CloudWatch's default encryption. Worth knowing when someone asks "are the flow logs CMK-encrypted?": the wiring is there, the value is not.
- **The ENI budget is the real concurrency ceiling.** A `/26` gives 59 usable addresses. Each Glue worker consumes one, and Glue's ENI teardown lags a finished run by 1–2 minutes. The P1 download therefore runs **3 lanes** — three Glue connections pinned to `data-0/1/2` — with `max_concurrent_runs = 2` per lane and `G.2X × 12` workers. `3 × 2 × 12 ≈ 24 ENIs` at peak, ~35 free. The Step Functions `Map` sets `MaxConcurrency = 6`, which **must** equal the sum of the lanes' `max_concurrent_runs`, or a retry stacks an extra run onto a subnet instead of queueing on the pool. Overshoot and chunks die with `Number of IP addresses on subnet is 0`.
- **`max_retries = 0` on the download jobs is part of the same budget.** A Glue-*internal* retry starts `<run>_attempt_1` without passing through `StartJobRun`, so it ignores the Map's lane budget and stacks another 12 ENIs. Retry lives only in the Map's `Retry` block, which re-issues `StartJobRun` and therefore honours the pool.
- **There is a second, softer ceiling:** `MaxConcurrency × max(jdbc_hash_partitions)` is the concurrent HANA connection count. Specs cap `jdbc_hash_partitions ≤ 8`, so `6 × 8 = 48` of a ~64 connection

L22 · Getting to SAP Safely

Words you'll hear

WORD	WHAT IT MEANS HERE
ENI	Elastic Network Interface — one private IP; every Glue worker takes one
Gateway endpoint	A route-table entry for S3/DynamoDB; works with no NAT, costs nothing
Interface endpoint	An ENI + SG in named subnets; here, private subnets only
TGW	Transit Gateway — the shared hub that reaches on-prem SAP
Attachment	The VPC↔TGW link; owned by the network-hub account, not by us
Lane	One Glue connection pinned to one data subnet; three of them spread the ENIs
`MaxConcurrency`	The Step Functions `Map` fan-out cap; must equal the sum of lane run caps
Flow log	A record of accepted/rejected traffic; forensics after the fact

In this repo

- `infra/modules/vpc/main.tf:18-53` — the three subnet tiers; `:112-148` the private route tables (inline NAT route) versus the bare data route tables; `:153-186` the S3 and DynamoDB gateway endpoints, with the QA incident note; `:193-230` the interface endpoints and their `:443` -from-VPC security group; `:236-300` the HIGH-08 flow-log stack.
- `infra/modules/vpc/variables.tf:41-72` — `enable_flow_logs`, `flow_log_kms_key_arn` (default `""`), and the seven `interface_endpoint_services`.
- `infra/env/dev/terraform.tfvars:3-12` — the CIDR plan, including why three data subnets exist; `:55-69` the TGW routes with the HIGH-19 comment.
- `infra/env/prod/terraform.tfvars:46-55` — Prod's `/32`, and why Prod must not path to non-prod SAP.
- `infra/env/prod/main.tf:642-666` — the two-reason comment for routing on the **data** tables. The best 15 lines of networking prose in the repo.
- `infra/modules/tgw-route/main.tf:43-79` — the attachment `precondition` and its error message.
- `infra/env/dev/sap_hana_source_download.tf:90-108` — `JDBC_CONNECTION_URL`, `SECRET_ID`, `JDBC_ENFORCE_SSL = "true"` and the `physical_connection_requirements` block that places the ENI.
- `infra/env/dev/main.tf:410-451` — the `sap-hana` / `sap-hana-lane1` / `sap-hana-lane2` Glue connections and the `/28` -exhaustion story that moved them onto the `/26`s.
- `infra/env/dev/glue_sources.tf:113-181` — the three lane job definitions with the ENI arithmetic in the comments.
- `infra/env/dev/source_download.tf:40-68` — `MaxConcurrency = 6` and the measured reason it dropped from 9.
- `docs/handoff/audit_deep_analysis.md:197-209` — HIGH-08 and HIGH-09 as written by the auditor.

L22 · Getting to SAP Safely

Do this

1. Draw the Dev VPC from `terraform.tfvars` alone: `10.248.4.0/24` split into two public `/28` s, two private `/28` s, three data `/26` s. Mark which tiers have a NAT route, which have gateway endpoints, and which have interface endpoints.
2. Answer this without running anything: a Lambda in a **private** subnet calls Secrets Manager — does the traffic leave the VPC? Now the same call from a Glue worker in a **data** subnet. (The second one has no path at all.)
3. Open `infra/env/dev/main.tf` and `infra/env/prod/main.tf` at their `module "sap_tgw_route"` blocks. One argument differs. Explain both reasons Prod gives, then decide whether Dev should change.
4. Do the ENI arithmetic yourself for a hypothetical fourth lane on a `/28` instead of a `/26` , and say at which worker count it breaks.
5. Grep for `JDBC_ENFORCE_SSL` and `encrypt=` across `infra/` and `scripts/` . Both switches must agree — find any place they do not.

You've got it when you can…

…name the four preconditions for a SAP pull (**route on the right table, an available TGW attachment, TLS on both switches, and a free IP**), explain why Prod puts the TGW route on the data route tables and Dev does not, and derive `MaxConcurrency = 6` from first principles: `59 usable IPs per /26 → 3 lanes × 2 runs × 12 workers ≈ 24 ENIs, with headroom for teardown lag.`

MODULE 2 · LESSON 23

It's 07:00 and Gold Is Missing

Five moves from the alert email to the right recovery — and the three recoveries you get to choose from.

1 · THE PATH — FIVE MOVES, IN ORDER

1 THE ALARM

SNS email: stage, table, cycle, the exact error

`shared/alerting.py`

→

2 THE CONSOLE

DynamoDB → runs → Query the by_cycle index

`tamimi-lakehouse-runs-dev`

→

3 THE RUN ROW

Key = (run_id, stage). Read error_message.

`control_plane/runs.py`

→

4 THE GLUE LOG

Glue → Jobs → that stage's job → Runs → CloudWatch

`…-sap-source_download`

→

5 THE WATERMARK

Did it advance? Is the value even plausible?

`control_plane/watermarks.py`

2 · SYMPTOM → LIKELY CAUSE → ACTION

SYMPTOM	LIKELY CAUSE	ACTION
source_download FAILED	SAP host down / secret rotated	Fix cause → rerun (Mode A)
Watermark frozen, 0 rows	Dirty MAX() the guard refuses	Reset value → backfill window
Cycle still 'running' at 07:00	Barrier died; nothing re-checks	cycle_sweeper resumes it
MERGE_CARDINALITY_VIOLATION	Duplicate PKs vs the merge_key	Fix merge_key → rerun
Gold is yesterday's at 07:00	gold-build failed post-barrier	Re-fire gold-build only

Every alert email links its own runbook — docs/runbooks/<stage>-failure.md

3 · THEN PICK THE SMALLEST FIX

RERUN — same-day recovery

Re-runs the failed tables in the same cycle → Gold

```
{"rerun":true,"cycle_id":"2026-08-10"}
```

BACKFILL — date-window re-pull

Own bf- cycle, watermark untouched, full-refresh Gold

```
{"rerun":true,"source":"sap","date_from":"2026-06-01"}
```

RE-BASELINE — full reload

Watermark is beyond saving — load the table from zero

```
{"initial_load":true,"force":true,"tables":["sap.mara"]}
```

IN PLAIN ENGLISH

Nobody guesses. The pipeline already wrote down what happened — read the row, then pick the smallest fix that still reaches Gold.

MODULE 2 · LESSON 23

L23 · It's 07:00 and Gold Is Missing — reading a broken cycle

Slide: `_render/L23-debugging-a-broken-cycle.html`

The point

Power BI is DirectQuery, so report freshness is exactly "the most recent dbt Gold build" — **daily at 07:00 KSA** (`docs/CLIENT-TECHNICAL-DESIGN.md:442`). When Gold isn't there, you have maybe twenty minutes and a room full of people asking. The instinct is to re-run something. **Don't.** The platform has already written down what happened, in a fixed order, in places you can name. This lesson is that order — five moves from the alert email to a diagnosis, then one decision between three recoveries of very different cost.

The whole discipline is: **read before you act, and then pick the smallest fix that still reaches Gold.**

L23 · It's 07:00 and Gold Is Missing — reading a broken cycle

Key ideas

- **The alert email is not a notification, it's the first evidence.** `run_status` sees every terminal Glue state and emails the *stage*, *table*, *cycle*, the run's own recorded `error_message`, console links and the runbook next-steps. The ERROR block **is** the `runs` row — you are not being told "something failed", you are being handed the blocker.
- **Expect two emails for one failure.** The per-table stage email (what broke), and the cycle-level skip email (`gold_skipped` — what got held back). They are different facts.
- **The `runs` table is the flight recorder.** Keyed `(run_id HASH, stage RANGE)`, so one table has *several rows sharing one run_id* — `source_download`, `bronze_pull`, `bronze_to_silver`. Query the `by_cycle` GSI to see a whole day; the barrier resolves each table to its **latest** stage, and so should you.
- **The console scan view lies by omission.** DynamoDB's console shows a *filtered page* ("2/50" = 2 matched of 50 examined). Use the paginating CLI scan for the full cycle picture.
- **All-or-nothing is deliberate.** One failed table → `gold_skipped` for the whole cycle. Gold must never build from a partial day. So "Gold is missing" almost always means "exactly one table failed" — find it, don't rebuild everything.
- **Barriers are edge-triggered; the sweeper is level-triggered.** Each barrier is invoked once, at the end of a source Step Function. If a barrier *dies*, nothing re-evaluates it and the cycle sits in `running` forever. `cycle_sweeper` runs daily and does two different things: **RESUME** (nudge the barrier the cycle's state implies) and, only past the deadline *and* stalled, **CONCLUDE** (force-fail the non-terminal tables so the cycle can end).
- **Age is not evidence of death.** The sweeper originally concluded on wall-clock alone and force-failed three healthy QA initial reloads of 67–70 tables. Now the deadline only *opens* the question (`SWEEP_DEADLINE_HOURS`, 12) and progress answers it (`SWEEP_STALL_HOURS`, 3), with an absolute ceiling (`SWEEP_MAX_AGE_HOURS`, 96).
- **A dirty watermark is the failure that doesn't look like a failure.** A garbage `MAX()` value that sorts above every real one becomes the stored watermark, and every future `wm_col > '<wm>'` predicate then matches **nothing, forever** — green runs, zero rows, silent data loss. Both connectors now refuse to persist a watermark they would refuse to splice, refuse to move it backwards, and (for typed columns) bound the aggregate to a plausible range.
- **`MERGE_CARDINALITY_VIOLATION` means your merge_key isn't the PK — or the batch has dupes.** Iceberg forbids matching one target row to multiple source rows. `bronze_pull` defensively `dropDuplicates(merge_key)` before the upsert, because a retried `source_download` can double-write the same `cycle=<id>/` prefix (seen for real on `mbew` / `mbewh`, QA 2026-07-23).
- **Three recoveries, escalating.** Re-run (reopen today's cycle), backfill (a date-window re-pull in its own `bf-…` cycle, watermark untouched, ending in a `--full-refresh` Gold), re-baseline (throw the incremental state away and load the table from zero). Pick the cheapest one that actually fixes the data.

L23 · It's 07:00 and Gold Is Missing — reading a broken cycle

The five moves

#	MOVE	WHERE	WHAT YOU'RE LOOKING FOR
1	**The alarm**	SNS `…-alerts` email	`Stage:` and `Table:` — and the ERROR block
2	**The console**	DynamoDB → `tamimi-lakehouse-runs-<env>` → Query index `by_cycle`	every table × stage × status for the cycle
3	**The run row**	the `(run_id, stage)` item	`error_message` verbatim; `rows_in` / `rows_out`
4	**The Glue log**	Glue → Jobs → `…-<source>-source_download` → Runs	the stack trace / driver error behind the message
5	**The watermark**	DynamoDB `…-watermarks-<env>`	did `last_watermark_value` advance? is it *plausible*? what is `init_state`?

Then decide. Not before.

Symptom → likely cause → action

SYMPTOM	LIKELY CAUSE	ACTION
`source_download` FAILED	SAP HANA host unreachable from the Glue VPC, or the `…-sap-db` secret rotated	fix the cause → **re-run Mode A**
Watermark frozen, 0 rows every day, no error	dirty/implausible `MAX()` the splice guard refuses (R29/R30)	reset the stored value → **backfill** the missed window
Cycle still `running` well past 07:00	a barrier died mid-cycle; barriers are edge-triggered (R52)	`cycle_sweeper` RESUME nudges it; else invoke the barrier with `{"cycle_id": …}`
`MERGE_CARDINALITY_VIOLATION`	duplicate PKs in the batch, or `merge_key` isn't the verified PK	fix the spec's `merge_key` → **re-run**
Cycle says `gold_built` but Gold is yesterday's	the dbt build failed *after* the barrier marked the cycle (the barrier marks on **start**)	re-fire `…-gold-build` only — no ingest re-run
`download_skipped` / `transform_skipped` / `conform_skipped`	all-or-nothing: an upstream table failed, so the next phase never started	fix the *named* table, then Mode A
Silver error reads `conform input(s) failed: [...]`	the derived table didn't fail — its input did	re-run the **input**, never the derived table

L23 · It's 07:00 and Gold Is Missing — reading a broken cycle

Two rules that catch everybody:

1. **An empty/absent `tables` list means "re-run the failures."** To re-run something that already `succeeded`, you must **name** it. Auto-detect never touches a green table.
2. **Never list a derived/conform table** (e.g. `sales.basket`) in `tables`. It has no source SFN; the conform phase rebuilds it once its inputs go green. List the inputs.

Words you'll hear

WORD	WHAT IT MEANS HERE
cycle_id	the day's identity — the UTC date (`2026-08-10`), or a scoped `bf-…` / `init-…` id
stage	`source_download` → `bronze_pull` → `abap_transform` → `bronze_to_silver` → `silver_to_gold`; the RANGE key of `runs`
barrier	the Lambda that decides whether a phase may start, once every expected table is terminal
single-flight lock	`PutItem` with `ConditionExpression=attribute_not_exists(pk)` — only one racer claims `goldlock#<cycle>`
all-or-nothing	any failed table → the next phase is skipped for the whole cycle
crash-net	`run_status` — patches a phantom RUNNING row to FAILED when a Glue job dies without self-reporting
sweeper	`cycle_sweeper` — resumes stranded cycles; force-concludes genuinely dead ones
wedged watermark	a stored value so high that every future delta predicate matches nothing
re-baseline	throw away incremental state and full-load the table again
freshness	`pipeline_state` entity `freshness:<table>` — `last_success_at` vs `expected_within_seconds`

In this repo

- `docs/OPERATIONS-RUNBOOK.md` — §2 monitor, §3 "did Gold build?", **§4 the four modes (A/B/C/D)**, §4b who gets the emails
- `docs/runbooks/` — one per stage: `source-download-failure.md`, `bronze-pull-failure.md`, `bronze-to-silver-failure.md`, `abap-transform-failure.md`, `gold-build-failure.md`, `cycle-skipped.md`, `dispatcher-failure.md`
- `src/lambdas/run_status/handler.py` — the crash-net **and** the per-stage failure notifier; note `_gold_failure`, which is the *only* place a failed Gold build becomes an alert
- `src/lambdas/cycle_sweeper/handler.py` — read the module docstring and `_should_conclude`; it is the best-written explanation of "slow ≠ dead" in the codebase
- `src/shared/shared/control_plane/runs.py` — `RunStage`, `RunStatus`, `RunKind`, the composite key and the two GSIs
- `src/shared/shared/control_plane/watermarks.py` — `InitState` (`pending` / `initial_loaded` / `cdc` / `needs_reinit`)
- `src/shared/shared/control_plane/coordination.py` — `CycleState`, `CONCLUDED_CYCLE_STATES`, the four lock prefixes
- `src/shared/shared/control_plane/pipeline_state.py` — `FreshnessState` / `AlarmState`; the freshness signal behind "is it late?"
- `src/glue/glue_engine/sources/rds_jdbc.py` — `read_incremental`'s three-part watermark guard (plausible / spliceable / forward-only)
- `src/glue/glue_engine/jobs/bronze_pull.py` — the `dropDuplicates(merge_key)` that exists because of a real cardinality violation

L23 · It's 07:00 and Gold Is Missing — reading a broken cycle

> **Honest note on "the ops console":** there is no bespoke operator UI shipped today — `src/../ui/` is empty and `pipeline_state` is modelled *for* one. The console you actually use is three AWS screens: **Step Functions** (the execution graph), **DynamoDB** (`runs` by `by_cycle` , `coordination`), and **Glue** → **Runs** (the log). Learn those three.

Do this

1. Take the last failed cycle you can find in Dev. Without asking anyone, produce four sentences: *which table*, *which stage*, *the exact* `error_message` , *which of the three recoveries you would run and why*. Then check yourself against the stage runbook.
2. Write out the Mode A / Mode D / re-baseline payloads from memory. Getting `cycle_id` wrong in a re-run is the most common self-inflicted incident.
3. Read `cycle_sweeper.handler` top to bottom and answer: *why are P1 (`source_download`) rows excluded from the force-fail pass but included in the liveness check?*
4. Explain `_is_sweeper_written` to a colleague — why must the sweeper ignore its own writes when deciding whether a cycle is alive?
5. Pick two of the eight Apparel Group sources and write their L23 table row in advance: what will "watermark frozen" look like for a SaaS API cursor (Epsilon, MoEngage) versus a JDBC date column (RMS, SIM, XStore)?

You've got it when you can…

- Name the five places you look, in order, without the slide.
- Say what `runs` , `watermarks` , `coordination` and `pipeline-state` each answer — and which one you would *never* look at first.
- Explain why a cycle can read `gold_built` while Gold is stale, and what you re-fire.
- Choose between rerun / backfill / re-baseline out loud, and justify the choice by cost.
- State the two re-run rules: absent `tables` = failures only; never list a derived table.
- Explain a wedged watermark to a non-engineer in one sentence, and name the guard that prevents it now.
- Say why "the cycle has been running for 14 hours" is **not**, by itself, a reason to force-fail it.

MODULE 2 · LESSON 24

Add a Source Without Touching the Engine

Seven artifacts, in order. None of them is engine code. Walked end-to-end on Oracle Retail RMS ITEM_MASTER.

1 · SEVEN ARTIFACTS — AMBER = ONLY IF NEEDED

2 · WORKED ONCE — ORACLE RETAIL RMS · ITEM_MASTER

STEP 2 · THE DOWNLOAD SPEC

specs/download/sap_mara.yaml — the template

```
table: rms.item_master
source_type: rds_jdbc
source_config:
  jdbc_schema: RMS
  jdbc_table: ITEM_MASTER
  watermark_column: LAST_UPDATE_DATETIME
  jdbc_hash_field: ITEM
  jdbc_hash_partitions: "8"
landing_prefix: "rms/rms.item_master"
schema: [ITEM, ITEM_DESC, DEPT, CLASS, ...]
```

STEP 3 + 4 · BRONZE, CATALOG

specs/bronze/ · config/ingestion_tables.yaml

```
table: rms.item_master
source_type: s3_landing
merge_key: [ITEM]      # the VERIFIED PK
schema: (mirrors the download spec)
- - - - - - - - - - - - - - - - - - - - -
- name: rms.item_master
  source: rms
  bronze_spec: bronze/rms_item_master.yaml
  download_spec: download/rms_item_mas...
  driver_by_mode: { full: rds_jdbc, ... }
```

STEP 5 · dbt STAGING + TEST

src/dbt/models/staging/ + _staging.yml

```
-- stg_rms_item_master.sql
{{ config(materialized='view') }}
SELECT item      AS item_id,
       item_desc AS item_name,
       dept, class, subclass
FROM {{ source('silver','rms_item_master') }}
- - - - - - - - - - - - - - - - - - - - - -
- name: item_id          # _staging.yml
  tests: [not_null, unique]
# Silver, Gold and Power BI never noticed.
```

3 · WHAT YOU NEVER TOUCH

glue_engine/jobs · writers · dispatcher · barriers · control plane · Silver + Gold contracts

THIS IS YOUR CAPSTONE — one source, one PR.

IN PLAIN ENGLISH

You are not building a pipeline. You are filling in seven forms — and the machine that reads them was finished months ago.

L24 · Add a Source Without Touching the Engine — the capstone bridge

Slide: `_render/L24-adding-a-source.html`

The point

Everything in Modules 1 and 2 was building to one claim, and this lesson cashes it: **onboarding a new table is a configuration change, not an engineering project.** Seven artifacts, in a fixed order, none of which is engine code. The dispatcher, the Step Functions, the barriers, the writers, the control plane, the Silver and Gold contracts — untouched.

That claim is not marketing. `sources/__init__.py` says it in its own docstring: *"To add a new source: implement the `SourceConnector` Protocol, decorate with `@register(...)`, add a row in DDB `source_catalog`, and you're done. NO engine changes."* This lesson walks the seven steps once, concretely, against **Oracle Retail RMS** `ITEM_MASTER` — the first table your team will onboard at Apparel Group.

Finish this lesson and you have the capstone spec: *one source, one PR.*

The seven artifacts

#	ARTIFACT	WHERE IT LIVES	WHEN YOU SKIP IT
1	**Connector class**	`src/glue/glue_engine/sources/<type>.py` + `@register("<type>")`	when the protocol is already satisfied — a Postgres or SQL Server source needs **nothing**
2	`download` **spec** (P1)	`src/glue/specs/download/<table>.yaml`	never — this is the pull
3	`bronze` **spec** (P2)	`src/glue/specs/bronze/<table>.yaml`	never — this is the load
4	**Catalog row**	`config/ingestion_tables.yaml` (+ a `connections.yaml` entry per source)	never — this is what the dispatcher scans
5	**dbt staging model + tests**	`src/dbt/models/staging/stg_<table>.sql` + `_staging.yml`	never — Silver must be typed and asserted
6	**Gold model**	`src/dbt/models/marts/gold/*.sql`	when nothing reports on it yet
7	**Deploy**	`bitbucket-pipelines.yml` → spec sync → `scripts/seed_control_plane.py`	never

Step 1 — the connector (only if the protocol isn't already satisfied)

The engine never imports a concrete connector. It calls `get_connector(source_type)` and gets whatever `@register`'d itself at import time. The contract is five methods (`sources/protocol.py`):

L24 · Add a Source Without Touching the Engine — the capstone bridge

For Oracle Retail, this is one branch, not one project. `rds_jdbc.py` already does Glue JDBC with `useConnectionProperties` + `sampleQuery` + `hashfield` parallel reads, watermark validation and injection guards. Its own docstring names the gap:

> *Out of scope … Oracle / MySQL / MariaDB engines — code path is identical; add another branch on the engine literal when needed.*

So: add `"oracle"` to `_SUPPORTED_ENGINES` (today `{"postgresql", "sqlserver"}`), confirm Glue 5.0's `connection_type` literal, and you have RMS, SIM and XStore. **Read `rds_jdbc.py` as the non-SAP template — not `sap_hana.py`**, which carries MANDT, `ngdbc.jar` and `NVARCHAR YYYYMMDD` dates that do not exist at Apparel Group.

> **Gotcha:** a genuinely *new* `source_type` string must also be added to the Literal in `glue_engine/spec.py` — today `Literal["sap_hana", "rds_jdbc", "excel_landing", "s3_landing"]`. Widening `rds_jdbc` with an engine branch avoids that; a sibling `oracle_jdbc` class does not. Choose deliberately.

Step 2 — the `download` spec (P1: source → S3 raw)

Template: `specs/download/sap_mara.yaml`.

```yaml
# src/glue/specs/download/rms_item_master.yaml
table: rms.item_master
source_type: rds_jdbc

source_config:
  cadence: daily
  jdbc_schema: RMS
  jdbc_table: ITEM_MASTER
  watermark_column: LAST_UPDATE_DATETIME   # monotonic, in the projection
  jdbc_hash_field: ITEM                    # PK member, high cardinality
  jdbc_hash_partitions: "8"                # parallelism is mandatory, not optional

landing_prefix: "rms/rms.item_master"      # RELATIVE to the raw bucket root
format: parquet

schema:
  - { name: ITEM,      type: string, pii_class: internal, description: "Item number (PK)" }
  - { name: ITEM_DESC, type: string, pii_class: internal }
  - { name: DEPT,      type: string, pii_class: internal }
  - { name: CLASS,     type: string, pii_class: internal }
  - { name: SUBCLASS,  type: string, pii_class: internal }
  - { name: LAST_UPDATE_DATETIME, type: string, pii_class: internal, description: "WATERMARK" }

partition_by: []
```

Three things this file decides, and each has bitten someone:

- **`watermark_column` must appear in the projection.** `read_incremental` computes the new watermark via `F.max(<watermark_column>)`; if it's absent, Spark raises *after* you've paid for the whole JDBC pull. `configure()` now fails fast instead.
- **Parallelism has no default.** Exactly one of `jdbc_hash_field` / `jdbc_hash_expression`, never both. Without it, `sampleQuery` runs on one executor and OOMs on anything real.
- **Curate the projection.** MARA has 317 columns; the spec declares the 16 that are consumed. `SELECT *` is banned so a source-schema drift can't silently leak untyped columns into Bronze.

Step 3 — the `bronze` spec (P2: raw → Iceberg)

Template: `specs/bronze/sap_mara.yaml`.

L24 · Add a Source Without Touching the Engine — the capstone bridge

```
# src/glue/specs/bronze/rms_item_master.yaml
table: rms.item_master
source_type: s3_landing          # reads back what P1 landed — no source creds, no DB hit

merge_key: [ITEM]                # the VERIFIED natural PK

schema:                          # MUST mirror the download spec
  - { name: ITEM, type: string, pii_class: internal, description: "Item number (PK)" }
  # ... identical to download/rms_item_master.yaml

source_config:
  landing_prefix: "rms/rms.item_master"   # the same relative prefix P1 landed under
  format: parquet
```

`merge_key` is the single highest-stakes line you will write. It is the Bronze zero-duplicates invariant: replays, overlapping windows and Glue auto-retries must land each row exactly once. `spec.py` enforces that every member exists in the schema, that there are no duplicates, and that no member is `pii_class: pii`. What it *cannot* enforce is that it is the real primary key — **verify it against the source's constraint catalog**, the way the SAP specs did (`SYS.CONSTRAINTS`, 2026-07-14). Get it wrong and you get `MERGE_CARDINALITY_VIOLATION` at best, silent row loss at worst.

Step 4 — the catalog row

One entry in `config/ingestion_tables.yaml` — *"the ONE place the technical team edits to add / modify / remove a table"* — plus one connection in `config/connections.yaml` per **source** (not per table).

```
  - name: rms.item_master
    source: rms                  # -> connections.yaml entry (carries source_type + secrets_arn)
    source_object: ITEM_MASTER
    bronze_spec:   bronze/rms_item_master.yaml
    download_spec: download/rms_item_master.yaml
    driver_by_mode: { full: rds_jdbc, range: rds_jdbc, incremental: rds_jdbc }
    initial_load: once
    cadence_band: daily
    enabled: true
```

- `driver_by_mode` is the seam that lets one table use different connectors for different read modes (that's how SAP was designed to do JDBC-init → OData-CDC before OData was retired). For Oracle, all three modes are the same driver — but declaring it keeps the option open.
- `initial_load: once` engages the init-once guard: the table gets one bulk historical load, `init_state` becomes `initial_loaded`, and only then does Gate 0 allow daily deltas.
- `enabled: false` is how you pause a table without deleting it. Use it while you're iterating — the dispatcher only scans enabled rows.

Step 5 — dbt staging + tests

```
-- src/dbt/models/staging/stg_rms_item_master.sql
{{ config(materialized='view') }}

SELECT
    item                 AS item_id,
    item_desc            AS item_name,
    dept, class, subclass,
    last_update_datetime  AS updated_at
FROM {{ source('silver', 'rms_item_master') }}
```

```
# src/dbt/models/staging/_staging.yml
  - name: stg_rms_item_master
    description: "1:1 mirror of silver.rms.item_master with snake_case column names."
    columns:
      - name: item_id
        tests: [not_null, unique]
```

L24 · Add a Source Without Touching the Engine — the capstone bridge

Staging is a **view**, and — because it reads an external schema — the whole staging layer is declared `+bind: false` (late-binding) in `dbt_project.yml`. A plain Redshift view cannot `SELECT` from an external schema. The `not_null` + `unique` pair on the natural key is the cheapest correctness insurance in the codebase; add it before you add anything clever.

Step 6 — the Gold model (only if something reports on it)

If RMS item master is a *dimension* feeding a fact that already exists, this step is a join, not a new model. If it's the first table of a new subject area, you write the Gold model and its tests — and Module 2's L14 correctness traps (grain discipline, fan-out joins, synthetic "all" rows) are the checklist. **Do not invent a Gold model to prove the pipeline works.** `runs` + row counts already prove that.

Step 7 — deploy

CI does exactly three things that matter here, in this order:

1. `aws s3 sync src/glue/specs/ s3://$ART_BUCKET/specs/` — Glue jobs read the YAML **from S3 at runtime**, so a spec that isn't synced doesn't exist.
2. `aws s3 sync src/dbt/ s3://$ART_BUCKET/dbt/project/`
3. `python scripts/seed_control_plane.py --env <env> --specs-from s3 --from-config --prune --commit` — reconciles `connections.yaml` → `source_catalog` and `ingestion_tables.yaml` → `bronze_mapping`, hashing the *same spec bytes* the artifact step uploaded into `spec_hash`.

Run it `--dry-run` first. It runs **after** both the artifact upload and the Terraform apply, because the seed hashes uploaded specs and writes rows for resources that must already exist.

Then smoke-test in Dev: invoke the dispatcher, watch `runs` go green at `source_download` → `bronze_pull` → `bronze_to_silver`, confirm the watermark advanced, confirm the cycle reaches `gold_built`. That sequence — not a passing unit test — is "done".

What you never touch

`glue_engine/jobs/` · `writers/` · `dispatcher` · the four barriers · `cycle_sweeper` · `run_status` · the control-plane models · Step Functions · the Silver and Gold contracts · the reporting views · Power BI.

If your change requires editing any of those, stop and ask whether you're solving the wrong problem. The one legitimate exception is Step 1's `_SUPPORTED_ENGINES` branch — and even that is four lines in a connector, not the engine.

L24 · Add a Source Without Touching the Engine — the capstone bridge

Words you'll hear

WORD	WHAT IT MEANS HERE
Protocol	the five-method Python `Protocol` every connector satisfies; the engine's only knowledge of a source
`@register("type")`	the class decorator that puts a connector in the registry at import time; duplicate registration raises
P1 / P2	P1 = `source_download` (source → S3 raw); P2 = `bronze_pull` (raw → Iceberg). Split so a re-pull and a re-load are separate failures
download spec	the P1 YAML — how to *pull*: schema, watermark column, hash field, landing prefix
bronze spec	the P2 YAML — how to *land*: `merge_key`, the mirrored schema, the same landing prefix
`merge_key`	the verified natural PK. Iceberg MERGE upserts on it; this is what makes replays idempotent
`driver_by_mode`	per-read-mode connector choice: `{full, range, incremental}`
`landing_prefix`	the RELATIVE raw path, qualified against the raw bucket at runtime
`init_state`	`pending` → `initial_loaded` → `cdc` (or `needs_reinit`). Gate 0 refuses deltas before the initial load
`spec_hash`	the seeder's fingerprint of the spec bytes in S3 — how the control plane knows the deployed spec matches the repo
late-binding view	`+bind: false` — required for any view over an external schema
RMS / SIM / XStore	Oracle Retail merchandising / store inventory / POS — three JDBC sources, one connector

In this repo

- `src/glue/glue_engine/sources/__init__.py` — the registry, `register`, `get_connector`, and the "NO engine changes" promise in the docstring
- `src/glue/glue_engine/sources/protocol.py` — the contract, read it before you write a line
- `src/glue/glue_engine/sources/rds_jdbc.py` — **your template.** `_SUPPORTED_ENGINES`, `_build_sample_query`, `_build_range_query`, the watermark guards
- `src/glue/specs/download/sap_mara.yaml` + `src/glue/specs/bronze/sap_mara.yaml` — the pair, read side by side
- `src/glue/glue_engine/spec.py` — `BronzeSpec`: the `source_type` Literal, the `merge_key` validators, the P1-vs-P2 field rules
- `config/ingestion_tables.yaml` + `config/connections.yaml` — the catalog and the per-source connection
- `scripts/seed_control_plane.py` — `--from-config --specs-from s3 --prune`
- `src/dbt/models/staging/` — `stg_sap_zsdcc.sql` as the shape, `_staging.yml` as the test pattern
- `bitbucket-pipelines.yml` — the spec sync + seed steps, per environment
- `docs/CONNECTOR-ACTIVATION-GUIDE.md` — the activation checklist (written pre-`sap_odata` retirement; the *shape* is right, the SAP OData sections are historical)

L24 · Add a Source Without Touching the Engine — the capstone bridge

Do this

1. **Write the two YAMLs for Oracle Retail `ITEM_MASTER` by hand**, using the MARA pair as the template. Then validate: `python -c "import yaml; from glue_engine.spec import BronzeSpec; BronzeSpec.model_validate(yaml.safe_load(open('specs/bronze/rms_item_master.yaml')))"`.
2. Open `rds_jdbc.py` and list every line that would change to support Oracle. It should be a short list. If yours is long, you're writing a new connector when you needed a branch.
3. For each of the eight Apparel Group sources, fill one row: **watermark column · natural key · does step 1 apply?** Epsilon and MoEngage will not fit the JDBC shape — say so, and say what the connector would need instead (paging, tokens, rate limits, and raw-first landing because you often cannot re-request an old page).
4. Trace one spec change from `git commit` to a running Glue job. Name every hop: pipeline → S3 specs → seed → `bronze_mapping.spec_hash` → dispatcher → SFN → Glue → spec read from S3.
5. Deliberately break it: set `merge_key` to a non-unique column and predict the exact error before you run it.

You've got it when you can…

- List the seven artifacts in order, from memory, and say which two are conditional.
- Say why adding Oracle Retail is a **branch in one connector**, not a new pipeline — and cite the docstring that says so.
- Explain `merge_key` to a room in one sentence, and name what you must verify before choosing it.
- Explain why the `download` spec and the `bronze` spec both exist, and what breaks if their schemas drift apart.
- Point at every file that must change, and every file that must **not**.
- Explain why a spec that isn't in S3 doesn't exist as far as the pipeline is concerned.
- Say the sentence that ends this module: **this is your capstone — one source, one PR.**